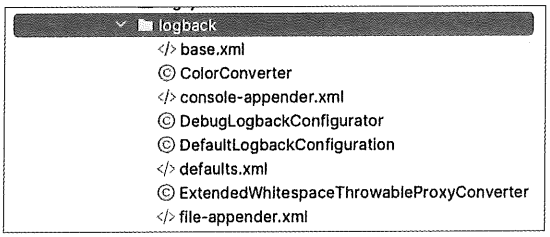
- defaults.xml。
- console-appender.xml。
- file-appender.xml。

此外，为了与早期的 Spring Boot 版本兼容，Spring Boot 在 Logback 包中还提供了一个遗留的 base.xml 模板文件，这些模板都定义在 org/springframework/boot/logging/logback/ 目录下，如下图所示。



它们都可以包含在自定义日志配置文件中，这些模板中的内容如下所示。

defaults.xml：

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
Default logback configuration provided for import
-->

<included>
    <conversionRule conversionWord="clr" converterClass="org.springframework.boot.
logging.logback.ColorConverter" />
    <conversionRule conversionWord="wex" converterClass="org.springframework.boot.
logging.logback.WhitespaceThrowableProxyConverter" />
    <conversionRule conversionWord="wEx" converterClass="org.springframework.boot.
logging.logback.ExtendedWhitespaceThrowableProxyConverter" />

    <property name="CONSOLE_LOG_PATTERN" value="${CONSOLE_LOG_PATTERN:-%clr
(%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint} %clr
(${LOG_LEVEL_PATTERN:-%5p}) %clr(${PID:- }){magenta} %clr(---){faint} %clr([%15.15t])
{faint} %clr(%-40.40logger{39}){cyan} %clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_
WORD:-%wEx}}"/>
    <property name="CONSOLE_LOG_CHARSET" value="${CONSOLE_LOG_CHARSET:-${file.encoding:
-UTF-8}}"/>
    <property name="FILE_LOG_PATTERN" value="${FILE_LOG_PATTERN:-%d{${LOG_DATEFORMAT_
PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}} ${LOG_LEVEL_PATTERN:-%5p} ${PID:- } --- [%t]
%-40.40logger{39} : %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
```

```
    <property name="FILE_LOG_CHARSET" value="${FILE_LOG_CHARSET:-${file.encoding:-UTF-8}}"/>

    <logger name="org.apache.catalina.startup.DigesterFactory" level="ERROR"/>
    <logger name="org.apache.catalina.util.LifecycleBase" level="ERROR"/>
    <logger name="org.apache.coyote.http11.Http11NioProtocol" level="WARN"/>
    <logger name="org.apache.sshd.common.util.SecurityUtils" level="WARN"/>
    <logger name="org.apache.tomcat.util.net.NioSelectorPool" level="WARN"/>
    <logger name="org.eclipse.jetty.util.component.AbstractLifeCycle" level="ERROR"/>
    <logger name="org.hibernate.validator.internal.util.Version" level="WARN"/>
    <logger name="org.springframework.boot.actuate.endpoint.jmx" level="WARN"/>
</included>
```

可以看到，defaults.xml 配置模板中提供了一些转换规则、模式参数和常用的包的日志级别配置。

console-appender.xml：

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
Console appender logback configuration provided for import, equivalent to the programmatic
initialization performed by Boot
-->

<included>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>${CONSOLE_LOG_PATTERN}</pattern>
            <charset>${CONSOLE_LOG_CHARSET}</charset>
        </encoder>
    </appender>
</included>
```

可以看到，console-appender.xml 配置模板使用 CONSOLE_LOG_PATTERN 参数添加了一个 ConsoleAppender 配置，这意味着如果要在控制台输出日志，那么只需要在自己的日志配置文件中引入该配置模板即可，反之如果要禁用控制台输出日志，则不包含此配置模板即可。

file-appender.xml：

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
File appender logback configuration provided for import, equivalent to the programmatic
initialization performed by Boot
```

```
-->

<included>
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <encoder>
            <pattern>${FILE_LOG_PATTERN}</pattern>
            <charset>${FILE_LOG_CHARSET}</charset>
        </encoder>
        <file>${LOG_FILE}</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
            <fileNamePattern>${LOGBACK_ROLLINGPOLICY_FILE_NAME_PATTERN: -${LOG_FILE}.%d
{yyyy-MM-dd}.%i.gz}</fileNamePattern>
            <cleanHistoryOnStart>${LOGBACK_ROLLINGPOLICY_CLEAN_HISTORY_ON_START: -false}
</cleanHistoryOnStart>
            <maxFileSize>${LOGBACK_ROLLINGPOLICY_MAX_FILE_SIZE:-10MB}</maxFileSize>
            <totalSizeCap>${LOGBACK_ROLLINGPOLICY_TOTAL_SIZE_CAP:-0}</totalSizeCap>
            <maxHistory>${LOGBACK_ROLLINGPOLICY_MAX_HISTORY:-7}</maxHistory>
        </rollingPolicy>
    </appender>
</included>
```

可以看到，file-appender.xml 配置模板使用 FILE_LOG_PATTERN 和 ROLLING_FILE_ NAME_ PATTERN 参数添加了一个 RollingFileAppender 配置，这意味着如果要在日志文件中输出日志并进行归档，则只需要在自己的日志配置文件中引入该配置模板即可，反之亦然。

以上三个内置的日志配置模板能满足大部分的应用需求，直接拿来用即可，如果不合适，就不需要引入，再参考自定义的配置即可，这也是 Spring Boot 为了简化配置所做出的努力。

5.8.3 自定义日志配置文件

了解了上面几节的内容，本节就可以自定义一个 Logback 日志配置文件。首先在应用的 src/main/resources/ 资源目录下创建一个 logback-spring.xml 日志配置文件，参考配置如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
    <include resource="org/springframework/boot/logging/logback/console-appender.xml" />

    <property name="LOG_FILE" value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP:
-${java.io.tmpdir:-/tmp}}}/javastack.log}"/>
    <include resource="org/springframework/boot/logging/logback/file-appender.xml" />
```

```
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
        <appender-ref ref="FILE" />
    </root>

    <springProfile name="dev | test">
        <logger name="cn.javastack" level="DEBUG"/>
    </springProfile>

    <springProfile name="prod">
        <logger name="cn.javastack" level="INFO"/>
    </springProfile>

    <logger name="org.springframework.web" level="INFO"/>
</configuration>
```

通过 include 标签引用内置的模板配置文件，通过 springProfile 扩展标签可以指定在 profiles 环境下才生效的日志配置。

如果想自定义日志配置参数，那么从模板内容可以看到是有 ${} 参数占位符的，其中“:-”表示如果没有此参数的值就继续再取后面参数的值，以此类推，如果都没有，那么会使用最后配置的临时目录，所以可以通过添加对应的系统参数来使用模板。

以下是相关测试代码：

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        System.setProperty("LOG_PATH", "./logs");
        System.setProperty("LOGBACK_ROLLINGPOLICY_MAX_FILE_SIZE", "1KB");
        SpringApplication.run(Application.class);
    }

    private static final org.apache.commons.logging.Log logger1 = org.apache.commons.logging
            .LogFactory
            .getLog(Application.class);

    private static final org.slf4j.Logger logger2 = org.slf4j.LoggerFactory
            .getLogger(Application.class);

    @Bean
    public CommandLineRunner commandLineRunner() {
```

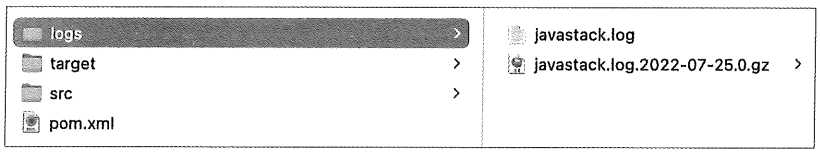
```
        return (args) -> {
            logger1.error("commons logging error...");
            logger1.info("commons logging info...");
            logger1.debug("commons logging debug...");

            logger2.error("slf4j info...");
            logger2.info("slf4j info...");
            logger2.debug("slf4j info...");
        };
    }

}
```

这里定义了一个日志文件目录，为了测试，这里还将日志文件归档的最大文件值设置为 1KB，并添加了对应的日志门面 DEBUG 日志输出功能。

然后指定当前 profiles 为 dev 环境：

```
spring:
  profiles:
active: dev
```

注释 application 中的日志配置，删除之前的 logs 目录再重启应用，如下图所示。



可以看到，目录和日志文件正常生成和归档了，并且当 dev 环境开启时，Debug 日志正常输出了：

```
commons logging error...
commons logging info...
commons logging debug...
slf4j error...
slf4j info...
slf4j debug...
```

如果切换为 prod 环境，则 Debug 日志是不会被输出的，这就是使用 -spring 格式的日志配置文件的好处。

5.9 切换 Log4j2 日志框架

在本章最开始的时候提到了 Spring Boot 支持的日志框架，其中 spring-boot-starter 包中默认使用的是 spring-boot-starter-logging 启动器，对应的是 Logback 日志框架，如果要使用 Log4j2 就必须先排除默认的日志框架，再引入新的 spring-boot-starter-log4j2 日志框架启动器。

以下是推荐的切换方法：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

然后创建对应的 log4j2-spring.xml 日志配置文件即可。

Spring Boot 3.0 新变化：

Log4j2 的功能增强了，和 Logback 中的扩展功能一样，支持以下几个 Log4j2 的新扩展功能：

- 支持使用基于 Profile 生效的配置。
- 支持使用 Spring 环境中的参数。
- 支持使用系统参数。

5.10 切换日志框架版本

日志框架经常出现漏洞需要升级到最新版本，而 Spring Boot 中的日志框架版本都是固定跟随 Spring Boot 版本升级的，如果默认集成的日志框架版本有问题，那么在 Spring Boot 最新版本发布

之前可以自行切换使用更新的日志框架版本。

前面的基础章节中介绍了，如果使用的是继承 spring-boot-starter-parent 的方式，那么直接使用对应的 properties 参数覆盖版本号即可：

```
<log4j2.version>2.18.0</log4j2.version>
```

如果使用导入 spring-boot-dependencies 依赖的集成方式，则需要显式导入依赖并指定版本号：

```
<dependencies>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-bom</artifactId>
        <version>2.18.0</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
 </dependencies>
```

这里只是日志框架依赖的升级示例，更多 Spring Boot 依赖的更新替换都可以参考此种方式。如果不是太大的版本升级，一般情况下可以直接升级具体的依赖，但不排除会与 Spring Boot 当前版本存在兼容性问题，所以也得谨慎升级。

5.11 输出彩色日志

5.11.1 开启彩色日志输出

如果你的终端支持 ANSI 字符标准，则可以设置控制台输出彩色日志来提高可读性。各个日志级别对应的颜色如下表所示。

日志级别	颜色
FATAL	红
ERROR	红
WARN	黄
INFO	绿
DEBUG	绿
TRACE	绿

其实也就是根据日志级别的严重性分为红、黄、绿三种颜色，开启配置的参数为 spring.output.ansi.enabled，可支持的值如下表所示。

选项	说明
ALWAYS	开启 ANSI 颜色输出
DETECT（默认）	尝试检测 ANSI 功能是否可用
NEVER	禁用 ANSI 颜色输出

最新选项可参考 AnsiOutput.Enabled 枚举类，如该枚举类的源码所示。

```
public enum Enabled {

    /**
     * Try to detect whether ANSI coloring capabilities are available. The default
     * value for {@link AnsiOutput}.
     */
    DETECT,

    /**
     * Enable ANSI-colored output.
     */
    ALWAYS,

    /**
     * Disable ANSI-colored output.
     */
    NEVER

}
```

默认值为 DETECT，如 AnsiOutput 类的源码所示。

```
public abstract class AnsiOutput {

    private static final String ENCODE_JOIN = ";";

    private static Enabled enabled = Enabled.DETECT;

    ...

}
```

DETECT 即只进行检测，并不会输出彩色日志，如下图的启动日志所示。

```
416+08:00  INFO 84579 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http)
425+08:00  INFO 84579 --- [  restartedMain] c.j.springboot.logging.Application       : Started Application in 2.705 seconds (
427+08:00 ERROR 84579 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging error...
427+08:00  WARN 84579 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
427+08:00  INFO 84579 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging info...
428+08:00 DEBUG 84579 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging debug...
428+08:00 ERROR 84579 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j error...
428+08:00  WARN 84579 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
428+08:00  INFO 84579 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j info...
429+08:00 DEBUG 84579 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j debug...
```

笔者这里把日志设置为开启颜色输出，如以下配置所示。

```
spring:
  output:
    ansi:
      enabled: always
```

然后重启应用，查看日志，如下图所示。

```
634+08:00  INFO 83883 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http)
647+08:00  INFO 83883 --- [  restartedMain] c.j.springboot.logging.Application       : Started Application in 2.734 seconds (
652+08:00 ERROR 83883 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging error...
653+08:00  WARN 83883 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
653+08:00  INFO 83883 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging info...
655+08:00 DEBUG 83883 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging debug...
655+08:00 ERROR 83883 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j error...
655+08:00  WARN 83883 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
655+08:00  INFO 83883 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j info...
655+08:00 DEBUG 83883 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j debug...
```

可以看到日志带颜色输出了（彩色图片下载地址见封底），除了日志级别有颜色，其他关键的信息也有默认的颜色，具体配置可见下一节的介绍。

5.11.2 日志上色原理

Spring Boot 中的日志颜色是使用 %clr 转换词来配置的，配置格式如下：

%clr(日志){ 颜色 }

可支持配置的颜色如下表所示。

选项	颜色
blue	蓝
cyan	青
faint	模糊
green	绿

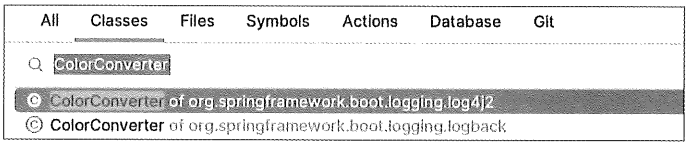
续表

选项	颜色
magenta	洋红
red	红
yellow	黄

下面介绍 Spring Boot 默认的控制台配置示例，内置的 defaults.xml 日志模板如下所示。

```
<included>
  <conversionRule conversionWord="clr" converterClass="org.springframework.boot.
logging.logback.ColorConverter" />
  <conversionRule conversionWord="wex" converterClass="org.springframework.boot.
logging.logback.WhitespaceThrowableProxyConverter" />
  <conversionRule conversionWord="wEx" converterClass="org.springframework.boot.
logging.logback.ExtendedWhitespaceThrowableProxyConverter" />

  <property name="CONSOLE_LOG_PATTERN" value="${CONSOLE_LOG_PATTERN:-%clr(%d {${LOG_
DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint} %clr(${LOG_LEVEL_PATTERN:
-%5p}) %clr(${PID:- }){magenta} %clr(---){faint} %clr([%15.15t]){faint} %clr(%-40.40logger
{39}){cyan} %clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>

  ...
```

这便是上一节中默认的彩色日志配置，定义了 clr、wex、wEx 三个转换词，关于颜色的转换便是 clr 这个转换词，对应的转换类为 ColorConverter 类，从源码看目前只有 Logback 和 Log4j2 实现了颜色日志，如下图所示。



具体上色的过程很简单，以下是 Logback 日志框架对应的 ColorConverter 类的核心源码：

```
public class ColorConverter extends CompositeConverter<ILoggingEvent> {

    ...

    static {
        Map<String, AnsiElement> ansiElements = new HashMap<>();
        ansiElements.put("faint", AnsiStyle.FAINT);
        ansiElements.put("red", AnsiColor.RED);
```

```
        ansiElements.put("green", AnsiColor.GREEN);
        ansiElements.put("yellow", AnsiColor.YELLOW);
        ansiElements.put("blue", AnsiColor.BLUE);
        ansiElements.put("magenta", AnsiColor.MAGENTA);
        ansiElements.put("cyan", AnsiColor.CYAN);
        ELEMENTS = Collections.unmodifiableMap(ansiElements);
    }

    @Override
    protected String transform(ILoggingEvent event, String in) {
        AnsiElement element = ELEMENTS.get(getFirstOption());
        if (element == null) {
            // Assume highlighting
            element = LEVELS.get(event.getLevel().toInteger());
            element = (element != null) ? element : AnsiColor.GREEN;
        }
        return toAnsiString(in, element);
    }

    protected String toAnsiString(String in, AnsiElement element) {
        return AnsiOutput.toString(element, in);
    }

}
```

实现很简单，在 DefaultLogbackConfiguration 配置类中注册了这个颜色转换器。

5.11.3 自定义日志颜色

了解了日志上色的原理和配置方法后，下面介绍一个简单的自定义颜色的示例。在应用的自定义日志配置文件中，修改日志的具体内容为黄色，如下面的配置示例所示。

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <property name="LOG_FILE" value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP: -${java.io.tmpdir:
-/tmp}}}/javastack.log}"/>
    <property name="CONSOLE_LOG_PATTERN" value="${CONSOLE_LOG_PATTERN:-%clr (%d{${LOG_
DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint} %clr(${LOG_LEVEL_PATTERN:
-%5p}) %clr(${PID:- }){magenta} %clr(---){faint} %clr([%15.15t]){faint} %clr
(%-40.40logger{39}){cyan} %clr(:){faint} %clr(%m%n){yellow}${LOG_EXCEPTION_CONVERSION_
WORD:-%wEx}}"/>
```

```
    ...
}
```

笔者新增了一个 CONSOLE_LOG_PATTERN 参数，这样就可以覆盖框架内置的默认的参数值，然后将 %m%n 修改为 %clr(%m%n){yellow}，再重启应用并查看日志，如下图所示。

```
229+08:00  INFO 89504 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http)
237+08:00  INFO 89504 --- [  restartedMain] c.j.springboot.logging.Application       : Started Application in 1.957 seconds (
239+08:00 ERROR 89504 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging error...
239+08:00  WARN 89504 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
239+08:00  INFO 89504 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging info...
240+08:00 DEBUG 89504 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging debug...
240+08:00 ERROR 89504 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j error...
240+08:00  WARN 89504 --- [  restartedMain] c.j.springboot.logging.Application       : commons logging warn
240+08:00  INFO 89504 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j info...
240+08:00 DEBUG 89504 --- [  restartedMain] c.j.springboot.logging.Application       : slf4j debug...
```

日志具体内容已经成功显示为黄色了，如果想定制自己的控制台日志颜色，那么“依葫芦画瓢”自定义日志配置即可。

5.12 日志关闭钩子

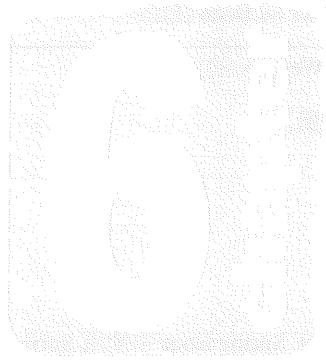
在 Spring Boot 启动过程与扩展应用一章中，我们知道 Spring Boot 提供了一个优雅关闭的钩子，在日志系统中，Spring Boot 也注册了日志相关的关闭钩子，该钩子可以在 JVM 退出时触发日志系统清理，以保证应用终止时及时释放日志使用的资源。

Spring Boot 应用会自动注册该关闭钩子，以 war 包形式部署的除外。

如果应用具有复杂的上下文层次结构，而 Spring Boot 的日志关闭钩子无法满足需求，则可以禁用这个日志关闭钩子，关闭配置如下所示。

```
logging:
  register-shutdown-hook: false
```

禁用注册日志关闭钩子后，可以考虑直接使用底层日志框架提供的相关资源管理机制，例如，Logback 提供了一种上下文选择器，它可以允许每个 Logger 对象在它们自己的上下文中进行创建，无须 Spring Boot 干预。



第6章 Spring Boot Web 核心应用

在 Java 编程应用中，最重要的一点就是 Web 与后端接口编程，使用 Spring Boot 框架会让这一点变得轻而易举，它可以轻松利用 Web 模块快速搭建 HTTP 接口服务，所以本章会介绍 Spring Boot 与 Web 的方方面面，包括 Web 模块基本介绍、嵌入式容器、自定义配置及注册一些 Web 组件、静态资源处理、模板引擎，以及异常处理、参数校验、国际化、分布式会话、跨域处理、安全性、调用 REST 服务等。

6.1 概述

在之前的 Spring Boot 启动过程分析中，笔者介绍了 Spring Boot 支持的两种 Web 类型：

- SERVLET（即传统的 WebMVC）。
- REACTIVE（响应式的 WebFlux）。

传统的 WebMVC 应用使用的是同步阻塞的 I/O 模型，而 WebFlux 响应式应用则使用异步非阻塞的 I/O 模型，所以 WebFlux 的性能要优于 WebMVC，但 WebFlux 还不是很成熟，生态也不是很完善，现在绝大多数的 Web 应用还是基于 Servlet 规范和容器的，所以，本章绝大多数应用也都是基于 Servlet 的。

通过集成 Spring Boot 提供的 spring-boot-starter-web 启动器可以快速启用一个基于 Servlet 的 HTTP 服务，在 Spring Boot 自动配置包中也提供了对 Spring MVC 框架的自动配置，具体可以查看 WebMvcAutoConfiguration 类的源码，源码较多，笔者在此不粘贴详细的代码，自动配置类的主要内容是：

- 包含对 ContentNegotiatingViewResolver 和 BeanNameViewResolver 的 Bean 注册。
- 支持对静态资源，包括对静态网页 index.html 的支持，以及对 WebJars 的支持。
- 包含对 Converter、GenericConverter、Formatter 的 Bean 自动注册。
- 支持 HttpMessageConverters 消息转换器。
- 支持对 MessageCodesResolver 的自动注册。
- 自动使用 ConfigurableWebBindingInitializer 的 Bean。

既然 Spring Boot 只是提供了对 Spring MVC 的自动配置，所以我们在 Spring Boot 中使用 Spring MVC 和独立使用没有任何区别，比如先加入基于 Servlet 的 Web 依赖启动器 spring-boot-starter-web，然后定义两个接口，分别返回 JSON、XML 格式的数据，如下面的代码所示。

```
@RestController
public class ResponseBodyController {

    @GetMapping(value = "/user/json/{userId}", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity getJsonUserInfo(@PathVariable("userId") String userId) {
        User user = new User("Java 技术栈 ", 18);
        user.setId(Long.valueOf(userId));
        return new ResponseEntity(user, HttpStatus.OK);
    }

    @GetMapping(value = "/user/xml/{userId}", produces = MediaType.APPLICATION_XML_VALUE)
    public ResponseEntity getXmlUserInfo(@PathVariable("userId") String userId) {
        UserXml user = new UserXml();
        user.setName(" 栈长 ");
        user.setId(userId);

        List<OrderInfo> orderList = new ArrayList<>();
        OrderInfo orderInfo1 = new OrderInfo("123456001", 999, new Date());
        OrderInfo orderInfo2 = new OrderInfo("123456002", 777, new Date());
        OrderInfo orderInfo3 = new OrderInfo("123456003", 666, new Date());
        orderList.add(orderInfo1);
        orderList.add(orderInfo2);
        orderList.add(orderInfo3);
        user.setOrderList(orderList);

        return new ResponseEntity(user, HttpStatus.OK);
    }

}
```

注解都还是 Spring MVC 那套东西，Spring Boot 只是自动完成了框架上的配置而已，使得开发者更容易上手。

6.2 嵌入式容器

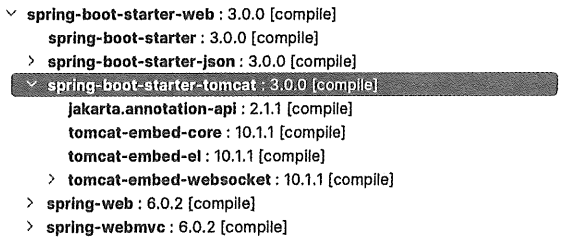
6.2.1 概述

嵌入式容器是 Spring Boot 最重要的特性之一，通过这个特性可以完成 Spring Boot 应用的快速启动和扩容，不需要自己安装和部署 Servlet 容器。

Spring Boot 支持的 Servlet 容器如下表所示。

容器	依赖
Tomcat	spring-boot-starter-tomcat
Jetty	spring-boot-starter-jetty
Undertow	spring-boot-starter-undertow

通过导入 spring-boot-starter-web 启动器依赖，Spring Boot 可以自动配置 Web 应用环境，该启动器导入的依赖如下图所示。



默认的嵌入式容器为 Tomcat，也可以更换为其他两个容器，具体见 6.2.3 节。

6.2.2 容器配置

默认情况下不用做任何参数配置，应用启动绑定的协议与端口默认为 HTTP 与 8080，容器参数绑定类为 ServerProperties 类，可以通过其绑定的以 server.* 开头的参数来配置所有容器的通用配置，比如：

```
server:
```

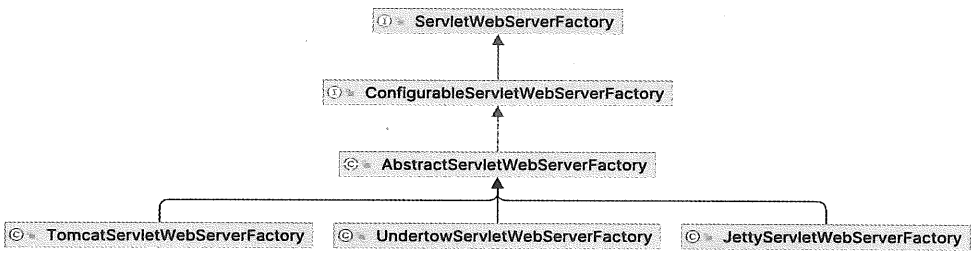
```
port: 8081
servlet:
  context-path: /javastack
```

还有更多容器的详细配置，有需要的读者可以参考 ServerProperties 参数类进行额外配置，比如 Tomcat 容器可以通过 server.tomcat.* 参数配置等。

另外，还能通过 Java 类的方式来自定义 Servlet 容器，只需实现 WebServerFactoryCustomizer< ConfigurableServletWebServerFactory> 接口即可：

```
@Component
public class CustomWebServerFactoryCustomizer implements WebServerFactoryCustomizer
<ConfigurableServletWebServerFactory> {

    @Override
    public void customize(ConfigurableServletWebServerFactory server) {
        server.setPort(8081);
    }

}
```

除了 ConfigurableServletWebServerFactory，还有 TomcatServletWebServerFactory、JettyServlet-WebServerFactory 和 UndertowServletWebServerFactory，ConfigurableServletWebServerFactory 工厂类的结构如下图所示。



可以看到 ConfigurableServletWebServerFactory 工厂类有三个不同的实现变体，它们分别为 Tomcat、Jetty 和 Undertow 容器提供了额外的自定义设置，比如以下示例提供了对 Tomcat 的特定配置：

```
@Component
public class TomcatWebServerFactoryCustomizer implements WebServerFactoryCustomizer
<TomcatServletWebServerFactory> {

    @Override
```

```
    public void customize(TomcatServletWebServerFactory server) {
        connector.setPort(8088);
        server.addConnectorCustomizers((connector) -> connector.setAsyncTimeout
(Duration.ofSeconds(20).toMillis()));
    }

}
```

6.2.3 切换容器

如果不想使用内置默认的 Tomcat 容器，也可以切换为其他支持的容器，只要排除 spring-boot-starter-web 启动器依赖包中的 Tomcat 依赖，再导入其他容器的依赖即可。

比如想使用 Undertow 作为应用的嵌入式容器，则替换方法如下面的示例所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <!-- Exclude the Tomcat dependency -->
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>
```

6.2.4 随机空闲端口

为了防止端口冲突，Spring Boot 可以配置并使用系统随机空闲的端口，只需要把 server.port 设置为 0 即可：

```
server:
  port: 0
```

Spring Boot 会自动扫描系统空闲的端口，并随机选择一个使用。

6.2.5 SSL

Spring Boot 嵌入式容器也可以通过 SSL 配置以支持 HTTPS 访问，通过一系列 server.ssl.* 参数来配置 SSL。

在本地环境下，可以先用 JDK 的 keytool 工具生成一个自签名的证书：

```
> $ keytool -genkey -alias https -keyalg RSA
输入密钥库口令：
再次输入新口令：
您的名字与姓氏是什么？
  [Unknown]:  test
您的组织单位名称是什么？
  [Unknown]:  test
您的组织名称是什么？
  [Unknown]:  test
您所在的城市或区域名称是什么？
  [Unknown]:  test
您所在的省 / 市 / 自治区名称是什么？
  [Unknown]:  test
该单位的双字母国家 / 地区代码是什么？
  [Unknown]:  test
CN=john, OU=test, O=test, L=test, ST=test, C=test 是否正确？
  [否]:  y

输入 <https> 的密钥口令
    （如果和密钥库口令相同，按回车键）:
再次输入新口令：

Warning:
JKS 密钥库使用专用格式。建议使用 "keytool -importkeystore -srckeystore .keystore -destkeystore
.keystore -deststoretype pkcs12" 迁移到行业标准格式 PKCS12。
> $ keytool -importkeystore -srckeystore .keystore -destkeystore .keystore -deststoretype
pkcs12
输入源密钥库口令：
已成功导入别名 https 的条目。
已完成导入命令：1 个条目成功导入，0 个条目失败或取消
```

然后把证书复制到应用资源根目录下，比如以下参考配置：

```
server:
  port: 8443
  ssl:
```

```
protocol: TLS
key-store-type: JKS
key-store: classpath:.keystore
key-store-password: javastack
```

上面的例子配置后就能支持 HTTPS 了，默认的 HTTP 就不再支持了，Spring Boot 不能以配置文件的方式同时支持 HTTP 和 HTTPS。

如果需要同时支持这两个协议，就需要把另外一个协议用 Java 配置类以程序化的方式来配置，因为通过程序的方式配置 HTTP 协议更加简单一点，所以，Spring Boot 推荐的做法是把 HTTPS 配置在配置文件中，HTTP 通过程序来配置。

通过程序的方式来额外支持 HTTP 的示例如下：

```
@SpringBootApplication
public class Application {

    @Bean
    public ServletWebServerFactory servletContainer() {
        TomcatServletWebServerFactory tomcat = new TomcatServletWebServerFactory();
        tomcat.addAdditionalTomcatConnectors(createStandardConnector());
        return tomcat;
    }

    private Connector createStandardConnector() {
        Connector connector = new Connector("org.apache.coyote.http11.Http11NioProtocol");
        connector.setPort(8080);
        return connector;
    }

    ...

}
```

启动 Spring Boot 之后就会看到同时支持两个协议的日志：

```
Tomcat started on port(s): 8443 (https) 8080 (http) with context path '/'
```

现在可以同时使用 HTTPS 和 HTTP 来访问应用了，如下图所示。

虽然 Spring Boot 支持 HTTPS 是如此简单，但实际应用中却很少使用，因为实际项目肯定要考虑负载均衡、高可用性，不可能由开发人员在每个应用中配置协议，所以 HTTPS 一般由运维人员在 Nginx 负载均衡组件上统一配置，方便运维人员统一管理。

另一方面，Spring Boot 应用一般不会直接对外开放，一般是内网部署的，都需要负载均衡组件或者服务网关进行路由转发，所以 HTTPS 会配置在外网负载均衡的组件上。

6.2.6 持久化

嵌入式容器也是支持会话持久化配置的，比如：

```
server:
  port: 8080
  servlet:
    session:
      # 开启持久化
      persistent: true
      # 持久化目录
      store-dir: /tmp/session-store
      # 追踪模式
      tracking-modes:
        - cookie
        - url
```

更多的配置参数可以查看 org.springframework.boot.web.servlet.server.Session 类。

> **需要注意的是：**
> 持久化需要使用 kill -15 pid 以触发优雅关闭应用，不能使用 kill -9 pid 强制“杀进程”等方式，否则应用无法接收退出信号、无法完成会话持久化。

6.2.7 优雅关闭

Spring Boot 内置的容器都支持优雅关闭，在应用正常退出时可以完成对现有请求的处理并关闭相应的资源，以达到平滑退出的目的。

实现 Spring Boot 的优雅关闭只需要添加以下配置参数：

```
server:
  shutdown: graceful
```

并且还支持一个超时参数设置：

```
spring:
  lifecycle:
timeout-per-shutdown-phase: 20s
```

当优雅关闭 Spring Boot 时，在超时时间限定内将允许完成对现有请求的处理，且停止接收新的请求，具体方式也因使用的 Web 容器而异，比如 Jetty、Reactor Netty、Tomcat 容器将停止接收请求，而 Undertow 会继续接收请求，但会立即响应服务不可用（503）错误。如果超时后请求还没处理完，则 Spring Boot 会强制关闭应用。所以需要注意的是，优雅关闭时一定要考虑当时的业务处理量，所设置的缓冲时间是否能处理完正在处理中的业务。

具体的实现机制可以查看 Spring Boot 启动过程中的 refreshContext 方法的源码：

```
private void refreshContext(ConfigurableApplicationContext context) {
    if (this.registerShutdownHook) {
        shutdownHook.registerApplicationContext(context);
    }
    refresh(context);
}
```

先检查是否注册优雅关闭的钩子，默认是允许的，然后将优雅关闭的钩子注册到 Spring 应用上下文中，最后刷新 Spring 应用上下文。

注册钩子的系列方法的源码：

```
void registerApplicationContext(ConfigurableApplicationContext context) {
    addRuntimeShutdownHookIfNecessary();
    synchronized (SpringApplicationShutdownHook.class) {
        assertNotInProgress();
        context.addApplicationListener(this.contextCloseListener);
        this.contexts.add(context);
    }
}

private void addRuntimeShutdownHookIfNecessary() {
    if (this.shutdownHookAdded.compareAndSet(false, true)) {
```

```
            addRuntimeShutdownHook();
        }
    }

    void addRuntimeShutdownHook() {
        Runtime.getRuntime().addShutdownHook(new Thread(this, "SpringApplicationShutdownHook"));
    }
```

可以看到，Spring Boot 的优雅关闭其实也是调用了 JDK 的 Runtime.getRuntime(). addShutdownHook 方法注册了一个 JVM 关闭的钩子。

> **需要注意的是：**
> 不建议使用 kill -9 pid 强制“杀掉”应用，另外，还可能出现其他异常情况，比如停电、系统崩溃等，一般可以使用 kill -15 pid 的方式以触发应用优雅关闭。

以上介绍了几个常用容器的应用配置，其实还有更多高级的应用，可以根据需要自行配置。

6.3 自定义 Web 配置

如果不想使用 Spring Boot 默认的 Web 配置，那么可以通过实现 Spring MVC 中的 WebMvcConfigurer 接口的方式自定义 Web 配置，比如自定义添加拦截器、格式化器、资源处理器等，WebMvcConfigurer 接口中的所有方法如下图所示。

所以，可以创建一个 @Configuration 配置类，并实现 WebMvcConfigurer 接口，然后实现需要扩展的方法即可，如下所示。

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    ...

}
```

具体内容可见后续的章节。

6.4 注册拦截器

可以通过实现 Spring MVC 中的 HandlerInterceptor 接口创建一个拦截器，笔者这里创建并注入了一个简单的登录拦截器，如下面的示例所示。

```
@Component
public class LoginInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response, Object handler) throws Exception {
        HttpSession session = request.getSession();
        String userSession = (String) session.getAttribute("userSession");

        if (userSession == null) {
            response.sendRedirect("/login");
            return false;
        } else {
            return true;
        }
    }

}
```

先获取对应的 userSession 会话值，如果该会话值存在则通过，返回 true 再到下一个拦截器，否则跳到登录页并返回 false。

前面的章节中介绍了，自定义的配置可以在自定义 Web 配置类中进行注册，如下面的示例所示。

```
@Configuration
@RequiredArgsConstructor
public class WebConfig implements WebMvcConfigurer {

    private final LoginInterceptor loginInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(loginInterceptor)
                .addPathPatterns("/**")
                .excludePathPatterns("/login/**")
                .excludePathPatterns("/static/**");
    }

}
```

先注入该拦截器，再通过 InterceptorRegistry.addInterceptor 方法添加并注册拦截器，还可以对该拦截器配置需要拦截的请求路径，以及要排除的请求路径等。

6.5 注册消息转换器

在 Spring MVC 框架中，使用了一个 HttpMessageConverter 消息转换器接口来转换 HTTP 请求和响应，Spring Boot 默认是“开箱即用”的，例如，JSON 或 XML 数据类型的请求格式可以自动转换为 Java Bean 对象，也可以互转。

Spring Boot 支持以下三种 JSON 库的自动配置：

- Jackson（首选默认）。
- JSON-B。
- Gson。

在 Jackson 的 JacksonHttpMessageConvertersConfiguration 消息转换器的自动配置类中就提供了 JSON、XML 的默认自动配置：

```
@Configuration(proxyBeanMethods = false)
class JacksonHttpMessageConvertersConfiguration {

    @Configuration(proxyBeanMethods = false)
    @ConditionalOnClass(ObjectMapper.class)
```

```
    @ConditionalOnBean(ObjectMapper.class)
    @ConditionalOnProperty(name = HttpMessageConvertersAutoConfiguration.PREFERRED_
            MAPPER_PROPERTY, havingValue = "jackson", matchIfMissing = true)
    static class MappingJackson2HttpMessageConverterConfiguration {

        @Bean
        @ConditionalOnMissingBean(value = MappingJackson2HttpMessageConverter.class,
                ignoredType = {
                        "org.springframework.hateoas.server.mvc.TypeConstrained-
MappingJackson2HttpMessageConverter",
                        "org.springframework.data.rest.webmvc.alps.AlpsJsonHttp-
MessageConverter" })
        MappingJackson2HttpMessageConverter mappingJackson2HttpMessageConverter
(ObjectMapper objectMapper) {
            return new MappingJackson2HttpMessageConverter(objectMapper);
        }

    }

    @Configuration(proxyBeanMethods = false)
    @ConditionalOnClass(XmlMapper.class)
    @ConditionalOnBean(Jackson2ObjectMapperBuilder.class)
    protected static class MappingJackson2XmlHttpMessageConverterConfiguration {

        @Bean
        @ConditionalOnMissingBean
        public MappingJackson2XmlHttpMessageConverter mappingJackson2XmlHttpMessageConverter(
                Jackson2ObjectMapperBuilder builder) {
            return new MappingJackson2XmlHttpMessageConverter(builder.createXmlMapper (true)
.build());
        }

    }

}
```

如果默认的配置不符合要求，则可以在自己的配置文件中进行自定义覆盖，比如下面的示例，覆盖了默认的 JSON 消息转换器：

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Bean
```

```
    public MappingJackson2HttpMessageConverter mappingJackson2HttpMessageConverter() {
        MappingJackson2HttpMessageConverter converter = new MappingJackson2HttpMessage-
Converter();
        ObjectMapper mapper = new ObjectMapper();
        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

        SimpleModule module = new SimpleModule();
        module.addDeserializer(String.class, new StringWithoutSpaceDeserializer (String.class));
        mapper.registerModule(module);

        converter.setObjectMapper(mapper);
        return converter;
    }

}

public class StringWithoutSpaceDeserializer extends StdDeserializer<String> {

    private static final long serialVersionUID = -6972065572263950443L;

    public StringWithoutSpaceDeserializer(Class<String> vc) {
        super(vc);
    }

    @Override
    public String deserialize(JsonParser p, DeserializationContext deserializationContext)
throws IOException {
        return StringUtils.trimToEmpty(p.getText());
    }

}
```

这个自定义的 JSON 消息转换器设置了反序列化过程中遇到不明确的参数时不用失败，并且配置了一个字符串反序列化器，用来过滤 HTTP 请求字符串参数首尾的空格。

如果不是覆盖自动配置已有的消息转换器 Bean，而是需要添加额外的自定义转换器，则需要使用 Spring Boot 的 HttpMessageConverters 类，如下面的示例所示。

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Bean
    public HttpMessageConverters customConverters() {
```

```
        HttpMessageConverter<?> additional = new AdditionalHttpMessageConverter();
        HttpMessageConverter<?> another = new AnotherHttpMessageConverter();
        return new HttpMessageConverters(additional, another);
    }

}
```

HttpMessageConverters类可以允许多个自定义HttpMessageConverter消息转换器的添加和注册。

6.6 注册类型转换器

Spring MVC 中的 Converter 接口可以转换参数的类型，比如笔者这里配置了一个 CustomConverter 转换器，如下面的示例所示。

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addFormatters(FormatterRegistry registry) {
        registry.addConverter(new CustomConverter());
    }

}

@Slf4j
public class CustomConverter implements Converter<String, String> {

    @Override
    public String convert(String source) {
        if (StringUtils.isNotEmpty(source)) {
            source = source.trim();
        }
        return source;
    }

}
```

这个转换器的逻辑很简单，就是去掉请求参数中的首尾空格。

6.7 注册 Servlet、Filter、Listener

6.7.1 Spring Boot 的手动注册

Spring Boot 提供了以下三个注册类：

- ServletRegistrationBean。
- FilterRegistrationBean。
- ServletListenerRegistrationBean。

分别用来注册 Servlet、Filter、Listener。

下面是注册 Servlet 的示例代码，首先创建一个 Servlet：

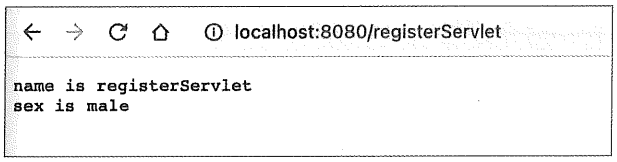
```
public class RegisterServlet extends HttpServlet {

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        String name = getServletConfig().getInitParameter("name");
        String sex = getServletConfig().getInitParameter("sex");

        resp.getOutputStream().println("name is " + name);
        resp.getOutputStream().println("sex is " + sex);
    }

}
```

然后在任何一个 @Configuration 配置类中进行注册，如下面的注册示例所示。

```
@Bean
public ServletRegistrationBean registerServlet() {
    ServletRegistrationBean servletRegistrationBean = new ServletRegistrationBean(
        new RegisterServlet(), "/registerServlet");
    servletRegistrationBean.addInitParameter("name", "javastack");
    servletRegistrationBean.addInitParameter("sex", "male");
    return servletRegistrationBean;
}
```

应用启动后访问该 Servlet，结果如下图所示。

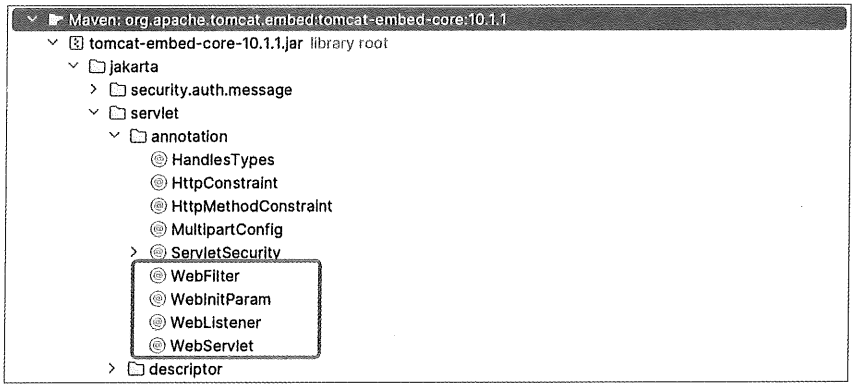
http://localhost:8080/registerServlet



注册 Filter、Listener 也是同样的方法，这里不再赘述。

6.7.2 组件扫描注册

Servlet 3.0 之前，Servlet、Filter、Listener 这些组件都需要在 web.xml 中进行配置，Servlet 3.0 之后不再需要 web.xml 这个配置文件了，所有的组件都可以通过代码配置或者注解来实现相关功能，嵌入式 Tomcat 的核心包如下图所示。



Servlet 3.0 提供了以下 3 个注解来代替相关配置：

- @WebServlet => 代替 servlet 配置。
- @WebFilter => 代替 filter 配置。
- @WebListener => 代替 listener 配置。

以下是配置 Servlet 的示例：

```
@WebServlet(name = "javaServlet", urlPatterns = "/javaServlet", asyncSupported = true,
        initParams = {
                @WebInitParam(name = "name", value = "javaServlet"),
                @WebInitParam(name = "sex", value = "male")})
public class JavaServlet extends HttpServlet {

    @Override
```

```
    protected void service(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
        String name = getServletConfig().getInitParameter("name");
        String sex = getServletConfig().getInitParameter("sex");

        resp.getOutputStream().println("name is " + name);
        resp.getOutputStream().println("sex is " + sex);

    }

}
```

以下是配置 Filter 的示例：

```
@WebFilter(filterName = "javaFilter", urlPatterns = "/*", initParams = {
        @WebInitParam(name = "name", value = "javastack"),
        @WebInitParam(name = "code", value = "123456")})
@Slf4j
public class JavaFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        log.info("java filter init.");
        String name = filterConfig.getInitParameter("name");
        String code = filterConfig.getInitParameter("code");
        log.info("name is " + name);
        log.info("code is " + code);
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        log.info("java filter processing.");
        chain.doFilter(request, response);
    }

    @Override
    public void destroy() {
        log.info("java filter destroy.");
    }

}
```

Listener 的配置方式与之类似，这里不再赘述。

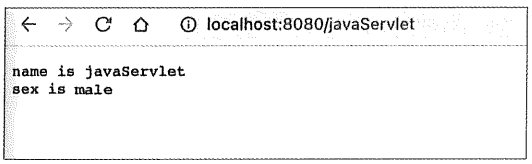
需要注意的是：

为了安全考虑，内嵌服务器不会直接执行 Servlet 3.0 中的 javax.servlet.ServletContainerInitializer 接口，或者 Spring 中的 org.springframework.web.WebApplicationInitializer 接口，否则会终止 Spring Boot 应用。

如果使用的是 Spring Boot 内嵌服务器，则需要在配置类上面添加额外的 @ServletComponentScan 注解来开启 Servlet 组件扫描功能，如果使用的是独立的服务器，则不需要添加，会使用服务器内部的自动发现机制。

现在访问该 Servlet，结果如下图所示。

http://localhost:8080/javaServlet

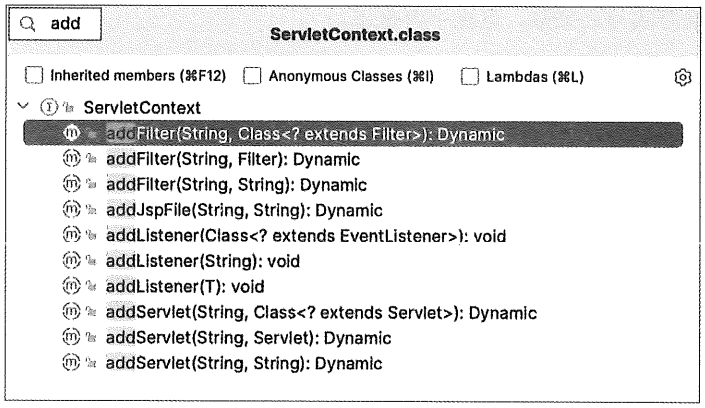


过滤器输出以下日志：

```
java filter init.
name is javastack
code is 123456

...

java filter processing.
```

6.7.3 动态注册

如果想在 Spring Boot 中完成 Servlet、Filter、Listener 的初始化操作，则需要在 Spring 中实现 ServletContextInitializer 接口并注册为一个 Bean，然后通过 ServletContext 提供的几个方法动态注册 Web 组件，如下图所示。



这里创建一个 Servlet：

```
public class InitServlet extends HttpServlet {

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
        String name = getServletConfig().getInitParameter("name");
        String sex = getServletConfig().getInitParameter("sex");

        resp.getOutputStream().println("name is " + name);
        resp.getOutputStream().println("sex is " + sex);

    }

}
```

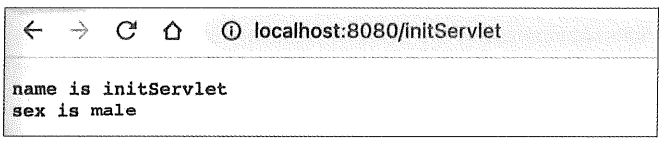
该 Servlet没有在任何地方注册，以下为动态注册该 Servlet 的代码：

```
// 扫描组件的注册方式
@Component
public class ServletConfig implements ServletContextInitializer {

    @Override
    public void onStartup(ServletContext servletContext) {
        ServletRegistration initServlet = servletContext
            .addServlet("initServlet", InitServlet.class);
        initServlet.addMapping("/initServlet");
        initServlet.setInitParameter("name", "initServlet");
        initServlet.setInitParameter("sex", "male");
```

```
    }

}

// Bean 注册
@Bean
public ServletContextInitializer servletContextInitializer() {
    return (servletContext) -> {
        ServletRegistration initServlet = servletContext.addServlet("initServlet",
InitServlet.class);
        initServlet.addMapping("/initServlet");
        initServlet.setInitParameter("name", "initServlet");
        initServlet.setInitParameter("sex", "male");
    };
}
```

以上两种方式是等效的，注册完成后再访问该 Servlet，结果如下图所示。

http://localhost:8080/initServlet



总结一下，注册 Servlet、Filter、Listener 的方式至少有上述三种，读者可以根据需要自行选择，但一般情况下扫描注册无疑是最简洁的。

6.8 静态资源处理

默认情况下，Spring Boot 加载的 classpath 下的几个静态资源目录如下：

- /static。
- /public。
- /resources。
- /META-INF/resources。

如果需要替换这些默认的资源目录位置，则可以通过 spring.web.resources.static-locations 参数指定自定义的资源目录。

Spring Boot 默认把静态资源映射到 /** 访问路径，但也可以使用 spring.mvc.static-path-pattern 参数对其进行调整，比如以下示例：

```
spring:
  mvc:
static-path-pattern: "/pub/**"
```

这里将所有资源访问路径重定位到 /pub/**，比如以前 static 目录下的静态资源是直接用 /** 访问的，现在需要用 /pub/** 进行访问。

另外，Spring MVC 默认使用 ResourceHttpRequestHandler 处理器类来处理静态资源，也可以实现自己的处理器来改变默认的处理器行为，只需要实现 WebMvcConfigurer 并覆盖 addResourceHandlers 方法即可。

来看下面的示例：

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/assets/**").addResourceLocations ("classpath:/
assets/");
        registry.addResourceHandler("/dist/**").addResourceLocations ("classpath:/
dist/");
    }

}
```

这里额外添加了两个 classpath 下静态资源目录的映射。

6.9 模板引擎

Spring Boot 不仅可以作为 HTTP 接口服务使用，还可以用来开发动态网页，Spring Boot 支持以下几个模板引擎的自动配置：

- FreeMarker。
- Thymeleaf。
- Mustache。

- Velocity（Spring Boot 1.5 不再支持）。
- Groovy（Spring Boot 2.7.1 不再支持）。

Spring Boot 早期的版本还支持 Velocity、Groovy，不过后面都废除了。除了以上这些模板引擎的“开箱即用”，Spring Boot 还支持 JSP，但这种古老的技术早已不推荐使用了，如果 JSP 在内嵌容器中运行还会存在很多限制。

现在 Spring Boot 使用比较广泛的是 Thymeleaf 模板引擎，本节以 Thymeleaf 模板引擎为示例进行集成演示。集成模板引擎只需要加入它的 Starter 启动器即可，比如下面添加 Thymeleaf 的 spring-boot-starter-thymeleaf 启动器示例：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Spring Boot 会完成 Thymeleaf 模板引擎的自动配置，根据 Spring Boot 自动配置类的命名规范可以找到它的自动配置类 ThymeleafAutoConfiguration，有需要的话可以自行覆盖里面的配置，也能找到对应的配置参数类 ThymeleafProperties，它的源码如下：

```
@ConfigurationProperties(prefix = "spring.thymeleaf")
public class ThymeleafProperties {

    private static final Charset DEFAULT_ENCODING = StandardCharsets.UTF_8;

    public static final String DEFAULT_PREFIX = "classpath:/templates/";

      public static final String DEFAULT_SUFFIX = ".html";

    ...
```

配置参数的前缀为 spring.thymeleaf，默认的模板引擎目录是 src/main/resources/templates，默认的模板文件后缀为 .html，默认的编码为 UTF-8，还有其他更多的参数，比如是否检查模板、是否开启缓存等，都可以自行配置。

笔者这里在默认模板目录下创建了一个 test.html 模板文件：

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
```

```
    <title>test</title>
    <link rel="stylesheet" type="text/css" th:href="@{/css/test.css}"/>
    <script th:src="@{/js/test.js}"></script>
</head>
<body>

<a id="content" th:utext="${content}"
   class="red" onclick="changeColor()"></a>

</body>
</html>
```

使用 Thymeleaf 模板引擎的语法来引入需要渲染的内容：

（1）在模板文件头部使用 <html lang=" en" xmlns:th=" http://www.thymeleaf.org" > 定义模板文件，使用 th 作为模板语法前缀。

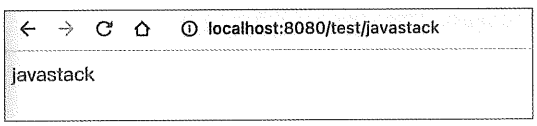
（2）使用 @{} 引入静态资源文件。

（3）使用 ${} 引入 Spring MVC 绑定的请求数据。

然后新建一个 HTTP 请求，返回模板页面路径名称，如下面的示例所示。

```
@GetMapping(value = "/test/{content}")
public String test(@PathVariable("content") String content) {
    request.setAttribute("content", content);
    return "test";
}
```

启动应用并访问 /test 页面，结果如下图所示。



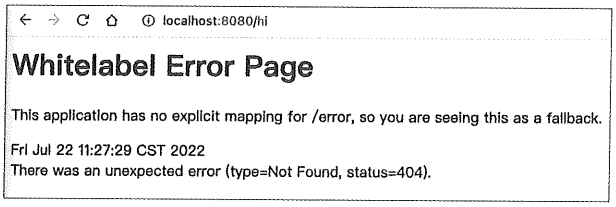
正常获取值，并能通过单击操作来更换颜色，引入的静态资源也都生效了。

6.10 异常处理

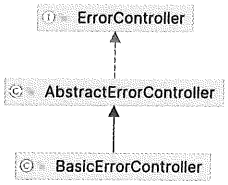
6.10.1 默认的异常处理

Spring Boot 提供了一个默认的 /error 映射处理所有的异常错误，并提供了一个默认的

Whitelabel 错误页面，如下图所示。



对应的错误处理器实现类为 BasicErrorController，它实现了 ErrorController 接口，类结构如下图所示。



该错误处理器可以通过 server.error 系列参数配置一些错误信息，比如更改默认的 /error 错误请求，或者包含更多指定的错误信息，或者禁用默认的错误页面等。

如果要完全替换默认的错误机制，则可以扩展 BasicErrorController 类，还可以实现 ErrorController 接口并注册该类型的 Bean 或者添加 ErrorAttributes 类型的 Bean 以使用现有的机制来替换相关内容。

6.10.2 自定义全局异常

很多时候在项目中不想使用默认的错误机制，需要自定义全局异常，这时一般使用 @ControllerAdvice+@ExceptionHandler 注解实现全局异常处理，比如下面的配置示例：

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ResponseBody
    @ExceptionHandler(value = { Exception.class })
    public ResponseEntity<?> handleException(HttpServletRequest request, Throwable ex) {
        return new ResponseEntity<>("global exception", HttpStatus.OK);
    }

}
```

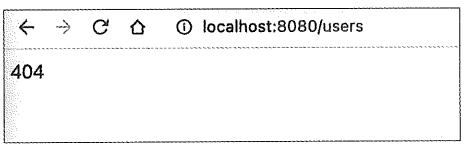
这个 @ExceptionHandler 注解用于指定要处理的异常，value 可以指定多个异常类，在 @ControllerAdvice 类中也可以指定多个 @ExceptionHandler 处理方法。

6.10.3 自定义异常状态码页面

如果想定制自定义错误码的错误页面，则可以在任何一个静态目录下创建 /error 目录，并创建对应的状态码静态模板文件，如下面的文件目录结构所示。

```
src/
 +- main/
    +- resources/
         +- public/
             +- error/
             |   +- 404.html
             |   +- 500.html
```

这里创建了 404.html 和 500.html 两个状态码模板文件，当发生 404、500 错误时会跳转到该目录下的页面。当访问一个不存在的路径时，浏览器返回到自定义的 404 页面，如下图所示。



6.11 参数校验

6.11.1 概述

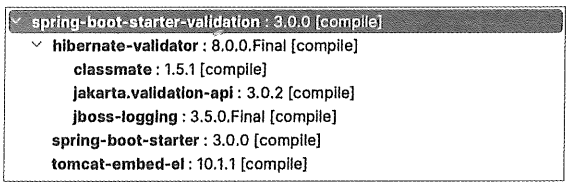
和配置参数校验一样，Spring Boot 同样支持基于 JSR-303 规范对接口参数进行校验，比如常用的 hibernate-validator 就是基于 JSR-303 规范实现的依赖：

```
<dependency>
  <groupId>org.hibernate.validator</groupId>
  <artifactId>hibernate-validator</artifactId>
</dependency>
```

Spring Boot 应用无须手动添加 hibernate-validator 依赖，可以使用导入 spring-boot-starter-validation 启动器的方式：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

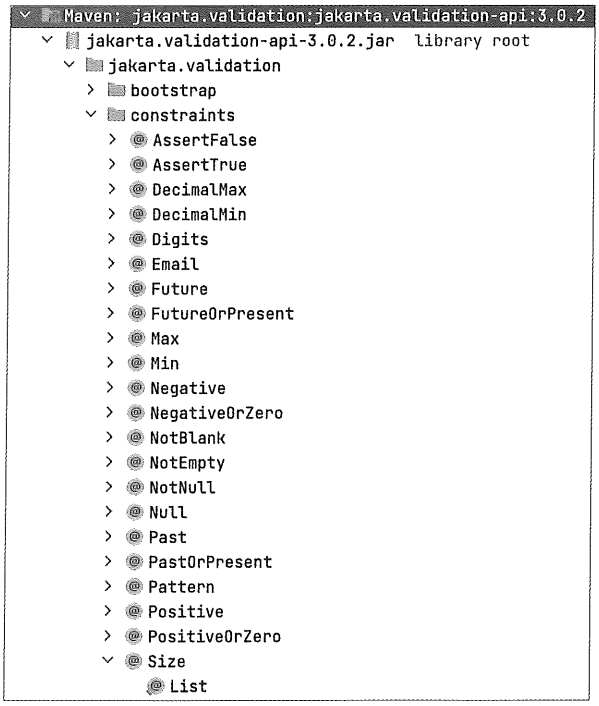
这样做更全面、更专业，它除了会涵盖 hibernate-validator 依赖，还会导入 Spring Boot 参数校验所需的全部依赖组件，如下面的依赖关系图所示。



6.11.2 约束注解

校验接口参数需要用到基于 J2EE 规范的 javax.validation 包下的约束注解，现在包名已经更改为 jakarta.validation，因为早在几年前 Java EE 已经正式更名为 Jakarta，所以，所有相关的名称都变了。

如下图所示，可以看到该包下的全部约束注解。



一看这些约束注解的名称就知道它们的作用，这里不再赘述，它们既可以用在方法参数上，也可以用在参数类成员变量上。

6.11.3 参数校验示例

导入spring-boot-starter-validation启动器之后，可以使用@Valid或者@Validated注解来校验参数，@Valid是Jakarta规范的注解，@Validated是Spring中支持的JSR-303注解，它其实就是@Valid注解的变体。

一般推荐使用Spring中的@Validated注解，功能比@Valid注解要强大，比如它支持分组效验等，常用的做法是：

- 校验接口参数类时，将@Validated注解放在方法参数类之前，然后将约束注解放在类成员变量上面。
- 校验接口单个参数时，将@Validated注解放在当前Controller类上面，然后将约束注解放在方法参数类之前。

比如在参数类上使用各种约束注解，示例如下：

```
@Data
@NoArgsConstructor
public class User {

    public User(String userName, Integer age) {
        this.userName = userName;
        this.age = age;
    }

    private Long id;

    @NotNull
    @Size(min = 5, max = 10)
    private String userName;

    @NotNull
    private Integer age;

    private String address;

    private String memo;


}
```

这里定义了用户名和年龄不能为空，且用户名长度在 5 ~ 10 之间。

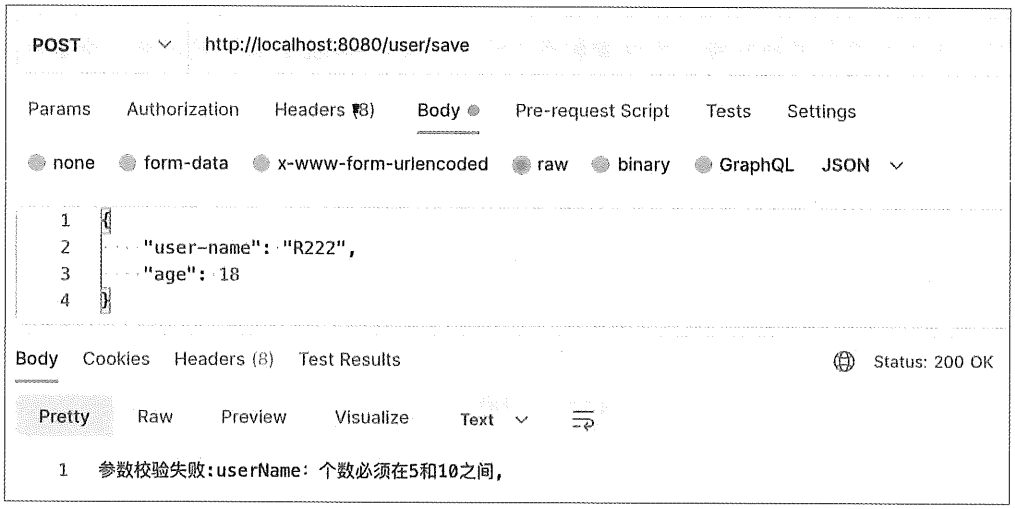
校验该参数类时需要在接口方法参数类之前放置 @Validated 注解，校验单个参数时就在 Controller 类上面放置 @Validated 注解，示例如下：

```
@RestController
@Validated
public class ResponseBodyController {

    @CrossOrigin
    @GetMapping(value = "/user/json/{userId}", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity getJsonUserInfo(@PathVariable("userId") @Size(min = 5, max = 8)
String userId) {
        User user = new User("Java 技术栈 ", 18);
        user.setId(Long.valueOf(userId));
        return new ResponseEntity(user, HttpStatus.OK);
    }

    @PostMapping(value = "/user/save")
    public ResponseEntity saveUser(@RequestBody @Validated User user) {
        user.setId(RandomUtils.nextLong());
        return new ResponseEntity(user, HttpStatus.OK);
    }


}
```

如果要返回具体的参数错误，则需要拦截对应的 MethodArgumentNotValidException 异常，并做对应的返回处理，示例如下：

```
@ResponseBody
@ExceptionHandler(value = {MethodArgumentNotValidException.class})
public ResponseEntity handleMethodArgumentNotValidException (MethodArgumentNotValid-
Exception ex) {
    BindingResult bindingResult = ex.getBindingResult();
    StringBuilder sb = new StringBuilder(" 参数校验失败 :");
    for (FieldError fieldError : bindingResult.getFieldErrors()) {
        sb.append(fieldError.getField()).append(": ").append(fieldError.getDefaultMessage()).
append(", ");
    }
    String msg = sb.toString();
    return new ResponseEntity(msg, HttpStatus.OK);
}
```

重启应用，调用保存用户接口测试一下，结果如下图所示。



如果参数不符合条件就会返回对应的错误。

以上仅为异常测试，实际项目中应把接口响应信息封装成带有 code、msg、data 的标准化 JSON 格式的数据返回，然后将错误信息放在 msg 字段中。

6.12 国际化

6.12.1 概述

Spring Boot 支持本地化的消息，也就是国际化，如果应用需要满足不同语言偏好的用户，这就很有用。默认情况下，Spring Boot 会在类路径的根目录下搜索消息资源包是否存在。

Spring Boot 中已经对国际化做了自动配置，国际化自动配置类如下：

org.springframework.boot.autoconfigure.context.MessageSourceAutoConfiguration

它同样被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中，该自动配置类的核心源码如下：

```
@AutoConfiguration
@ConditionalOnMissingBean(name = AbstractApplicationContext.MESSAGE_SOURCE_BEAN_NAME,
search = SearchStrategy.CURRENT)
@AutoConfigureOrder(Ordered.HIGHEST_PRECEDENCE)
```

```
@Conditional(ResourceBundleCondition.class)
@EnableConfigurationProperties
public class MessageSourceAutoConfiguration {

    private static final Resource[] NO_RESOURCES = {};

    @Bean
    @ConfigurationProperties(prefix = "spring.messages")
    public MessageSourceProperties messageSourceProperties() {
        return new MessageSourceProperties();
    }

    @Bean
    public MessageSource messageSource(MessageSourceProperties properties) {
        ResourceBundleMessageSource messageSource = new ResourceBundleMessageSource();
        if (StringUtils.hasText(properties.getBasename())) {
            messageSource.setBasenames(StringUtils
                    .commaDelimitedListToStringArray(StringUtils.trimAllWhitespace
(properties.getBasename())));
        }
        if (properties.getEncoding() != null) {
            messageSource.setDefaultEncoding(properties.getEncoding().name());
        }
        messageSource.setFallbackToSystemLocale(properties.isFallbackToSystemLocale());
        Duration cacheDuration = properties.getCacheDuration();
        if (cacheDuration != null) {
            messageSource.setCacheMillis(cacheDuration.toMillis());
        }
        messageSource.setAlwaysUseMessageFormat(properties.isAlwaysUseMessageFormat());
        messageSource.setUseCodeAsDefaultMessage(properties.isUseCodeAsDefaultMessage());
        return messageSource;
    }

    ...

}
```

主要是注册了一个MessageSource实例，通过一系列spring.messages.*参数可以完成自动配置，查看参数绑定类MessageSourceProperties的源码，有以下主要几个重要的配置参数：

```
private String basename = "messages";

private Charset encoding = StandardCharsets.UTF_8;
```

```
@DurationUnit(ChronoUnit.SECONDS)
private Duration cacheDuration;

private boolean fallbackToSystemLocale = true;
```

这几个参数的说明如下：

- basename：指定要扫描的国际化文件名，默认为 messages，即在 resources 资源目录下建立 messages_xx.properties 资源文件，可以通过逗号指定多个，如果不指定包名，则默认从 classpath 根目录下搜索。
- encoding：默认的编码为 UTF-8。
- cacheDuration：国际化资源文件被加载后的缓存时间，默认单位为秒，如果不指定则默认为永久缓存。
- fallbackToSystemLocale：找不到当前语言的资源文件时是否降级为当前操作系统的语言对应的资源文件，默认为 true，比如在中国则加载 messages_zh_CN.properties，如果为 false 则加载系统默认的文件，如 messages.properties。

6.12.2 自动国际化

了解了国际化的自动配置和参数绑定类的作用，本节就可以进行国际化应用实践了，本节将介绍如何使用 Spring Boot 和 Thymeleaf 模板引擎实现页面国际化。

应用国际化支持在 Spring Boot 自动配置核心包中，引入 Spring Boot Web 包即可，不需要额外引入其他依赖包，因为本节使用 Thymeleaf 作为模板页面来显示国际化消息，所以还需要引入 Thymeleaf 的依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

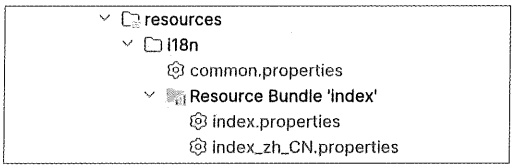
然后配置国际化资源文件的搜索路径：

```
spring:
  messages:
    basename: i18n/common, i18n/index
fallback-to-system-locale: false
```

配置参数说明如下：

- basename：这里指定了多个国际化资源文件，在 common 资源文件中定义了公共的消息，在 index 资源文件中定义了首页的消息，让页面可以读取多个资源文件的消息。
- fallback-to-system-locale：设置为 false 表示如果找不到当前语言的文件就使用默认的资源文件，这里默认的资源文件是 index.properties，里面定义为英文，即除了中文，其他都使用默认的英文资源文件。

然后在资源根路径下创建 i18n 国际化资源目录，用来统一管理国际化资源文件，以及创建对应的国际化资源文件，如下图所示。



common.properties文件的内容如下：

```
brand=Java 技术栈
```

index.properties 文件的内容如下：

```
index.hi=hello
index.welcome=welcome
```

index_zh_CN.properties 文件的内容如下：

```
index.hi= 你好
index.welcome= 欢迎光临
```

创建 Thymeleaf 模板文件，引用国际化资源文件中的消息：

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
```

```
    <title>index</title>
</head>
<body>

<label th:text="#{brand}"></label>
<label th:text="#{index.hi}"></label>
<label th:text="#{index.welcome}"></label>

</body>
</html>
```

创建首页接口：

```
@GetMapping(value = "/index")
public String index() {
    return "index";
}
```

然后在浏览器上访问 index 页面，如下图所示。

因为笔者的计算机是中文环境，所以页面显示了 index_zh_CN.properties 资源文件中的消息内容，这就是页面的自动国际化。

6.12.3 切换国际化

大部分场景下，页面需要支持手动切换语言，比如很多网站会在页面顶部或者底部设置切换语言的链接，这时之前的自动国际化配置就不符合要求了。

这时需要注册一个 LocaleResolver 区域解析器和区域拦截器：

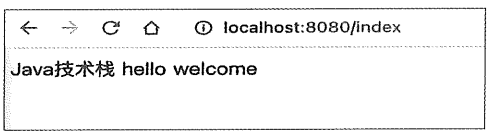
```
@Slf4j
@Configuration
public class WebConfig implements WebMvcConfigurer {

    /**
     * Locale 默认设置为英文
     * @return
     */
```

```
    @Bean
    public LocaleResolver localeResolver() {
        SessionLocaleResolver sessionLocaleResolver = new SessionLocaleResolver();
        sessionLocaleResolver.setDefaultLocale(Locale.US);
        return sessionLocaleResolver;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(localeChangeInterceptor());
    }

    /**
     * 切换语言拦截器，通过 url?lang=zh_CN 形式进行切换
     * @return
     */
    private LocaleChangeInterceptor localeChangeInterceptor() {
        LocaleChangeInterceptor localeChangeInterceptor = new LocaleChangeInterceptor();
        localeChangeInterceptor.setParamName("lang");
        return localeChangeInterceptor;
    }

}
```

配置类说明如下：

- 首先注册了一个 LocaleResolver 实例，LocaleResolver 接口有许多实现，比如可以通过 Session、Cookie、Accept-Language Header 或者一个固定的值来判断当前的语言环境，上面使用的是 SessionLocaleResolver（基于 Session）的方式来判断的，并设置默认区域语言为英文。
- 然后注册了一个 LocaleChangeInterceptor 实例，并且设置了切换语言的参数名为 lang，即可以通过传递区域语言参数 url?lang=zh_CN 的形式进行区域语言的切换。

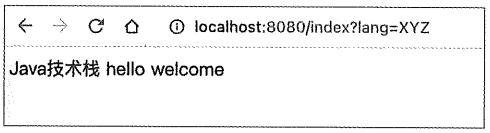
现在再在浏览器上访问 index 页面，如下图所示。



页面显示了默认的英文区域的内容，前面的中文是因为加载了 common 公共资源文件中的中文内容，common 是公共资源内容，没有做国际化资源配置。

现在再通过传递中文区域来访问 index页面，如下图所示。

可以看到语言切换成功了，然后传递一个不存在的区域语言，如下图所示。



可以看到显示的是默认的英文区域的内容了。

6.13 分布式会话

传统的会话机制保存在应用服务器端，会话共享需要借助 Servlet 容器来实现，比如 Tomcat 提供了 Session 复制的功能，简单的集群复制没有什么问题，但是，如果集群节点过多，就可能会因网络延迟等问题造成会话不能及时同步，进而给应用带来性能影响，严重的问题会影响用户体验。

Spring Session 让集群会话变得轻而易举，它是 Spring 提供的一个用于管理用户会话的框架，可以将 Session 数据存储到第三方存储中并替换容器中的 HttpSession 机制，而无须绑定特定的应用容器。

Spring Boot 为 Spring Session 及各种数据存储提供了自动配置，支持 Servlet Web 应用和 Reactive Web 应用。以 Servlet Web 应用为例，可以自动配置基于以下组件的 Session 存储：

- Redis。
- JDBC。
- Hazelcast。
- MongoDB。

如果类路径中存在以上单个 Spring Session 模块，则 Spring Boot 会自动使用该存储实现，如果有多个实现，则 Spring Boot 按以上列举的组件顺序来选择，如果一个都没有，则使用默认的 Session 机制。

Session 自动配置类为 org.springframework.boot.autoconfigure.session.SessionAutoConfiguration，它同样被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中，Spring Boot 自动配置后，就不再需要使用 @Enable*HttpSession 注解了，比如基于 Redis 方式

实现会话则需要使用 @EnableRedisHttpSession 注解，现在都不需要了。

Session 参数绑定类为 SessionProperties，通过一系列 spring.session.* 参数配置 Session 会话，每个存储也都有自己的扩展实现，比如 Redis 参数绑定类为 RedisSessionProperties，通过一系列 spring.session.redis.* 可以扩展 Redis 会话存储配置。

本节就以 Redis 存储会话为例，演示如何使用 Spring Boot 集成 Redis 实现分布式的会话。

Redis 的安装、集成及具体介绍请参考第 7章。

首先添加集成 Redis 及会话相关的依赖：

```
<dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-data-redis</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

然后添加以下 Redis 会话及连接配置：

```
spring:
  session:
    timeout: 30s
  redis:
    host: localhost
port: 6379
```

Spring Boot 集成 Spring Session 后，可以使用 spring.session.timeout 参数设置超时配置，如果没有设置，就会取 server.servlet.session.timeout 设置的值（默认为 30 分钟），如果两个参数都未设置，则为后者的默认值（30 分钟），这个逻辑可以在 RedisSessionConfiguration 配置类中找到，如下图所示。

```
PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
return (sessionRepository) -> {
    map.from(sessionProperties
            .determineTimeout(() -> serverProperties.getServlet().getSession().getTimeout()))
            .to(sessionRepository::setDefaultMaxInactiveInterval);
    map.from(redisSessionProperties::getNamespace).to(sessionRepository::setRedisKeyNamespace);
    map.from(redisSessionProperties::getFlushMode).to(sessionRepository::setFlushMode);
    map.from(redisSessionProperties::getSaveMode).to(sessionRepository::setSaveMode);
};
```

然后新建几个基础接口用于测试会话：

```
@Slf4j
@RequiredArgsConstructor
@Controller
public class IndexController {

    private final HttpSession httpSession;

    /**
     * 登录页面
     * @return
     */
    @ResponseBody
    @RequestMapping("/login")
    public String login() {
        return "login page.";
    }

    /**
     * 登录请求
     * @param username
     * @return
     */
    @RequestMapping("/login/submit")
    public String loginSubmit(@RequestParam("username") String username) {
        if (StringUtils.isNotBlank(username)) {
            httpSession.setAttribute("username", username);
            return "/index";
        }
        return "/login";
    }

    /**
     * 首页
     * @return
     */
    @ResponseBody
    @RequestMapping("/index")
    public String index() {
        log.info("session id: {}", httpSession.getId());
        return "index page.";
    }
```

```
    /**
     * 退出登录
     * @return
     */
    @RequestMapping("/logout")
    public String logout() {
        httpSession.invalidate();
        return "/login";
    }

}
```

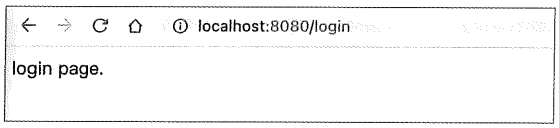
这里包括登录页面、首页、登录请求、退出登录接口，可以像往常一样正常注入并使用HttpSession，它会自动存储到Redis中。

过滤器、Web相关配置代码略，请查看笔者提供的Spring Boot最佳实践项目仓库中的spring-boot-session模块的完整源代码。

笔者这里同时启动了两个应用实例：

- 8080。
- 8081。

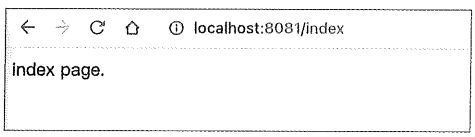
在没有登录的情况下，访问两个实例的/index首页接口都会跳转到登录页面，如下图所示。



然后在8080实例上发起登录请求：

http://localhost:8080/login/submit?username=test

这时Redis中应该有会话数据了，再次访问两个实例的/index首页接口，都成功跳转到了首页，如下图所示。

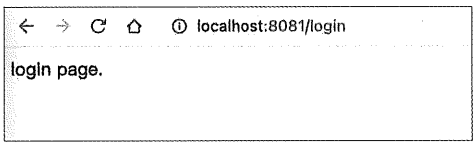


这就说明分布式会话生效了，然后验证是否在Redis中存储了会话：

```
127.0.0.1:6379> keys *
1) "spring:session:sessions:bbb3dc40-7d77-426f-b079-bdd55533061f"
127.0.0.1:6379> ttl spring:session:sessions:bbb3dc40-7d77-426f-b079-bdd55533061f
(integer) 24
```

已经在Redis中查询到了一个会话数据，过期时间是配置的30秒，在笔者查询时还有24秒过期，并且每次访问应用的任何接口都可以进行会话续期。

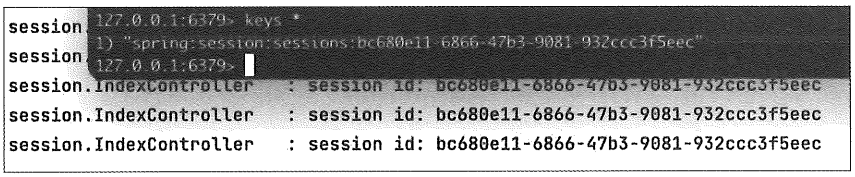
24秒后再访问两个实例的/index首页接口，如下图所示。



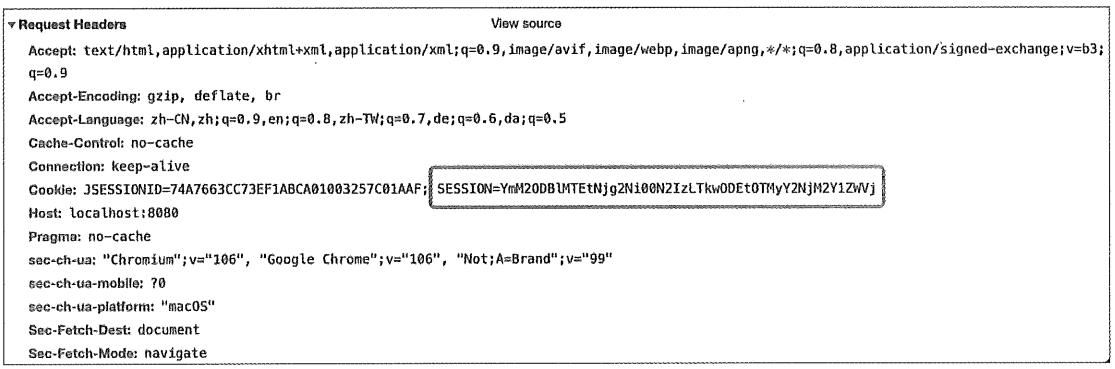
因为会话过期了，所以又跳转到了登录页面。

除了会话自动过期，还可以通过主动退出登录让Redis中的会话过期，具体见上面的/logout退出登录方法，当退出登录后，再去Redis中查询，会话数据会马上消失。

另外，Redis中存储的Session ID是应用的HttpSession中的Session ID，如下图所示。



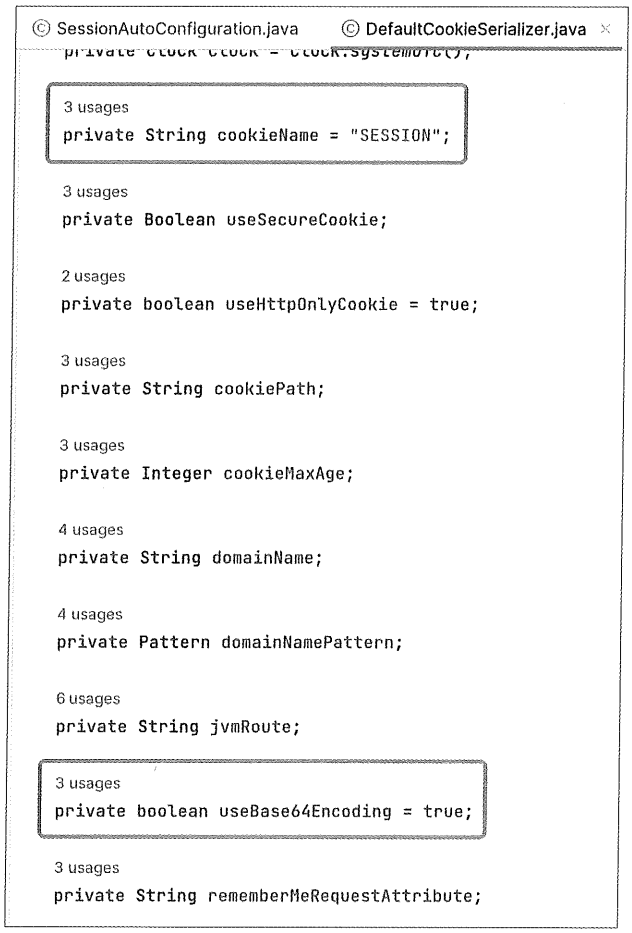
通过读取请求头Cookie中的SESSION值实现会话，如下图所示。



Cookie中传递了JSESSIONID和SESSION两个值，Spring Session取的是SESSION的值，同一个客户端的多个应用会分配同一个SESSION值，这样就能实现分布式的会话，如果不是基于浏

览器的多个应用之间的相互调用，则只需手动传输 SESSION 值即可。

可以发现这里的 SESSION 值和应用、Redis 中的 Session ID 值并不一样，这是因为这个 SESSION 值是经过 BASE64 编码后返回的，SessionAutoConfiguration 自动配置类中注册了一个 DefaultCookieSerializer（默认 Cookie 序列化器），默认的 Cookie 名称为 SESSION 并启用了 BASE64 编码，如下图所示。



如果希望这个 SESSION 值和服务器中的保持一致，则可以自定义一个 DefaultCookieSerializer 实例，覆盖自动配置注册的默认实例，然后设置该实例不使用 BASE64 编码即可。

以上演示了 Redis 的分布式会话的使用方式，其他存储组件也是同样的道理。

6.14 跨域

要了解跨域，先要说一说同源策略，同源策略是由 Netscape 公司提出的一个著名的安全策略，所有支持 JavaScript 的浏览器都会使用这个策略。所谓同源是指，域名、协议、端口相同，当页面在执行一个脚本时会检查访问的资源是否同源，如果非同源，那么在请求数据时，浏览器会在控制台中报一个异常，提示拒绝访问。

所以，只要协议、域名、端口有任何一个不同，就被当作跨域，跨域情况下浏览器是不能执行其他域名网站的脚本的，这是由浏览器的同源策略造成的，是浏览器施加的安全限制。

实际工作开发中经常会有跨域的情况，因为公司会有很多项目，也会有很多域名，各个项目或者网站之间需要相互调用对方的资源，避免不了跨域请求。

Spring MVC 框架本身是支持跨域请求的，在需要跨域的方法上使用 @CrossOrigin 注解即可：

```
@RestController
public class ResponseBodyController {

    @CrossOrigin
    @GetMapping(value = "/user/json/{userId}", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity getJsonUserInfo(@PathVariable("userId") String userId) {
        User user = new User("Java 技术栈 ", 18);
        user.setId(Long.valueOf(userId));
        return new ResponseEntity(user, HttpStatus.OK);
    }

}
```

默认情况下 @CrossOrigin 支持：

- 所有的请求来源。
- 所有的 HTTP 头。
- 所有的 HTTP 方法。

如果有需要，则可以在 @CrossOrigin 注解中指定跨域的详细参数，除了作用在指定方法上，@CrossOrigin 注解还可以作用在类上：

```
@CrossOrigin(origins = "https://javastack.cn", maxAge = 3600)
@RestController
public class ResponseBodyController {
```

```
...
```

这样所有方法都能实现跨域支持，当然也可以同时作用于类和方法上，比如公共的跨域参数在类注解上指定，其他的跨域参数在各自接口方法上指定。

如果不想在类或者方法上指定，也可以在WebMvcConfigurer配置类中全局添加跨域注册的功能：

```
@Configuration
public class WebConfig implements WebMvcConfigurer {


    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/user/**")
            .allowedMethods("GET", "POST")
            .allowedOrigins("https://javastack.cn")
            .allowCredentials(true).maxAge(3600);
    }


}
```

如果应用需要全局统一配置跨域请求，则可以考虑使用最后这种方式，但又回到6.2.5节的情况，实际项目中Spring Boot应用不可能直接对外，需要使用Nginx之类的负载均衡组件，所以会由运维人员统一在Nginx上配置跨域。

6.15 安全性

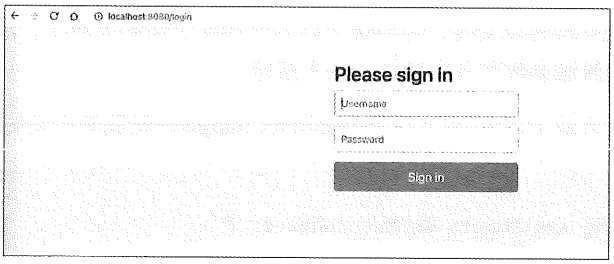
6.15.1 默认的安全机制

如果存在Spring Security类库依赖，Spring Boot就会对Web应用实现默认的安全保护，包括对Actuator Endpoints端点的保护，具体见第12章Spring Boot监控与报警。

加入Spring Security启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

加入 Spring Security 启动器依赖后重启应用，再访问应用的 /test/** 接口，如下图所示。



如果没有登录授权，那么 Spring Boot 默认会跳到登录页面，需要输入默认的用户信息才能访问。默认的用户名为 user，密码则在应用启动时生成并输出到日志中：

```
Using generated security password: fff9585f-741b-4a58-bf0b-53ce924a20c2
```

Spring Security 自动配置类都被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中，主要包括以下几个自动配置类：

- SecurityAutoConfiguration。
- UserDetailsServiceAutoConfiguration。

SecurityAutoConfiguration 是 Spring Security 框架的主要自动配置类，源码如下：

```
@AutoConfiguration
@ConditionalOnClass(DefaultAuthenticationEventPublisher.class)
@EnableConfigurationProperties(SecurityProperties.class)
@Import({ SpringBootWebSecurityConfiguration.class, SecurityDataConfiguration.class })
public class SecurityAutoConfiguration {

      @Bean
      @ConditionalOnMissingBean(AuthenticationEventPublisher.class)
      public DefaultAuthenticationEventPublisher authenticationEventPublisher
(ApplicationEventPublisher publisher) {
            return new DefaultAuthenticationEventPublisher(publisher);
      }

}
```

它注册了一个 DefaultAuthenticationEventPublisher 实例，用于发布授权成功、失败事件，在类上还导入了一些配置类，最主要的就是 SpringBootWebSecurityConfiguration 配置类，它的源码如下：

```
@Configuration(proxyBeanMethods = false)
@ConditionalOnWebApplication(type = Type.SERVLET)
class SpringBootWebSecurityConfiguration {

    /**
     * The default configuration for web security. It relies on Spring Security's
     * content-negotiation strategy to determine what sort of authentication to use. If
     * the user specifies their own {@link SecurityFilterChain} bean, this will back-off
     * completely and the users should specify all the bits that they want to configure as
     * part of the custom security configuration.
     */
    @Configuration(proxyBeanMethods = false)
    @ConditionalOnDefaultWebSecurity
    static class SecurityFilterChainConfiguration {

        @Bean
        @Order(SecurityProperties.BASIC_AUTH_ORDER)
        SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
            http.authorizeHttpRequests().anyRequest().authenticated();
            http.formLogin();
            http.httpBasic();
            return http.build();
        }

    }

    ...

}
```

它主要注册了一个 SecurityFilterChain，并提供了默认的授权机制，即默认所有请求都需要授权，并且支持 formLogin 和 httpBasic 两种授权方式，基于请求中的 Accept 请求头的值决定使用哪种方式。

UserDetailsServiceAutoConfiguration 自动配置类提供了 Spring Security 默认授权用户的自动配置，它的源码如下：

```
@AutoConfiguration
@ConditionalOnClass(AuthenticationManager.class)
@ConditionalOnBean(ObjectPostProcessor.class)
@ConditionalOnMissingBean(
        value = { AuthenticationManager.class, AuthenticationProvider.class,
                UserDetailsService.class, AuthenticationManagerResolver.class },
        type = { "org.springframework.security.oauth2.jwt.JwtDecoder",
```

```
                "org.springframework.security.oauth2.server.resource.introspection.
OpaqueTokenIntrospector",
                "org.springframework.security.oauth2.client.registration.
ClientRegistrationRepository",
                "org.springframework.security.saml2.provider.service.registration.
RelyingPartyRegistrationRepository" })
public class UserDetailsServiceAutoConfiguration {

    ...

    @Bean
    @Lazy
    public InMemoryUserDetailsManager inMemoryUserDetailsManager(SecurityProperties properties,
            ObjectProvider<PasswordEncoder> passwordEncoder) {
        SecurityProperties.User user = properties.getUser();
        List<String> roles = user.getRoles();
        return new InMemoryUserDetailsManager(
                User.withUsername(user.getName()).password(getOrDeducePassword(user,
passwordEncoder.getIfAvailable()))
                        .roles(StringUtils.toStringArray(roles)).build());
    }

    ...

}
```

它注册了一个InMemoryUserDetailsManager，提供了单个用户的授权管理器，对应的SecurityProperties参数类的源码如下：

```
@ConfigurationProperties(prefix = "spring.security")
public class SecurityProperties {

    ...

    public static class User {

        /**
         * Default user name.
         */
        private String name = "user";

        /**
         * Password for the default user name.
```

```
         */
        private String password = UUID.randomUUID().toString();

        /**
         * Granted roles for the default user name.
         */
        private List<String> roles = new ArrayList<>();

        private boolean passwordGenerated = true;

        ...

    }

}
```

默认授权用户为：

- name：user。
- password：通过 UUID生成的随机值。

用户类提供了默认的用户名和密码，默认的密码是生成的 UUID随机值，可以通过 spring.security.user.* 来配置并替换默认的用户信息，例如：

```
spring:
  security:
    user:
      name: root
      password: root
```

默认的用户信息和明文密码只供开发阶段使用，如果应用部署到生产环境中，则建议做好严格的安全配置。

Spring Security 同样支持 Actuator Endpoints 端点的默认安全保护机制，具体见 12.2 节。

6.15.2 自定义安全机制

如果不想使用 Spring Boot 自动配置的默认机制，那么可以通过注册一个 SecurityFilterChain 类和 UserDetailsService 类到 Spring 容器中实现自定义的安全机制，Spring Boot 中的默认安全机制就不会自动配置。

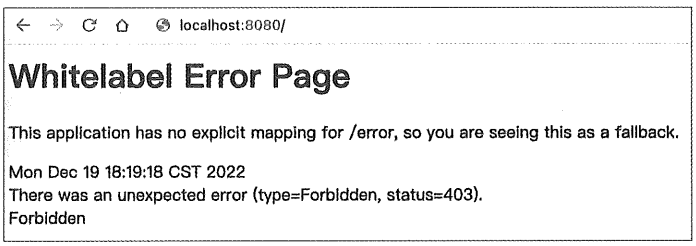
自定义安全机制的示例代码如下：

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http.authorizeHttpRequests((authorize) -> {
                    authorize.requestMatchers("/test/**").hasRole("TEST")
                            .requestMatchers("/**").permitAll();
                })
                .logout().logoutSuccessUrl("/")
                .and().formLogin(withDefaults())
                .build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        InMemoryUserDetailsManager manager = new InMemoryUserDetailsManager();
        manager.createUser(User.withUsername("test").password("{noop}test")
                .roles("ADMIN", "TEST").build());
        manager.createUser(User.withUsername("root").password("{noop}root")
                .roles("ADMIN").build());
        return manager;
    }

}
```

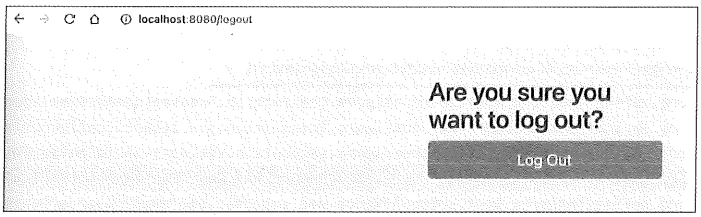
这里注册了一个 SecurityFilterChain 实例，首先配置了以 /test/** 开头的 URL 需要拥有 TEST 角色才能访问，然后其他所有 URL 都能正常访问，并且提供了 formLogin 授权登录方式及注销后跳转首页功能，默认注销地址为 /logout，注销成功后跳转到 / 首页。

注册的 UserDetailsService 实例创建了两个用户，只有 test 用户才拥有 TEST 角色，然后重启应用，只有输入拥有 TEST 角色的 test 用户才能访问 /test/** 接口，输入其他用户会返回 403 错误。

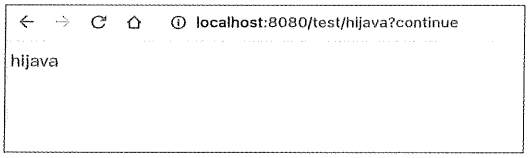
先使用 root 用户登录并访问 /test/** 接口，接口会返回 403 错误页面，如下图所示。



然后访问 /logout 注销登录接口，如下图所示。



注销登录后再次以 test 用户登录并访问 /test/**接口，如下图所示。



因为 test 用户拥有 TEST 角色，所以授权访问成功。

6.16 REST 服务调用

在 Spring Boot 中实现 REST 服务调用有以下两种方式：

- RestTemplate（SERVLET）。
- WebClient（REACTIVE）。

本节将分别介绍这两种调用方式。

6.16.1 RestTemplate（Servlet）

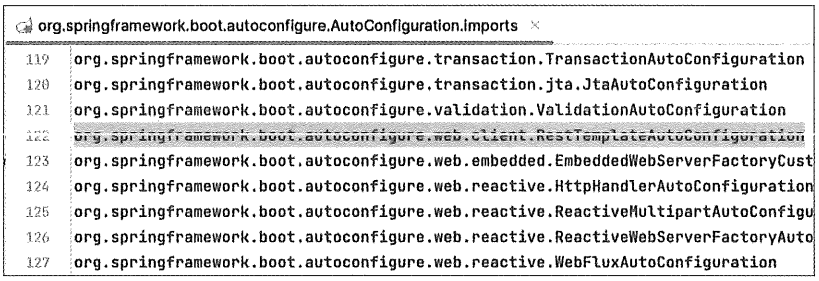
1. 实现 REST 服务调用

RestTemplate 是 Spring 框架中实现 REST 服务调用最基础的方式，因为 RestTemplate 是基于 Servlet 的 Web 应用，所以需要加入 spring-boot-starter-web 启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

根据 Spring Boot 自动配置规则可以知道，RestTemplate 的自动配置类为 RestTemplateAuto-

Configuration，它同样被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration. imports 自动配置文件中，如下图所示。



因为 RestTemplate 一般要根据应用自身情况自定义配置后才能使用，所以 Spring Boot 没有提供任何默认的 RestTemplate 注册实例，但是通过 RestTemplateAutoConfiguration 自动配置提供了 RestTemplateBuilder 的默认注册实例，通过它可以创建 RestTemplate 实例。

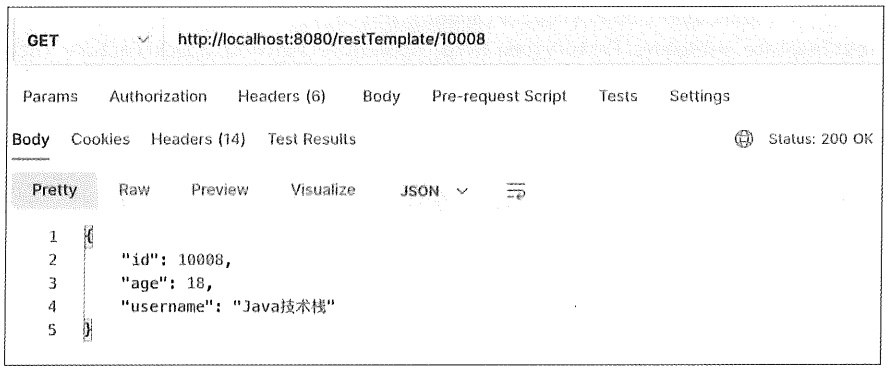
RestTemplateBuilder 提供了许多有用的方法，可用于快速配置 RestTemplate，比如笔者这里注册了一个简单的 RestTemplate 实例，如下面的示例所示。

```
@Bean
public RestTemplate defaultRestTemplate(RestTemplateBuilder restTemplateBuilder) {
    return restTemplateBuilder.build();
}
```

添加一个测试接口，然后注入并使用 RestTemplate，如下面的示例所示。

```
@Slf4j
@RestController
@RequiredArgsConstructor
public class CallRestController {

    public static final String GET_USERINFO_URL = "http://localhost:8080/user/json/{uid}";

    private final RestTemplate restTemplate;

    @GetMapping("/restTemplate/{uid}")
    public User restTemplate(@PathVariable("uid") String uid) {
        return this.restTemplate.getForObject(GET_USERINFO_URL, User.class, uid);
    }

}
```

RestTemplate 的用法很简单，这里直接调用自身应用的获取用户信息接口，传入要调用的 URL、反序列化的类型、接口请求参数，然后发起调用测试，如下图所示。



数据返回成功。RestTemplate 还有更多的 API 及用法，读者可以自行研究。

2. 自定义 RestTemplate

一般的自定义 RestTemplate 可以通过 RestTemplateBuilder 的方法实现，示例如下：

```
@Bean
public RestTemplate defaultRestTemplate(RestTemplateBuilder restTemplateBuilder) {
    return restTemplateBuilder
            .setConnectTimeout(Duration.ofSeconds(5))
            .setReadTimeout(Duration.ofSeconds(5))
            .basicAuthentication("test", "test")
            .build();
}
```

这里设置了请求和读取超时时间，以及 Basic 认证信息。

如果要实现其他附加的自定义配置，则可以实现并注册一个 RestTemplateCustomizer 接口的实例，它会应用到 RestTemplateBuilder 实例及由它构建的 RestTemplate 实例上，如下面的示例所示。

```
@Slf4j
@Component
public class CustomRestTemplateCustomizer implements RestTemplateCustomizer {

    @Override
    public void customize(RestTemplate restTemplate) {
        HttpRoutePlanner routePlanner = new CustomRoutePlanner(new HttpHost("proxy.
javastack.cn"));
```

```
        RequestConfig requestConfig = RequestConfig.custom().build();
        HttpClient httpClient = HttpClientBuilder.create()
                .setDefaultRequestConfig(requestConfig)
                .setRoutePlanner(routePlanner).build();
        restTemplate.setRequestFactory(new HttpComponentsClientHttpRequestFactory (httpClient));
    }

    static class CustomRoutePlanner extends DefaultProxyRoutePlanner {

        CustomRoutePlanner(HttpHost proxy) {
            super(proxy);
        }

        @Override
        protected HttpHost determineProxy(HttpHost target, HttpContext context) throws
HttpException {
            log.info("hostName is {}", target.getHostName());
            if ("localhost".equals(target.getHostName())) {
                return null;
            }
            return super.determineProxy(target, context);
        }

    }

}
```

上面使用了 httpclient5 作为连接客户端，所以需要引入 httpclient5 相关的依赖：

```
<dependency>
    <groupId>org.apache.httpcomponents.client5</groupId>
    <artifactId>httpclient5</artifactId>
</dependency>

<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpcore</artifactId>
</dependency>
```

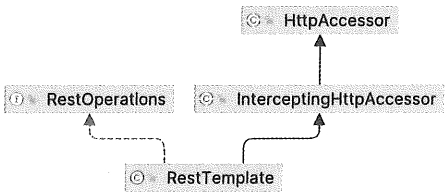
注册这个 CustomRestTemplateCustomizer 实例后，所有 localhost 之外的 URL 调用都会经过指定的代理。

除此之外，还可以自定义 RestTemplateBuilder 实例，以覆盖自动配置的 RestTemplateBuilder 实例，

不过一般很少会使用，默认提供的机制完全可以满足日常需要。

3. 底层实现客户端

RestTemplate 的类结构如下图所示。

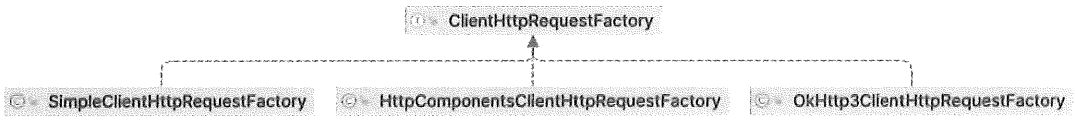


HttpAccessor 是顶层的抽象类，RestOperations 接口则定义了一系列的常用的 RESTful 操作方法，RestTemplate 便是该接口默认和唯一的实现。

HttpAccessor 默认实例化了一个简单的 SimpleClientHttpRequestFactory 工厂类，如 HttpAccessor 类的源码所示。

```
public abstract class HttpAccessor {

    /** Logger available to subclasses. */
    protected final Log logger = HttpLogging.forLogName(getClass());

    private ClientHttpRequestFactory requestFactory = new SimpleClientHttpRequestFactory();

    private final List<ClientHttpRequestInitializer> clientHttpRequestInitializers = new
ArrayList<>();

    ...

}
```

ClientHttpRequestFactory 是用来创建 HTTP 连接的工厂接口，它包含了几个主流的客户端连接工厂实现类，如下图所示。



默认使用的是 SimpleClientHttpRequestFactory工厂类，其内部使用的是 JDK 自带的 URLConnection 类创建的连接，如该类中创建连接方法的源码所示。

```
public class SimpleClientHttpRequestFactory implements ClientHttpRequestFactory {

    ...

    /**
     * Opens and returns a connection to the given URL.
     * <p>The default implementation uses the given {@linkplain #setProxy(java.net.
Proxy) proxy} -
     * if any - to open a connection.
     * @param url the URL to open a connection to
     * @param proxy the proxy to use, may be {@code null}
     * @return the opened connection
     * @throws IOException in case of I/O errors
     */
    protected HttpURLConnection openConnection(URL url, @Nullable Proxy proxy) throws
IOException {
        URLConnection urlConnection = (proxy != null ? url.openConnection(proxy) :
url.openConnection());
        if (!(urlConnection instanceof HttpURLConnection)) {
            throw new IllegalStateException(
                 "HttpURLConnection required for [" + url + "] but got: " + urlConnection);
        }
        return (HttpURLConnection) urlConnection;
    }

    ...

}
```

RestTemplate 中的构造器和 setRequestFactory 方法都可以用来切换 ClientHttpRequestFactory，比如在自定义 RestTemplate 示例中，就实现了 HttpComponentsClientHttpRequestFactory 工厂类的切换，它其实就是 httpclient5 连接客户端的实现。

6.16.2 WebClient（Reactive）

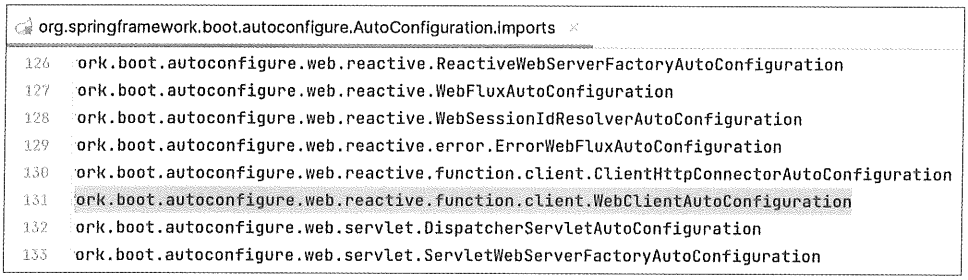
1. 实现 REST 服务调用

如果使用的是 Spring WebFlux 基于 Reactive 的 Web 应用，则可以使用 WebClient 来调用远程 REST 服务。与 RestTemplate 相比，WebClient 客户端的功能更强大，并且是完全基于响应式的。

首先加入 spring-boot-starter-webflux 启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

根据 Spring Boot 自动配置规则可以知道，WebClient 的自动配置类为 WebClientAutoConfiguration，它同样被注册在新的 org.springframework.boot.autoconfigure. AutoConfiguration.imports 自动配置文件中，如下图所示。



Spring Boot 在该自动配置类中注册了一个 WebClient.Builder 接口默认实现类 DefaultWebClientBuilder 的实例，官方推荐使用它来创建 WebClient 客户端实例，比如笔者这里注册了一个简单的 WebClient 实例，如下面的示例所示。

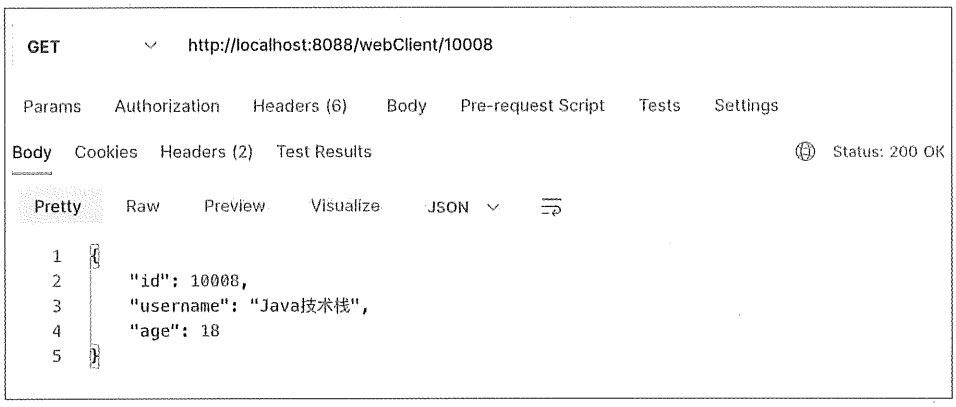
```
@Bean
public WebClient webClient(WebClient.Builder webClientBuilder) {
    return webClientBuilder.build();
}
```

添加一个测试接口，然后注入 WebClient 就能使用了，如下面的示例所示。

```
@Slf4j
@RestController
@RequiredArgsConstructor
public class CallRestController {

    public static final String GET_USERINFO_URL = "http://localhost:8080/user/json/{uid}";

    private final WebClient webClient;

    @GetMapping("/webClient/{uid}")
    public Mono<User> webClient(@PathVariable("uid") String uid) {
        return this.webClient.get().uri(GET_USERINFO_URL, uid)
```

```
            .retrieve().bodyToMono(User.class);
    }

}
```

WebClient 的用法也很简单，这里直接调用上一节 Servlet 应用的获取用户信息接口，传入要调用的 URL、反序列化的类型、接口请求参数，然后发起调用测试，如下图所示。



数据返回成功。WebClient 还有更多的 API 及用法，读者可以自行研究。

另外，本节介绍的是基于 Webflux 的应用，所以默认使用的是 Netty 嵌入式容器，如下面的启动日志所示。

```
.   ____          _            __ _ _
/\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
\\/  ___)| |_)| | | | | || (_| |  ) ) ) )
'  |____| .__|_| |_|_| |_\__, | / / / /
=========|_|==============|___/=/_/_/_/
:: Spring Boot ::                (v3.0.0)

...
... o.s.b.web.embedded.netty.NettyWebServer  : Netty started on port 8088
...
```

2. 自定义 WebClient

一般的自定义 WebClient 可以通过 WebClient.Builder 的方法来实现，示例如下：

```
@Bean
public WebClient webClient(WebClient.Builder webClientBuilder) {
    HttpClient httpClient = HttpClient.create()
            .tcpConfiguration(client ->
                    client.option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3)
                            .doOnConnected(conn -> {
                                conn.addHandlerLast(new ReadTimeoutHandler(3000));
                                conn.addHandlerLast(new WriteTimeoutHandler(3000));
                            })
            );
    ReactorClientHttpConnector connector = new ReactorClientHttpConnector(httpClient);
    return webClientBuilder.clientConnector(connector).build();
}
```

这里创建了一个基于 Reactor Netty 的 ReactorClientHttpConnector 连接器，通过它来设置连接、读写超时时间，通过自定义 ClientHttpConnector 的方式来完全控制 Client 端的配置，它会改变 WebClient.Builder 注册实例的默认配置。所以，如果需要使用这种方式构建多个不同配置的 WebClient 客户端，则可以在 WebClient.Builder 被修改前调用其 clone 方法复制一个新实例再进行构建。

> 如果使用的是 Jetty 和 Reactor Netty 响应式服务器，则可以通过自定义 ReactorResourceFactory、JettyResourceFactory 的方式来覆盖资源配置，它会同时作用于 Server 端和 Client 端。

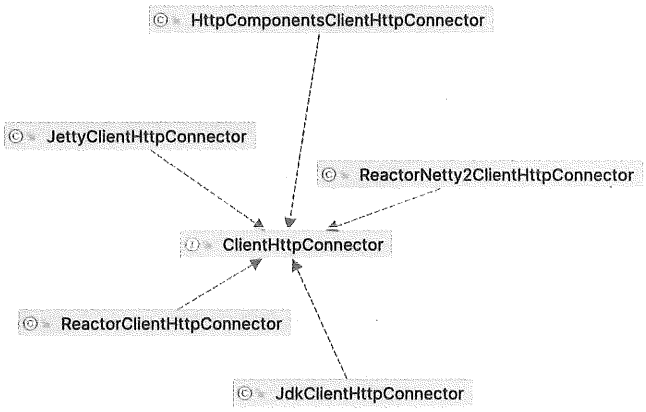
另外，和 RestTemplate 一样，WebClient 也提供了附加的自定义配置，实现并注册一个 WebClientCustomizer 类型的 Bean 即可。

最后，还可以通过使用 WebClient.create() 方法来手动创建 WebClient 实例，此时，自动配置中的 WebClient 就会自动取消注册。同时，它也不会应用 WebClientCustomizer 配置。

3. 底层实现客户端

前面使用了 ClientHttpConnector 连接器，它是 HTTP 客户端的底层抽象驱动接口，用于驱动 WebClient 客户端连接到服务器，并提供发送 ClientHttpRequest 和接收 ClientHttpResponse 所需的所有支撑。

Spring Boot 会自动检测应用要使用的 ClientHttpConnector 连接器类型，并通过它来驱动 WebClient 客户端。ClientHttpConnector 的类结构如下图所示。



具体使用哪种 ClientHttpConnector 连接器，就要看应用使用了哪个客户端，Spring Boot 目前支持以下几种类型的客户端：

- Reactor Netty（默认）。
- Jetty RS Client。
- Apache HttpClient。
- JDK HttpClient。

DefaultWebClientBuilder#build 方法调用的 DefaultWebClientBuilder#initConnector 初始化连接器方法的实现源码如下所示。

```
final class DefaultWebClientBuilder implements WebClient.Builder {

    ...

    static {
        ClassLoader loader = DefaultWebClientBuilder.class.getClassLoader();
        reactorNettyClientPresent = ClassUtils.isPresent("reactor.netty.http.client.
HttpClient", loader);
        reactorNetty2ClientPresent = ClassUtils.isPresent("reactor.netty5.http.client.
HttpClient", loader);
        jettyClientPresent = ClassUtils.isPresent("org.eclipse.jetty.client.HttpClient",
loader);
        httpComponentsClientPresent =
                ClassUtils.isPresent("org.apache.hc.client5.http.impl.async.
CloseableHttpAsyncClient", loader) &&
```

```
                        ClassUtils.isPresent("org.apache.hc.core5.reactive.
ReactiveDataConsumer", loader);
    }

    @Override
    public WebClient build() {
        ClientHttpConnector connectorToUse =
                (this.connector != null ? this.connector : initConnector());
       ...
    }

    private ClientHttpConnector initConnector() {
        if (reactorNettyClientPresent) {
            return new ReactorClientHttpConnector();
        }
        else if (reactorNetty2ClientPresent) {
            return new ReactorNetty2ClientHttpConnector();
        }
        else if (jettyClientPresent) {
            return new JettyClientHttpConnector();
        }
        else if (httpComponentsClientPresent) {
            return new HttpComponentsClientHttpConnector();
        }
        else {
            return new JdkClientHttpConnector();
        }
    }

    ...

}
```

上述源码的含义是判断当前应用使用的是哪种响应式服务器配套的客户端，如果使用的是spring-boot-starter-webflux 启动器，则它默认使用的是 Reactor Netty，再根据使用的 Reactor Netty 版本来创建对应的 ClientHttpConnector 连接器。

> Spring Boot 为 Reactor Netty 带来了服务端和客户端的实现，如果使用了其他的响应式服务器，则建议使用相同的 Server 和 Client 技术，这样就能在 Server 和 Client 之间共享 HTTP 资源。

第 7 章
Spring Boot 数据访问

数据库几乎是所有 Java 应用都需要用到的技术，一般项目中主要有关系型数据库和非关系型数据库（NoSQL）的应用，而在 Spring Boot 框架中提供了对主流数据库的自动配置，可以直接拿来使用。本章介绍关系型数据库的使用方式，包括嵌入式数据库、数据源、连接池、事务管理、JdbcTemplate、Spring Data JPA 等常用的数据库技术，以及对常用框架 MyBatis、MyBatis-Plus 的集成使用，另外还会介绍几个常用的非关系型数据库的安装、集成和简单应用，主要包括 Redis、MongoDB、Elasticsearch。

7.1 概述

Spring Boot 支持关系型数据库相关组件的自动配置，包括数据库、连接池、事务等的自动配置，还提供了 JdbcTemplate 及 Spring Data JPA 数据库技术，极大地简化了关系型数据库的使用难度，帮助开发者快速集成和操作关系型数据库。

Spring Boot 还支持 H2、HSQL、Derby 等嵌入式数据库的自动配置，以及 MongoDB、Neo4J、Elasticsearch、Redis、GemFire or Geode、Cassandra、Couchbase、Couchbase、LDAP、InfluxDB 等常用的 NoSQL 数据库的自动配置。

对于其他不支持的 NoSQL，可以查找官网有没有提供对应的 Starter，如果没有就需要自己配置了。本章主要以常用的 Redis、MongoDB、Elasticsearch 为例来介绍 Spring Boot 是如何集成和使用的。

7.2 嵌入式数据库

嵌入式数据库相当于“内存形式的关系型数据库”，它在应用中只有一个 jar 文件，而没有其他的外部环境，所有数据都存储在内存中，也不能持久化数据，应用关闭就会丢弃所有数据。

嵌入式数据库最大的好处就是足够轻量，可以集成在应用中一起打包发布，不需要连接额外的数据库服务器，这样就可以快速存储少量的结构化数据，省去了大量的连接资源消耗。

使用嵌入式数据库不需要指定数据库 URL 的连接地址，只需要导入对应数据库的依赖即可，比如：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.hsqldb</groupId>
    <artifactId>hsqldb</artifactId>
    <scope>runtime</scope>
</dependency>
```

嵌入式数据库在实际工作中很少使用，一般可用于演示 Demo 项目，以及单元测试时需要临时数据库的场景，因为不能持久化数据，所以也不可能将嵌入式数据库用于实际生产项目，实际生产项目中还是要使用 MySQL 等专业的持久化关系型数据库。

7.3 数据源

7.3.1 概述

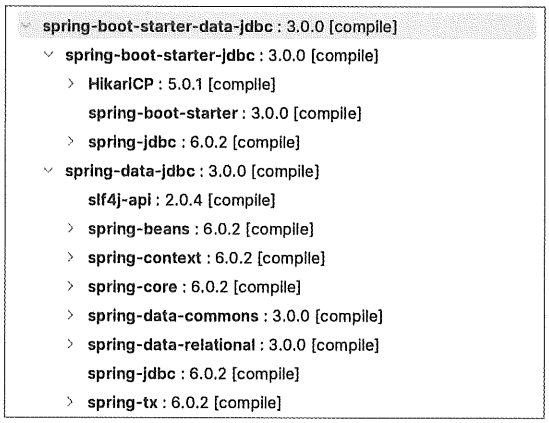
Java 是通过 javax.sql.DataSource 数据源接口来连接关系型数据库的，它提供了连接和处理关系型数据库的一系列标准方法和规范。

Spring Boot 支持数据库数据源的自动配置，只需要导入 spring-boot-starter-data-jdbc 启动器依赖及对应的数据库驱动依赖即可，如下面的依赖配置：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jdbc</artifactId>
```

```
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
</dependency>
```

spring-boot-starter-data-jdbc 的相关依赖如下图所示。



spring-boot-starter-data-jdbc 包含一个默认的 HikariCP 连接池，以及连接 JDBC 需要的相关依赖，但是，具体要使用的数据库驱动是需要额外导入的，上面笔者使用的是 MySQL 的连接驱动，如果需要换成其他数据库，那么替换这个驱动依赖即可。

根据 Spring Boot 的自动配置规则可以知道，数据源的自动配置类为 DataSourceAutoConfiguration，和别的自动配置类一样，在 Spring Boot 3.0.0 中，它已经被移到了新的 org.springframework. boot. autoconfigure.AutoConfiguration.imports 自动配置文件中，数据源的自动配置类如下列核心源码所示。

```
@AutoConfiguration(before = SqlInitializationAutoConfiguration.class)
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@ConditionalOnMissingBean(type = "io.r2dbc.spi.ConnectionFactory")
@EnableConfigurationProperties(DataSourceProperties.class)
@Import(DataSourcePoolMetadataProvidersConfiguration.class)
public class DataSourceAutoConfiguration {

    @Configuration(proxyBeanMethods = false)
    @Conditional(EmbeddedDatabaseCondition.class)
    @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
    @Import(EmbeddedDataSourceConfiguration.class)
    protected static class EmbeddedDatabaseConfiguration {
```

```
    }

    @Configuration(proxyBeanMethods = false)
    @Conditional(PooledDataSourceCondition.class)
    @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
    @Import({ DataSourceConfiguration.Hikari.class, DataSourceConfiguration.Tomcat.class,
            DataSourceConfiguration.Dbcp2.class, DataSourceConfiguration.OracleUcp.class,
            DataSourceConfiguration.Generic.class, DataSourceJmxConfiguration.class })
    protected static class PooledDataSourceConfiguration {

    }

    ...

}
```

如数据源自动配置类使用的 @ConditionalOnClass 注解所示，如果类路径下有以下两个类，就会完成自动配置：

- DataSource.class。
- EmbeddedDatabaseType.class。

优先配置外部化关系型数据库的 DataSource 数据源，再尝试配置嵌入式数据库，如果没有自定义的数据源，就会通过导入的连接池配置类进行默认配置。

如自动配置类上的注解所示，数据源绑定配置类为 DataSourceProperties，通过一系列的 spring.datasource.* 数据源配置参数即可完成自动配置。

```
spring:
  datasource:
#    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://localhost:3306/javastack
    username: root
    password: 12345678
```

数据库驱动类参数 driver-class-name 可以不用指定，主流的数据库 Spring Boot 都可以进行自动推断，更多的配置参数可以参考 DataSourceProperties 参数类，另外还有特定于具体连接池的参数，比如 spring.datasource.hikari.* 等。

应用启动后可以看到连接池完成初始化的相关日志：

```
HikariPool-1 - Starting...
```

```
HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@1376883
HikariPool-1 - Start completed.
```

7.3.2 自定义数据源

如果想自定义数据源，则直接注册一个 DataSource 数据源实现类的 Bean 即可，根据 Spring Boot 自动配置类使用的条件注解的配置规则，如果应用已注册相关数据源实例，那么数据源自动配置类中的相关连接池的 DataSource 数据源就不会再自动配置了，如下面的示例所示。

```
@Configuration(proxyBeanMethods = false)
public class DsConfig {

    @Bean
    @ConfigurationProperties(prefix = "spring.datasource")
    public DataSource dataSource() {
        return new XxxDataSource();
    }

}
```

还可以自定义多个数据源，如下面的示例所示。

```
@Configuration
public class DsConfig {

    @Primary
    @Bean
    @ConfigurationProperties("spring.datasource.xx.one")
    public DataSource dataSource1() {
        return new XxxDataSource();
    }

    @Bean
    @ConfigurationProperties("spring.datasource.xx.two")
    public DataSource dataSource2() {
        return new XxxDataSource();
    }

}
```

需要注意的是：

如果应用同时出现多个相同类型的 Bean，则需要使用 @Primary 注解定义一个主 Bean，否则在

根据类型注入 DataSource 的地方就会产生 Bean 注入冲突错误。

自定义多个数据源一般用于小型项目读写分离的场景，然后通过实现抽象类的 AbstractRoutingDataSource#determineCurrentLookupKey 方法决定选择哪个数据源，在大型项目中一般会通过第三方的数据库中间件技术来实现。

7.4 连接池

7.4.1 概述

上一节介绍了 Spring Boot 默认使用的是 HikariCP 连接池，Spring Boot 推荐和优先选择使用 HikariCP 连接池，因为它性能好、并发高，所以如果使用的是以下两个官方的启动器依赖：

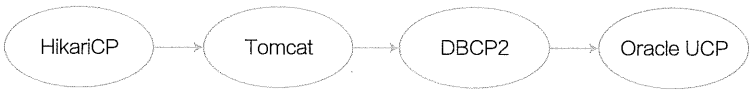
- spring-boot-starter-jdbc。
- spring-boot-starter-data-jpa。

那么它们默认使用的就是 HikariCP，再回到上面的 DataSourceAutoConfiguration 数据源自动配置类源码，进入其 PooledDataSourceConfiguration 配置类导入的 DataSourceConfiguration 类的核心源码，如下所示。

```
abstract class DataSourceConfiguration {

    ...

    /**
     * Hikari DataSource configuration.
     */
    @Configuration(proxyBeanMethods = false)
    @ConditionalOnClass(HikariDataSource.class)
    @ConditionalOnMissingBean(DataSource.class)
    @ConditionalOnProperty(name = "spring.datasource.type", havingValue =
"com.zaxxer.hikari.HikariDataSource",
            matchIfMissing = true)
    static class Hikari {

        @Bean
        @ConfigurationProperties(prefix = "spring.datasource.hikari")
        HikariDataSource dataSource(DataSourceProperties properties) {
            HikariDataSource dataSource = createDataSource(properties, HikariDataSource.class);
```

```
            if (StringUtils.hasText(properties.getName())) {
                dataSource.setPoolName(properties.getName());
            }
            return dataSource;
        }

    }

    ...

}
```

Spring Boot 在这里提供了常用的几个连接池的自动配置，比如 HikariCP、Tomcat、DBCP2 等，因为默认导入的是 HikariCP 连接池依赖，所以默认使用的是 HikariCP 连接池，如果没有 HikariCP 连接池依赖，则根据类路径下的包按顺序选择对应的连接池，选择连接池的顺序如下图所示。



如果以上连接池依赖都不存在，则使用非连接池的通用方案，即生成并返回一个对应数据库的默认 DataSource 数据源实例，如下面的 DataSourceConfiguration.Generic 配置类的源码所示。

```
abstract class DataSourceConfiguration {

    ...

    @Configuration(proxyBeanMethods = false)
    @ConditionalOnMissingBean(DataSource.class)
    @ConditionalOnProperty(name = "spring.datasource.type")
    static class Generic {

        @Bean
        DataSource dataSource(DataSourceProperties properties) {
            return properties.initializeDataSourceBuilder().build();
        }

    }

    ...

}
```

如果不想通过自动检测方式使用连接池，也可以通过 spring.datasource.type 参数指定具体要使用的连接池，如以下配置示例所示。

```
spring:
  datasource:
    type: org.apache.commons.dbcp2.BasicDataSource
```

7.4.2 使用 Druid 连接池

在国内，笔者推荐使用阿里开源的 Druid 连接池，这是一个为监控而生的连接池，它提供了强大的监控和扩展功能，对于那些需要监控 SQL 执行性能的项目就显得尤为重要。

Druid 连接池也提供了一站式集成的 Starter：

```
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>1.2.11</version>
</dependency>
```

不过它不是 Spring Boot 官方提供的，是 Druid 自行提供的，这个从 Starter 的命名也可以看出来，集成后再重启应用，如下面的启动日志所示。

```
com.alibaba.druid.pool.DruidDataSource   : {dataSource-1} inited
```

从启动日志可以发现已经切换到 Druid 连接池了，该 Starter 提供了 Druid 连接池的自动配置，对应的自动配置类为 DruidDataSourceAutoConfigure，如果不存在 DataSource 数据源的实例，则创建一个 Druid 自己的数据源，下面是它的核心源码：

```
@Configuration
@ConditionalOnClass(DruidDataSource.class)
@AutoConfigureBefore(DataSourceAutoConfiguration.class)
@EnableConfigurationProperties({DruidStatProperties.class, DataSourceProperties.class})
@Import({DruidSpringAopConfiguration.class,
    DruidStatViewServletConfiguration.class,
    DruidWebStatFilterConfiguration.class,
    DruidFilterConfiguration.class})
public class DruidDataSourceAutoConfigure {
    private static final Logger LOGGER = LoggerFactory.getLogger
(DruidDataSourceAutoConfigure.class);

    @Bean(initMethod = "init")
```

```
    @ConditionalOnMissingBean
    public DataSource dataSource() {
        LOGGER.info("Init DruidDataSource");
        return new DruidDataSourceWrapper();
    }
}

@ConfigurationProperties("spring.datasource.druid")
public class DruidDataSourceWrapper extends DruidDataSource implements InitializingBean {

    @Autowired
    private DataSourceProperties basicProperties;

    @Override
    public void afterPropertiesSet() throws Exception {
        //if not found prefix 'spring.datasource.druid' jdbc properties , 'spring.datasource'
prefix jdbc properties will be used.
        if (super.getUsername() == null) {
            super.setUsername(basicProperties.determineUsername());
        }
        if (super.getPassword() == null) {
            super.setPassword(basicProperties.determinePassword());
        }
        if (super.getUrl() == null) {
            super.setUrl(basicProperties.determineUrl());
        }
        if (super.getDriverClassName() == null) {
            super.setDriverClassName(basicProperties.getDriverClassName());
        }
    }

    ...

}
```

可以使用 spring.datasource.druid.* 参数完成一系列自动配置，同时注入 Spring Boot 的数据源通用配置参数类 DataSourceProperties，如果 Druid 没有配置数据源参数（username、password、url、driverClassName），则再取 Spring Boot 通用的数据源配置参数，这也是为什么集成 Druid 之后什么都不用修改就可以正常使用的原因。

7.5 数据库初始化

Spring Boot 可以在应用启动时初始化数据库，包括 DDL、DML 操作，但在 Spring Boot 2.5 中，又把数据库初始化逻辑重构了，最新的自动配置类为 SqlInitializationAutoConfiguration，这个类同样注册在新的自动配置文件中，以下是 SqlInitializationAutoConfiguration 自动配置类的源码：

```
@AutoConfiguration
@EnableConfigurationProperties(SqlInitializationProperties.class)
@Import({ DatabaseInitializationDependencyConfigurer.class, R2dbcInitializationConfiguration.class,
        DataSourceInitializationConfiguration.class })
@ConditionalOnProperty(prefix = "spring.sql.init", name = "enabled", matchIfMissing = true)
@Conditional(SqlInitializationModeCondition.class)
public class SqlInitializationAutoConfiguration {

    static class SqlInitializationModeCondition extends NoneNestedConditions {

        SqlInitializationModeCondition() {
            super(ConfigurationPhase.PARSE_CONFIGURATION);
        }

        @ConditionalOnProperty(prefix = "spring.sql.init", name = "mode", havingValue = "never")
        static class ModeIsNever {

        }

    }

}
```

对应的参数绑定类为 SqlInitializationProperties，可以通过一系列 spring.sql.init.* 参数配置数据库的初始化，如下面的示例所示。

```
spring:
  datasource:
    ...
  sql:
    init:
      mode: ALWAYS
      continueOnError: true
      schemaLocations:
        - classpath:sql/create_t_user.sql
```

```
    dataLocations:
      - classpath:sql/insert_t_user.sql
```

配置参数说明如下：

- mode：数据库初始化模式。
- continueOnError：初始化错误时是否继续启动。
- schemaLocations：需要执行初始化的数据库 DDL 脚本。
- dataLocations：需要执行初始化的数据库 DML 脚本。

初始化模式（mode）支持的选项可以参考 DatabaseInitializationMode 枚举类，如下所示。

- ALWAYS。
- EMBEDDED（默认）。
- NEVER。

因为默认为 EMBEDDED，即只对嵌入式数据库自动初始化，因此如果使用外部关系型数据库，那么首次启动时需要切换为 ALWAYS，成功初始化之后需要关闭，不然会重复插入数据。如果有唯一主键就会产生冲突导致启动失败，虽然可以通过 continueOnError 参数设置发生错误后继续启动，但始终会影响应用的启动速度。

在资源目录下分别创建两个脚本，Spring Boot 启动时就会运行这些脚本，脚本内容如下所示。

sql/create_t_user.sql：

```
DROP TABLE IF EXISTS `t_user`;
CREATE TABLE `t_user` (
  `id` int NOT NULL AUTO_INCREMENT,
  `username` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `phone` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `create_time` datetime NOT NULL,
  `status` int NOT NULL DEFAULT '1' COMMENT '-1: 禁用, 1: 正常, 0: 初始 ',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE KEY `idx_username` (`username`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

sql/insert_t_user.sql：

```
BEGIN;
INSERT INTO `t_user` VALUES (1, 'Lily', '18800000001', NOW(), 1);
INSERT INTO `t_user` VALUES (2, 'Jod', '18800000002', NOW(), 1);
```

```
INSERT INTO `t_user` VALUES (3, 'Jack', '18800000003', NOW(), 1);
INSERT INTO `t_user` VALUES (4, 'Sherry', '18800000004', NOW(), 1);
INSERT INTO `t_user` VALUES (5, 'James', '18800000005', NOW(), 1);
COMMIT;
```

然后启动应用，验证表是否创建，数据是否插入，如下所示。

```
mysql> desc t_user;
+-------------+-------------+------+-----+---------+----------------+
| Field       | Type        | Null | Key | Default | Extra          |
+-------------+-------------+------+-----+---------+----------------+
| id          | int         | NO   | PRI | NULL    | auto_increment |
| username    | varchar(50) | NO   | UNI | NULL    |                |
| phone       | varchar(20) | NO   |     | NULL    |                |
| create_time | datetime    | NO   |     | NULL    |                |
| status      | int         | NO   |     | 1       |                |
+-------------+-------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

可以看到表在 Spring Boot 应用启动后成功创建了，数据也成功插入了。

这就是 Spring Boot 数据库初始化的目的，实际项目中可能不被允许这样做，任何 SQL 脚本的执行都需要运维人员操作或经授权后处理，但在某些场景下可能会很有用，比如非正式项目中需要进行的数据库操作演示、测试等场景，后面的 JdbcTemplate 章节就需要用到数据库临时操作。

7.6 事务管理

7.6.1 概述

Spring Boot 提供了事务的自动配置，spring-boot-starter-data-jdbc 启动器会引入事务相关的包，相关的事务自动配置类有以下几个：

- TransactionAutoConfiguration。
- DataSourceTransactionManagerAutoConfiguration。

它们同样注册在新的自动配置文件中，前者是事务相关的自动配置，后者是事务管理器的自动配置，对应的参数绑定类为 TransactionProperties，通过一系列 spring.transaction.* 参数可以自定义一些事务配置。

如果没有自定义的事务需求，那么使用默认自动配置的事务就相当简单，只需要在代理方法上使用 @Transactional 事务注解即可，比如下面的示例：

```
@RequiredArgsConstructor
@Service
public class UserDaoImpl implements UserDao {

    public final JdbcTemplate jdbcTemplate;

    @Transactional
    @Override
    public void update() {
        jdbcTemplate.execute("update t_user set username = 'Petty' where id = 1");
        jdbcTemplate.execute("update t_user set username = 'Yoga' where id = 2");
        throw new RuntimeException("test exception");
    }

}
```

这里连续执行了两条更新语句，再手动抛出一个异常，用于测试事务回滚，当异常抛出后事务就会回滚，然后把异常代码注释后，两条语句正常更新。

7.6.2 事务失效的场景

前面的事务测试为什么要单独创建一个 DAO 实现类呢？这是为了能启用 Spring 中的代理，使用 @Transactional 事务注解必须注意下面几个失效的场景。

1. 数据库引擎不支持事务

这里以 MySQL 为例，MyISAM 引擎是不支持事务操作的，一般要支持事务都会使用 InnoDB 引擎，根据 MySQL 的官方文档说明，从 MySQL 5.5.5 开始的默认存储引擎是 InnoDB，之前默认的都是 MyISAM，所以这一点要值得注意，如果底层引擎不支持事务，那么再怎么设置也没有用。

2. 没有被 Spring 管理

示例如下：

```
// @Service
public class OrderServiceImpl implements OrderService {

    @Transactional
```

```
    public void updateOrder(Order order) {
        // update order
    }

}
```

如果此时把 @Service 注解注释掉，那么这个类就不会被加载成一个 Bean，这个类就不会被 Spring 管理了，事务自然就失效了。

3. 方法不是 public 的

@Transactional 注解只能用于 public 的方法上，否则事务不会生效，如果要用在非 public 的方法上，则可以开启基于 AspectJ 框架的静态代理模式。

4. 发生自身调用

示例如下：

```
@Service
public class OrderServiceImpl implements OrderService {

    public void update(Order order) {
        updateOrder(order);
    }

    @Transactional
    public void updateOrder(Order order) {
        // update order
    }

}
```

update 方法上面没有加 @Transactional 注解，如果调用有 @Transactional 注解的 updateOrder 方法，那么 updateOrder 方法上的事务还可以生效吗？读者可以先想一想，后面会揭晓答案。

再来看下面这个例子：

```
@Service
public class OrderServiceImpl implements OrderService {

    @Transactional
    public void update(Order order) {
        updateOrder(order);
```

```
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void updateOrder(Order order) {
        // update order
    }

}
```

这次在 update 方法上加了 @Transactional，如果在 updateOrder 上加了 REQUIRES_NEW 新开启一个事务，那么新开启的事务可以生效吗？

这两个例子中的事务都不会生效，因为它们发生了自身调用，就调用了该类自己的方法，而没有经过 Spring 的代理类，默认只有调用外部代理类的方法，事务才会生效，这也是老生常谈的问题了。

这个问题的解决方案之一就是在事务所在的类中注入自己，用注入的对象再调用另外一个方法，这个不太优雅，在 Spring 中可以在当前线程中暴露并获取当前代理类，通过在启动类上添加以下注解来启用暴露代理类，如下面的示例所示。

```
@EnableAspectJAutoProxy(exposeProxy = true)
```

然后通过以下代码获取当前代理类，并调用代理类的事务方法：

```
((OrderService) AopContext.currentProxy()).updateOrder();
```

Spring 默认只有调用 Spring 代理类的 public 方法，事务才能生效。

5. 没有配置事务管理器

如果没有配置以下 DataSourceTransactionManager 数据源事务管理器，那么事务也不会生效：

```
@Bean
public PlatformTransactionManager transactionManager(DataSource dataSource) {
    return new DataSourceTransactionManager(dataSource);
}
```

但在 Spring Boot 中只要引入了 spring-boot-starter-data-jdbc 启动器依赖就会自动配置 DataSourceTransactionManager 数据源事务管理器，所以 Spring Boot 框架不存在这个问题，但在传统的 Spring 框架中需要注意。

6. 设置了不支持事务

示例如下：

```
@Service
public class OrderServiceImpl implements OrderService {

    @Transactional
    public void update(Order order) {
        updateOrder(order);
    }

    @Transactional(propagation = Propagation.NOT_SUPPORTED)
    public void updateOrder(Order order) {
        // update order
    }

}
```

这里的 Propagation.NOT_SUPPORTED 表示当前方法不以事务方式运行，当前若存在事务则挂起，这就是主动不支持以事务方式运行了。

7. 异常没有被抛出

示例如下：

```
// @Service
public class OrderServiceImpl implements OrderService {

    @Transactional
    public void updateOrder(Order order) {
        try {
            // update order
        } catch {

        }
    }

}
```

这个方法把异常给捕获了，但没有抛出来，所以事务不会回滚，只有捕捉到异常事务才会生效。

8. 异常类型不匹配

示例如下：

```
// @Service
public class OrderServiceImpl implements OrderService {

    @Transactional
    public void updateOrder(Order order) {
        try {
            // update order
        } catch {
            throw new Exception("更新错误");
        }
    }

}
```

因为 Spring 默认回滚的是 RuntimeException 异常，和程序抛出的 Exception 异常不匹配，所以事务也是不生效的。如果要触发默认 RuntimeException 之外异常的回滚，则需要在 @Transactional 事务注解上指定异常类，示例如下：

```
@Transactional(rollbackFor = Exception.class)
```

本节总结了使用 @Transactional 注解导致事务失效的几个常见场景，如果 @Transactional 事务不生效，则可以根据这几种情形排查一下，其实次数最多的也就是发生自身调用、异常被捕获、异常抛出类型不匹配这几种场景。

7.7 JdbcTemplate

7.7.1 数据库操作

JDBC 的全称为 Java Database Connectivity，即 Java 数据库连接，是 Java 语言规范应用访问数据库的接口，从而不需要关心底层特定数据库的细节，提供了如查询、更新、删除数据库中的数据等方法。

JdbcTemplate 即 JDBC 模板，是 Spring 用于简化 JDBC 操作的包装模板类，只需要导入 spring-boot-starter-data-jdbc 启动器依赖即可，它在新的自动配置文件中同样提供了对 JdbcTemplateAutoConfiguration 自动配置类的注册，以下是该自动配置类的源码：

```
@AutoConfiguration(after = DataSourceAutoConfiguration.class)
@ConditionalOnClass({ DataSource.class, JdbcTemplate.class })
@ConditionalOnSingleCandidate(DataSource.class)
@EnableConfigurationProperties(JdbcProperties.class)
@Import({ DatabaseInitializationDependencyConfigurer.class, JdbcTemplateConfiguration.class,
        NamedParameterJdbcTemplateConfiguration.class })
public class JdbcTemplateAutoConfiguration {

}

@Configuration(proxyBeanMethods = false)
@ConditionalOnMissingBean(JdbcOperations.class)
class JdbcTemplateConfiguration {

    @Bean
    @Primary
    JdbcTemplate jdbcTemplate(DataSource dataSource, JdbcProperties properties) {
        JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
        JdbcProperties.Template template = properties.getTemplate();
        jdbcTemplate.setFetchSize(template.getFetchSize());
        jdbcTemplate.setMaxRows(template.getMaxRows());
        if (template.getQueryTimeout() != null) {
            jdbcTemplate.setQueryTimeout((int) template.getQueryTimeout().getSeconds());
        }
        return jdbcTemplate;
    }

}
```

JdbcTemplateAutoConfiguration 自动配置类中导入了 JdbcTemplateConfiguration 配置类，它提供了默认的 JdbcTemplate 实例，在应用中直接注入就能使用，示例如下：

```
@Slf4j
@RequiredArgsConstructor
@SpringBootApplication
public class Application {
```

```
    public final JdbcTemplate jdbcTemplate;

    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

    @Bean
    public CommandLineRunner commandLineRunner() {
        return (args) -> {
            String username = jdbcTemplate.queryForObject("select username from t_user
where id = 2",
                    String.class);
            log.info("query username is : {}", username);
        };
    }


}
```

这里的逻辑是，在应用启动之后，在数据库中查询 id=2 的 username 字段的值并输出日志。

7.7.2 自定义 JdbcTemplate

如果默认注册的 JdbcTemplate 实例不符合要求，则可以自定义 JdbcTemplate，或者通过一系列 spring.jdbc.template.* 参数进行定制，如下面的示例所示。

```
spring:
  jdbc:
    template:
      max-rows: 3
```

这里对 JdbcTemplate 进行了定制，最多可返回 3 条记录，然后新增一个查询操作测试一下，如下面的示例所示。

```
@Bean
public CommandLineRunner commandLineRunner() {
    return (args) -> {
        ...
```

```
        List<Map<String, Object>> list = jdbcTemplate.queryForList("select id from t_user");
        log.info("total list: {}", list.size());
    };
}
```

重启应用会输出以下日志：

```
query username is : Jod
total list: 3
```

虽然用户表中有超过3条的记录，但是JdbcTemplate只返回了最多3条记录。更多的JdbcTemplate定制参数可以参考JdbcProperties#Template参数配置类。

7.8 Spring Data JPA

7.8.1 概述

很多人可能对JPA相关技术有误解，下面先对相关技术术语做一个梳理。

1. JPA

JPA其实不是指具体框架，它是Java Persistence API的简称，是Java提供的持久化API，它为POJO提供持久化的标准规范，可以把Java对象映射为数据库中的记录。

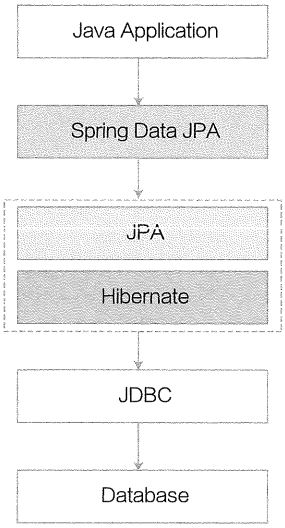
2. Hibernate

JPA是持久化规范，而Hibernate、TopLink、OpenJPA等都是JPA这种标准规范的实现，Hibernate则是Java最流行的JPA规范的实现框架。

3. Spring Data JPA

Spring Data系列项目为数据访问提供基于Spring的统一编程模型，而其中之一的Spring Data JPA则是Spring为了简化JPA的使用而推出的一个项目，底层使用的是Hibernate框架，可以将Spring Data JPA理解为对Hibernate更上一层的封装。

三者的关系如下图所示。



Spring Data JPA 框架是对 JPA 顶层的封装而已，底层使用的是基于 JPA 规范的 Hibernate 框架，最后通过 JDBC 连接底层数据库。

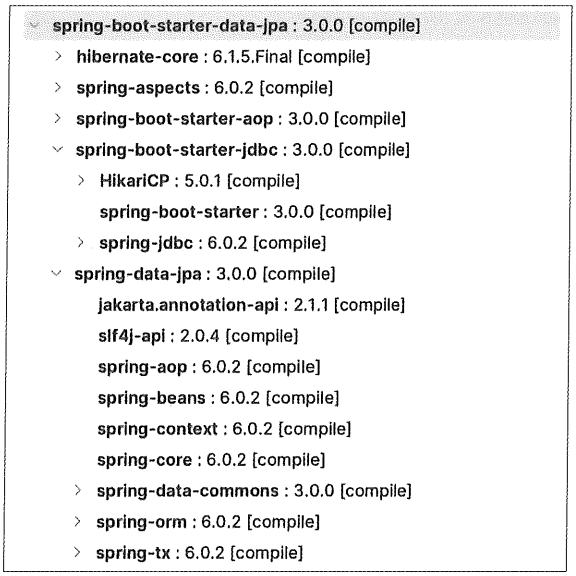
7.8.2 数据库操作

Spring Data JPA 框架简化了 JPA 的复杂操作，Spring Boot 又提供了 spring-boot-starter-data- jpa 启动器，自动配置了 JPA 相关组件，让开发者可以更加快速地集成和使用 JPA。

首先在应用中导入该启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

然后查看它导入的相关依赖，如下图所示。



spring-boot-starter-data-jpa 包含了 JPA 访问数据库所需要的所有依赖，另外，在新的自动配置文件中注册了以下 JPA 自动配置类：

- JpaRepositoriesAutoConfiguration。
- HibernateJpaAutoConfiguration。

对应的参数绑定类为 JpaProperties 和 HibernateProperties，通过一系列 spring.jpa.* 和 spring.jpa.hibernate.* 参数可以自定义 JPA 配置，比如以下常用的两个配置：

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

这里指定了 ddl-auto 为 update 更新模式，并且输出 SQL 语句日志，方便调试 SQL 问题，后面的章节中会看到输出的 SQL 语句日志。

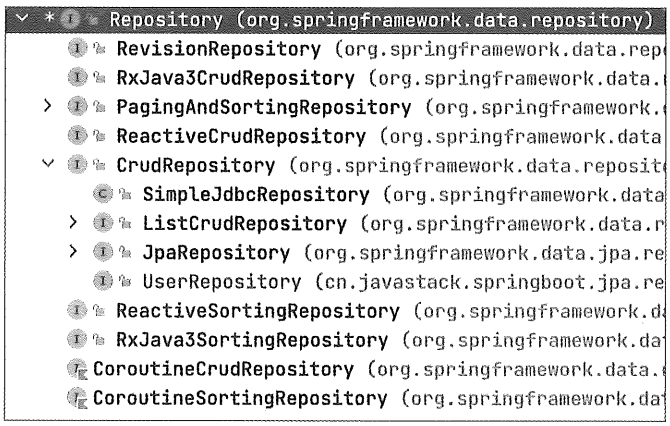
传统的 JPA 实体需要在 persistence.xml 文件中指定，而在 Spring Boot 中就没有必要，实体类上面只需要一个 @Entity 注解，启动类上有 @SpringBootApplication 或者 @EnableAutoConfiguration 注解就能自动扫描所有 @Entity 实体类，如下面的用户表的实体类所示。

```
@Data
@Entity(name = "t_user")
```

```
public class UserDO implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String phone;

    @Column(name = "create_time", nullable = false)
    private Date createTime;

    @Column(nullable = false)
    private Integer status;

}
```

如果自动配置注解扫描不到，则可以使用 @EntityScan 注解指定需要扫描的实体类所在的包，或者直接指定实体类。

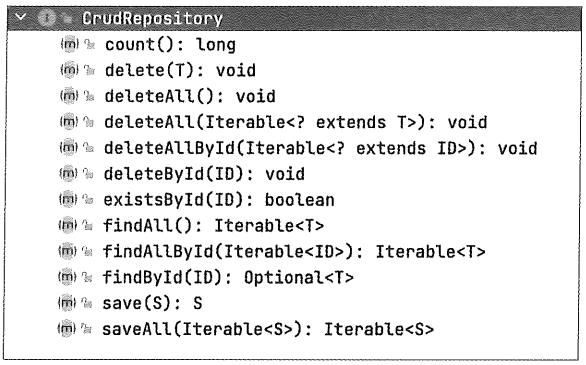
Spring Data JPA 的数据库操作被称为 repositories，Repository 是所有 repositories 的顶级接口，它是一个空接口，官方还提供了几个常用的 Repositorie，如下图所示。



这些 Repositorie 接口一般可以满足常用的需求，项目中的数据库操作可以根据需要选择对应的 Repository 继承 / 扩展即可，比如下面的创建一个用户 UserRepository 的示例：

```
public interface UserRepository extends CrudRepository<UserDO, Long> {

}
```

UserRepository 接口继承了 CrudRepository 接口，CrudRepository 接口又继承自 Repository 接口，CrudRepository 接口提供了常用的增删改查操作方法，如果有更复杂的操作，则在自己的 Repository 接口中再额外定义即可，如下图所示。

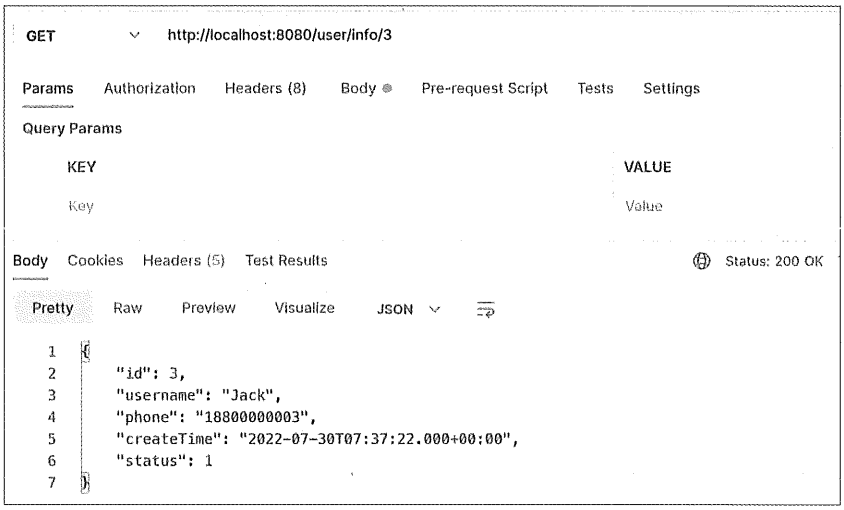


UserRepository 接口定义后也会被自动扫描到，可以直接拿来注入使用，如果自动配置注解扫描不到，则可以使用 @EnableJpaRepositories 注解指定 Repository 所在的包，或者直接指定 Repository 类。

这里为了方便测试，直接在 UserController 控制器类中注入 UserRepository，并调用其 findById 方法查询返回用户表的某条记录，如以下示例所示。

```
@RequiredArgsConstructor
@RestController
public class UserController {

    private final UserRepository userRepository;

    @GetMapping("/user/info/{id}")
    public UserDO getUserInfo(@PathVariable("id") long id){
        UserDO userDO = userRepository.findById(id).orElseGet(null);
        return userDO;
    }

}
```

重启应用，再调用查询用户接口，如下图所示。



结果正常返回，控制台也会打印相关 SQL 语句：

```
Hibernate: select u1_0.id,u1_0.create_time,u1_0.phone,u1_0.status,u1_0.username from
t_user u1_0 where u1_0.id=?
```

Spring Data JPA 的使用绝不仅限于此，一般的应用使用 Spring Data JPA 即可满足需求，还可以有效提升效率。如果是复杂的业务系统，涉及大量复杂的 SQL 语句，则建议使用 MyBatis 框架。MyBatis 是把 SQL 语句配置在 XML 文件中，编写灵活，易于调试和对 SQL 调优，另外加上 JPA 有一定的使用门槛，需要团队成员熟练掌握，所以国内大部分公司主要还是使用 MyBatis 框架。

7.9 MyBatis

7.9.1 概述

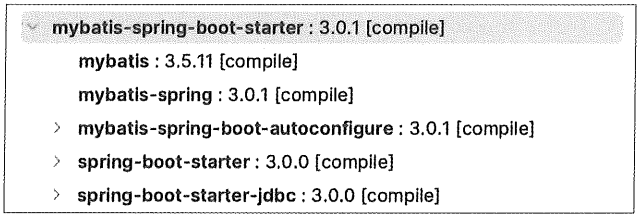
国内一般都使用 MyBatis 框架，可以直接写 SQL，上手容易，维护也方便。

Spring Boot 官方没有提供 MyBatis Starter，但 MyBatis 官方提供了 mybatis-spring-boot-starter 一站式启动器，如以下依赖配置所示。

```
<dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
```

```
    <version>3.0.1</version>
</dependency>
```

在 Spring Boot 3.0.0 发布不久后，MyBatis Starter 也进行了兼容性适配，Spring Boot 3.0.0 对应的 mybatis-spring-boot-starter 版本为 3.x，它导入了使用 MyBatis 所需的所有依赖，如下图所示。



Starter 包含 Spring Boot 使用 MyBatis 时所需的自动配置，对应的自动配置类为 MybatisAutoConfiguration，它代替了传统 Spring 项目集成 MyBatis 所需的 XML 配置文件，自动配置类中主要注册了一个 SqlSessionFactory 和 SqlSessionTemplate 实例，对应的参数绑定配置类为 MybatisProperties，通过一系列 mybatis.* 参数可以自定义 MyBatis 配置。

7.9.2 数据库操作

本节使用 Spring Boot 集成 MyBatis 实现一个简单的查询操作，使用 MyBatis 框架一般需要以下两个组件：

- *Mapper.java：数据库映射接口，提供对应的数据库操作方法。
- *Mapper.xml：数据库 SQL 映射配置文件，提供数据库映射接口对应执行的 SQL。

首先导入 mybatis-spring-boot-starter 启动器依赖，然后在应用配置文件中添加 MyBatis 配置参数，如下面常用的配置示例所示。

```
mybatis:
  mapper-locations: classpath:mapper/*.xml
  type-aliases-package: cn.javastack.springboot.mybatis.entity
```

配置参数说明如下：

- mapper-locations：指定 *.xml SQL 配置文件的路径。
- type-aliases-package：指定数据库实体类的全包路径，方便在 SQL 配置文件中使用实体类时不用写包名。

然后定义一个 *Mapper 映射接口，接口需要使用 @Mapper 注解修饰，如以下 UserMapper 类的

代码所示。

```
@Mapper
public interface UserMapper {

    UserDO findById(@Param("id") long id);

}
```

UserDO 实体类的代码如下所示。

```
@Data
public class UserDO implements Serializable {

    private static final long serialVersionUID = 1L;

    private Integer id;

    private String username;

    private String phone;

    private Date createTime;

    private Integer status;

}
```

然后在资源目录下新建对应的 mapper/UserMapper.xml 配置文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="cn.javastack.springboot.mybatis.mapper.UserMapper">

    <resultMap id="BaseResultMap" type="UserDO">
            <id property="id" column="id" jdbcType="INTEGER"/>
            <result property="username" column="username" jdbcType="VARCHAR"/>
            <result property="phone" column="phone" jdbcType="VARCHAR"/>
            <result property="createTime" column="create_time" jdbcType="TIMESTAMP"/>
            <result property="status" column="status" jdbcType="INTEGER"/>
    </resultMap>
```

```
    <sql id="Base_Column_List">
        id,username,phone,create_time,status
    </sql>

    <select id="findById" parameterType="long" resultMap="BaseResultMap">
        select
            <include refid="Base_Column_List" />
        from t_user
        where id = #{id}
    </select>


</mapper>
```

XML 配置说明如下：

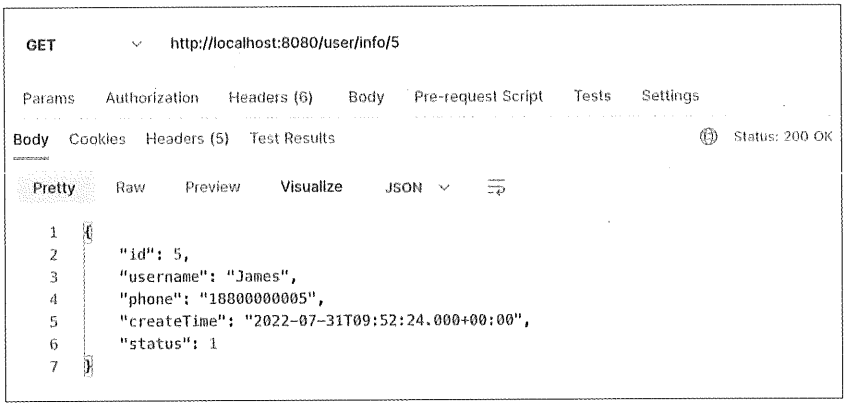
- namespace：定义命名空间，即 Mapper 映射接口的全路径，比如 cn.javastack.springboot.mybatis.mapper.UserMapper。
- resultMap：定义结果映射集，将数据库表字段映射为 Java 实体类。
- sql：定义通用的 SQL 语句，可包含在其他 SQL 语句中。
- select：定义查询语句，id 要和 Mapper 映射接口中的方法一致。

如果 *Mapper 在启动类所在的子包中，那么 MyBatis 自动配置会自动扫描该注解修饰的接口，应用可以直接注入使用，否则需要使用 @MapperScan 注解实现自定义的 Mapper 扫描。

这里为了方便测试，直接在 UserController 控制器类中注入 UserMapper，并调用其 findById 方法查询并返回用户表的某条记录，如以下代码所示。

```
@RequiredArgsConstructor
@RestController
public class UserController {

    private final UserMapper userMapper;

    @GetMapping("/user/info/{id}")
    public UserDO getUserInfo(@PathVariable("id") long id) {
        UserDO userDO = userMapper.findById(id);
        return userDO;
    }

}
```

启动应用测试一下，如下图所示。



7.10 MyBatis-Plus

7.10.1 概述

MyBatis-Plus 是针对 MyBatis 的开源增强框架，它只是增强，而不改变 MyBatis 框架本身，在 MyBatis 框架的基础上提供了许多人性化的功能，比如通用的 Mapper、Service 接口，可以节省大量基本的 CRUD 操作，还有更多丰富的实用特性，它是使用 MyBatis 框架的最佳搭档。

开源地址为链接 7。

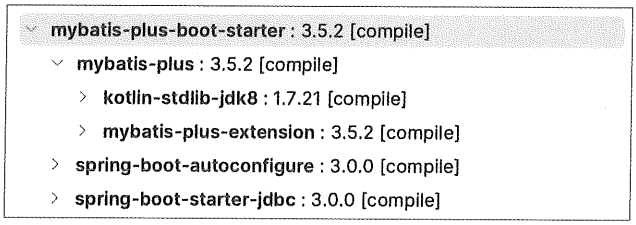
本节使用 Spring Boot 集成 MyBatis-Plus，并实现上一节使用 MyBatis 实现的同样的功能。

> 在笔者撰写本章及后续复核期间，MyBatis-Plus 最新只支持 Spring Boot 2.x，使用最新的 Spring Boot 3.0.0 会出现兼容性问题，所以本章基于 Spring Boot 2.7.0 进行实战演示。最新的 MyBatis-Plus 版本只是没有兼容 Spring Boot 3.x 而已，框架的底层架构和 API 是不会有大的改动的，所以不必担心。

MyBatis-Plus 官方也提供了 Starter 一站式启动器，如下面的依赖配置所示。

```
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>${mybatis-plus-boot-starter.version}</version>
</dependency>
```

导入 MyBatis-Plus 的依赖，它会自动引入使用 MyBatis 框架所需的所有依赖，如下图所示。



对应的自动配置类为 MybatisPlusAutoConfiguration，它其实就是代替了官方的 MyBatis Starter 及 MybatisAutoConfiguration 自动配置类，注册了一个 SqlSessionFactory 实例并对其进行定制，如下面的核心源码所示。

```
@Configuration(proxyBeanMethods = false)
@ConditionalOnClass({SqlSessionFactory.class, SqlSessionFactoryBean.class})
@ConditionalOnSingleCandidate(DataSource.class)
@EnableConfigurationProperties(MybatisPlusProperties.class)
@AutoConfigureAfter({DataSourceAutoConfiguration.class, MybatisPlusLanguageDriverAuto-
Configuration.class})
public class MybatisPlusAutoConfiguration implements InitializingBean {

    ...

    @Bean
    @ConditionalOnMissingBean
    public SqlSessionFactory sqlSessionFactory(DataSource dataSource) throws Exception {
        // TODO 使用 MybatisSqlSessionFactoryBean 而不是 SqlSessionFactoryBean
        MybatisSqlSessionFactoryBean factory = new MybatisSqlSessionFactoryBean();

        ...

        // TODO 修改源码支持定义 TransactionFactory
        this.getBeanThen(TransactionFactory.class, factory::setTransactionFactory);

        // TODO 对源码做了一定的修改（因为源码适配了老旧的 Mybatis 版本，但我们不需要适配）
        Class<? extends LanguageDriver> defaultLanguageDriver = this.properties.
getDefaultScriptingLanguageDriver();
        if (!ObjectUtils.isEmpty(this.languageDrivers)) {
            factory.setScriptingLanguageDrivers(this.languageDrivers);
        }
        Optional.ofNullable(defaultLanguageDriver).ifPresent(factory::
setDefaultScriptingLanguageDriver);
```

```
        applySqlSessionFactoryBeanCustomizers(factory);

        // TODO 此处必为非 NULL
        GlobalConfig globalConfig = this.properties.getGlobalConfig();
        // TODO 注入填充器
        this.getBeanThen(MetaObjectHandler.class, globalConfig::setMetaObjectHandler);
        // TODO 注入主键生成器
        this.getBeansThen(IKeyGenerator.class, i -> globalConfig.getDbConfig().
setKeyGenerators(i));
        // TODO 注入 SQL 注入器
        this.getBeanThen(ISqlInjector.class, globalConfig::setSqlInjector);
        // TODO 注入 ID 生成器
        this.getBeanThen(IdentifierGenerator.class, globalConfig::setIdentifierGenerator);
        // TODO 设置 GlobalConfig 到 MybatisSqlSessionFactoryBean
        factory.setGlobalConfig(globalConfig);
        return factory.getObject();
    }

    ...

}
```

对应的参数绑定类为 MybatisPlusProperties，通过一系列 mybatis-plus.* 参数可以对 Mybatis-Plus 进行定制，而 MyBatis 的自动配置参数为 mybatis.*，mybatis-plus.* 继承了 mybatis.* 的部分原生参数并进行了增强。

7.10.2 通用数据库操作

在应用配置文件中添加对应的 MyBatis-Plus 配置，如以下配置所示。

```
mybatis-plus:
  mapper-locations: classpath:mapper/*.xml
  type-aliases-package: cn.javastack.springboot.mybatis.entity
```

参数和 MyBatis一致，只需要把 mybatis 前缀改为 mybatis-plus 即可。

与 MyBatis 不同的是，MyBatis-Plus 会自动加载 mapper 资源目录下的 XML 映射文件，如果使用默认目录，则这里的 mapper-locations 可以不用定义，如下面的参数绑定类的源码所示。

```
@ConfigurationProperties(
    prefix = "mybatis-plus"
)
```

```
public class MybatisPlusProperties {
    private static final ResourcePatternResolver resourceResolver = new PathMatching-
ResourcePatternResolver();
    private String configLocation;
    private String[] mapperLocations = new String[]{"classpath*:/mapper/**/*.xml"};

    ...

}
```

还有一点与 MyBatis 框架不同的是，因为引入的不是 MyBatis 官方的 Starter，所以 MyBatis-Plus 中的 Mapper 映射类不能自动扫描注册，需要使用 @MapperScan 注解手动开启扫描，如下面的配置代码所示。

```
@MapperScan(basePackages = {"cn.javastack.springboot.mybatisplus.mapper"})
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

}
```

新建一个 UserMapper 接口，如下面的示例代码所示。

```
public interface UserMapper extends BaseMapper<UserDO> {

}
```

这里继承了 MyBatis-Plus 提供的通用 BaseMapper 接口，它提供了 DAO 层常用的 CRUD 操作，UserMapper 可以直接注入使用。

MyBatis-Plus 实体类与 MyBatis 也有不同，因为 MyBatis-Plus 不需要 XML 配置，所以需要使用一系列注解完成实体类字段的映射，MyBatis-Plus 提供了一系列注解对实体类进行定义、填充、约束等，如以下 UserDO 实体类的示例代码所示。

```
@TableName("t_user")
@Data
public class UserDO implements Serializable {

    @TableField()
```

```
    private static final long serialVersionUID = 1L;

    @TableId(type = IdType.AUTO)
    private Integer id;

    @TableField(insertStrategy = FieldStrategy.NOT_NULL)
    private String username;

    @TableField(insertStrategy = FieldStrategy.NOT_NULL)
    private String phone;

    @TableField(insertStrategy = FieldStrategy.NOT_NULL, fill = FieldFill.INSERT)
    private LocalDateTime createTime;

    @TableField(insertStrategy = FieldStrategy.NOT_NULL)
    private Integer status;

}
```

新建一个 UserService 接口，如以下示例代码所示。

```
public interface UserService extends IService<UserDO> {

}
```

这里继承了 MyBatis-Plus 提供的通用 IService 接口，它提供了 Serivice 层常用的 CRUD 操作，然后创建一个实现类实现该接口，并继承 MyBatis-Plus 提供的通用 ServiceImpl 实现，如以下示例代码所示。

```
@Slf4j
@RequiredArgsConstructor
@Service
public class UserServiceImpl
        extends ServiceImpl<UserMapper, UserDO>
        implements UserService {

}
```

然后直接在 UserController 控制器类中注入 UserService，并调用 IService 通用接口中的 getById 方法查询并返回用户表的某条记录，如以下示例代码所示。

```
@RequiredArgsConstructor
@RestController
```

```
public class UserController {

    private final UserService userService;

    @GetMapping("/user/info/{id}")
    public UserDO getUserInfoById(@PathVariable("id") long id) {
        UserDO userDO = userService.getById(id);
        return userDO;
    }


}
```

在 MyBatis 原生框架中，一个简单的操作也要写一套相关的代码，包括 Mapper 方法、SQL 映射语句、Service 接口方法及实现方法，而 MyBatis-Plus 提供的通用 Mapper、通用 IService、通用 ServiceImpl 实现可以节省大量的冗余代码。

7.10.3 自定义数据库操作

如果通用的接口不符合要求，也可以自定义数据库操作，包括直接编写 SQL 映射文件，或者使用 MyBatis-Plus 提供的 QueryWrapper 和 LambdaQueryWrapper、UpdateWrapper 实现无 XML 化操作。

下面实现了除通用接口外的其他两种处理方式：

```
@Slf4j
@RequiredArgsConstructor
@Service
public class UserServiceImpl
        extends ServiceImpl<UserMapper, UserDO> implements UserService {

    private final UserMapper userMapper;

    @Override
    public UserDO getByUsername(String username, int type) {
        if (type == 0) {
            // XML
            log.info("query from xml");
            return userMapper.selectByUsername(username);
        } else {
            // QueryWrapper
            log.info("query from wrapper");
            LambdaQueryWrapper<UserDO> queryWrapper = new LambdaQueryWrapper();
```

```
            queryWrapper.eq(UserDO::getUsername, username);
            queryWrapper.eq(UserDO::getStatus, 1);
            return userMapper.selectOne(queryWrapper);
        }

    }

}
```

其他基本代码略，完全版本见仓库源码。

第一种方式就是 MyBatis 原生的 XML 映射文件的处理方式，根据方法名称查询并运行 XML 映射文件中的 SQL 语句，这里不再赘述。

第二种方式是 MyBatis-Plus 提供的条件构造器，无须 XML 文件，无须编写 SQL 语句，通过条件构造器同样可以实现对一些简单 SQL 语句的处理。

以上介绍的 MyBatis-Plus 的相关功能已经很强大了，MyBatis-Plus 还有更多实用的功能，比如主键自动生成策略、字段自动填充、分页功能等，这里不再赘述。总之，MyBatis 结合 MyBatis-Plus，效率翻倍。

7.11 Redis

7.11.1 概述

Redis 的全称为 REmote DIctionary Server，它是一种 key-value 形式的 NoSQL 内存数据库，它最大的特性是将所有数据都存储在内存中，所以读写速度快、性能好，它也支持将内存中的数据持久化到硬盘中，这样就不怕数据会丢失，兼顾性能和安全。

Spring Boot 为 Redis 的集成及使用提供了许多便利，只需要简单的配置，基于 RedisTemplate 模板就能“开箱即用”。

7.11.2 Redis 环境搭建

1. 安装 Redis

Redis 支持多种操作系统的安装方式（官方不支持 Windows），包括 Docker 镜像，这里以 CentOS 为例进行演示。

首先从 Redis 官网下载最新稳定版本包，下载地址为链接 8。

下载后解压并切换到根目录进行编译安装，参考命令如下：

```
$ tar -zxvf redis-7.0.5.tar.gz
$ cd redis-7.0.5
$ make
$ make install
```

最后在 src 目录下安装下面几个核心命令的可执行文件：

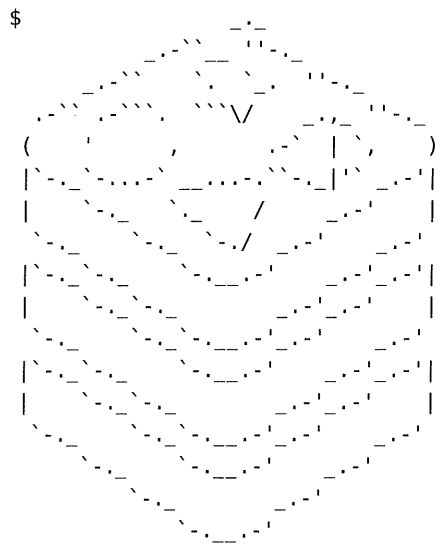
- redis-server（服务端程序）。
- redis-benchmark（基准性能测试）。
- redis-cli（客户端程序）。

2. 启动 Redis

运行 redis src 目录下的 redis-server 命令来启动 Redis 服务，参考命令如下：

```
$ ./redis-server ../redis.conf &
```

Redis 服务启动成功的画面如下所示。




```
... WARNING: The TCP backlog setting of 511 cannot be enforced because kern.ipc.
somaxconn is set to the lower value of 128.
```

```
... Server initialized
... Loading RDB produced by version 7.0.5
... RDB age 7 seconds
... RDB memory usage when created 1.04 Mb
... Done loading RDB, keys loaded: 0, keys expired: 0.
... DB loaded from disk: 0.000 seconds
... Ready to accept connections
```

上面显示了Redis启动的默认端口号（6379）、进程ID（62293），以及Redis使用的资源信息等。

3. 连接 Redis

运行 redis src 目录下的 redis-cli 命令来连接 Redis 服务，参考命令如下：

```
$ ./redis-cli
127.0.0.1:6379>
```

连接后可以使用 ping 命令测试 Redis 的连通性，参考命令如下：

```
$ ./redis-cli
127.0.0.1:6379> ping
PONG
```

PONG 表示连接 Redis 成功。

再进行一些简单的键值操作，参考命令如下：

```
127.0.0.1:6379> set test 123
OK
127.0.0.1:6379> get test
"123"
127.0.0.1:6379>
```

设置值和查询都没问题，说明连接和操作都正常了。

4. 关闭 Redis

为了避免数据丢失，不要使用 kill -9 方式关闭 Redis，建议使用 redis-cli shutdown 命令关闭 Redis，参考命令如下：

```
./redis-cli shutdown
62293:M 04 Nov 2022 16:33:26.572 # User requested shutdown...
62293:M 04 Nov 2022 16:33:26.572 * Saving the final RDB snapshot before exiting.
62293:M 04 Nov 2022 16:33:26.593 * DB saved on disk
```

```
62293:M 04 Nov 2022 16:33:26.593 * Removing the pid file.
62293:M 04 Nov 2022 16:33:26.594 # Redis is now ready to exit, bye bye...
[1] + 62293 done ./redis-server ../redis.conf
```

如以上日志所示，Redis 会安全关闭。

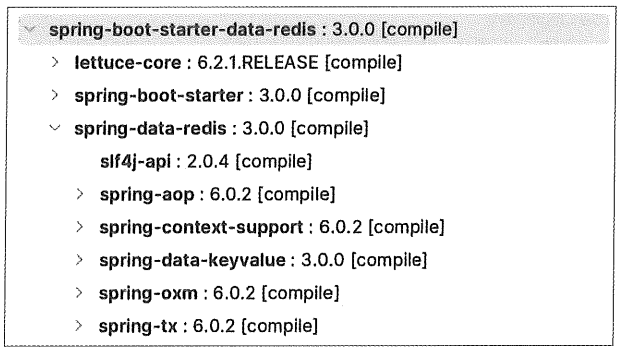
7.11.3 Spring Boot 集成 Redis

1. 集成并连接 Redis

Spring Boot 提供了一个 spring-boot-starter-data-redis 启动器依赖，如以下依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

该启动器依赖会导入 Spring Boot 集成 Redis 所需的所有依赖，如下图所示。



Spring Data Redis 提供了为 Spring 应用轻松配置和访问 Redis 的能力，spring-boot-starter- data-redis 启动器又在 Spring Data Redis 项目的基础上提供了 Redis 的自动配置功能，包括自动配置的 Lettuce 和 Jedis 客户端，如上图所示，默认客户端为 Lettuce。

Spring Boot在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了几个 Redis 相关的自动配置类，具体如下：

- RedisAutoConfiguration。
- RedisReactiveAutoConfiguration。
- RedisRepositoriesAutoConfiguration。

比如自动配置类 RedisAutoConfiguration 对应的参数绑定类为 RedisProperties，通过一系列 spring.redis.* 参数可以自动配置 Redis，如下面的自动配置类的源码所示。

```
@AutoConfiguration
@ConditionalOnClass(RedisOperations.class)
@EnableConfigurationProperties(RedisProperties.class)
@Import({ LettuceConnectionConfiguration.class, JedisConnectionConfiguration.class })
public class RedisAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean(name = "redisTemplate")
    @ConditionalOnSingleCandidate(RedisConnectionFactory.class)
    public RedisTemplate<Object, Object> redisTemplate(RedisConnectionFactory
redisConnectionFactory) {
        RedisTemplate<Object, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(redisConnectionFactory);
        return template;
    }

    @Bean
    @ConditionalOnMissingBean
    @ConditionalOnSingleCandidate(RedisConnectionFactory.class)
    public StringRedisTemplate stringRedisTemplate(RedisConnectionFactory
redisConnectionFactory) {
        return new StringRedisTemplate(redisConnectionFactory);
    }

}
```

只需要在应用配置文件中添加以下 Redis 配置参数即可连接 Redis：

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      database: 0
```

Spring Boot 3.0.0 新变化：

RedisProperties 参数绑定类的前缀已经由 spring.redis.* 变更为 spring.data.redis.*。

Redis 支持 Sentinel（哨兵）、Cluster（集群）、Standalone（单机）部署方式，这里的配置默认是单机模式，如果 host、port、database 参数都是以上默认值，则可以不用配置。

从自动配置类中可以看到导入了 Lettuce 和 Jedis 两个客户端的配置类，Lettuce 是 Redis 内置默认支持的，切换为 Jedis 需要额外导入 Jedis 的依赖，并配合使用以下参数：

```
spring:
  redis:
client-type: jedis
```

2. 使用 RedisTemplate

自动配置类中还注册了以下两个默认的 Redis 模板：

- RedisTemplate。
- StringRedisTemplate。

这两个模板可以直接在应用中注入使用，它们都是线程安全的，“开箱即用”，可以在多个实例中重复使用，区别如下：

- 两者是处理不同 key-value 类型的数据操作模板，StringRedisTemplate 继承自 RedisTemplate，只能处理 <String, String> 的泛型类型，而 RedisTemplate 可以处理 <Object, Object> 所有泛型类型。
- StringRedisTemplate 默认使用 StringRedisSerializer 序列化器，RedisTemplate 默认使用 JDK 自带的序列化器。

因为 StringRedisTemplate 继承自 RedisTemplate，相当于要同时配置两个 RedisTemplate，所以需要配合 @ConditionalOnSingleCandidate 注解根据指定的类名注入。

RedisTemplate 模板还提供了很多通用的操作接口，用于对特定的数据类型或某些键进行操作，常用的接口如下表所示。

接口	描述
GeoOperations	地理空间操作
HashOperations	散列类型操作
HyperLogLogOperations	HyperLogLog 类型操作
ListOperations	列表操作
SetOperations	集合操作
ValueOperations	字符串操作
ZSetOperations	有序集合操作

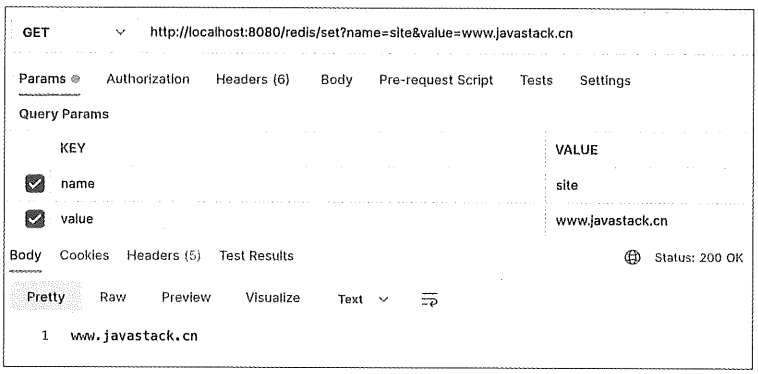
新建一个接口注入 StringRedisTemplate 并使用，如以下示例代码所示。

```
@RequiredArgsConstructor
```

```
@RestController
public class RedisController {

    private final StringRedisTemplate stringRedisTemplate;

    @GetMapping("/redis/set")
    public String set(@RequestParam("name") String name, @RequestParam("value")
String value) {
        ValueOperations<String, String> valueOperations = stringRedisTemplate.opsForValue();
        valueOperations.set(name, value);
        return valueOperations.get(name);
    }

}
```

这里将请求参数名称（name）、参数值（value）作为 key-value 设置到 Redis 数据库中，最后返回从 Redis 数据库中取出来的 key 为 name 的值。

请求成功示例如下图所示。



连接 Redis 控制台也可以获取对应 key 的值：

```
127.0.0.1:6379> get site
"www.javastack.cn"
```

每次使用 *Operations 时无须通过 RedisTemplate 获取，可以直接注入对应的 RedisTemplate 模板来使用 *Operations：

```
@Resource(name = "stringRedisTemplate")
private ValueOperations<String, String> valueOperations;
```

或者提前将其作为一个 Bean 注册到 Spring 容器中，然后直接注入使用：

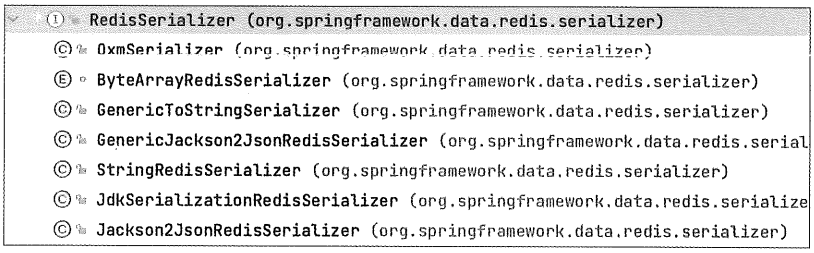
```
@Bean
public ValueOperations<String, Object> valueOperations(RedisTemplate<String, Object>
redisTemplate) {
    return redisTemplate.opsForValue();
}

@Autowired
private ValueOperations<String, Object> valueOperations;
```

如果自动配置的 RedisTemplate 模板不符合要求，那么可以对其进行自定义，覆盖自动配置的 RedisTemplate 模板，如下面的示例所示。

```
@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);

        StringRedisSerializer stringSerializer = new StringRedisSerializer();
        RedisSerializer jacksonSerializer = getJacksonSerializer();

        template.setKeySerializer(stringSerializer);
        template.setValueSerializer(jacksonSerializer);
        template.setHashKeySerializer(stringSerializer);
        template.setHashValueSerializer(jacksonSerializer);
        template.setEnableTransactionSupport(true);
        template.afterPropertiesSet();

        return template;
    }

    private RedisSerializer getJacksonSerializer() {
        ObjectMapper om = new ObjectMapper();
        om.setVisibility(PropertyAccessor.ALL, JsonAutoDetect.Visibility.ANY);
        om.activateDefaultTyping(LaissezFaireSubTypeValidator.instance,
                ObjectMapper.DefaultTyping.NON_FINAL);
        return new GenericJackson2JsonRedisSerializer(om);
    }

}
```

这里的 key 使用了 StringRedisSerializer 序列化器，value 使用了 GenericJackson2JsonRedisSerializer 序列化器。

Redis 序列化器的接口为 RedisSerializer，可以查看该接口的实现了解所有内置的序列化器，如下图所示。



下表是常用的几个序列化器。

序列化器	说明
StringRedisSerializer	String/byte[] 转换器，速度快
JdkSerializationRedisSerializer	JDK 自带序列化器
OxmSerializer	XML 序列化器，占空间，速度慢
Jackson2JsonRedisSerializer	JSON 序列化器，需要定义 JavaType
GenericJackson2JsonRedisSerializer	JSON 序列化器，不需要定义 JavaType

如果只是简单的字符串类型，那么使用 StringRedisSerializer 即可，如果是存储对象，则推荐使用 JSON 序列化器，可以很方便地把 JSON 类型的数据组装成 Java 中的对象。

7.12 MongoDB

7.12.1 概述

MongoDB 是一个开源 NoSQL 文档数据库，它使用的是类似 JSON 的数据结构，面向集合存储，而不是传统的基于表的关系型数据库，它可以支持非常松散、复杂的数据类型，并且支持非常强大的类似于面向对象的查询语言，支持索引，几乎能完成关系型数据库单表查询的大部分功能，所以，MongoDB 是非关系型数据库中功能最丰富的，又最像关系型数据库的 NoSQL 数据库。

Spring Boot 为 MongoDB 的集成及使用提供了许多便利，只需要简单的配置，基于 MongoTemplate 模板或者 MongoRepository 存储库就能“开箱即用”，十分方便。

7.12.2 MongoDB 环境搭建

1. 安装 MongoDB

MongoDB 支持多种操作系统的安装方式，包括 Docker 镜像，这里以 CentOS 为例，使用 yum 包管理器的方式安装 MongoDB 社区免费版。

首先创建并配置 yum 包管理系统，参考命令如下：

```
sudo vi /etc/yum.repos.d/mongodb-org-6.0.repo
```

配置文件的内容如下：

```
[mongodb-org-6.0]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/6.0/x86_64/
gpgcheck=1
enabled=1
gpgkey=https://www.mongodb.org/static/pgp/server-6.0.asc
```

当然，也可以不用配置 yum 仓库，可以选择直接从 MongoDB 存储库下载 MongoDB，配置 yum 的目的是可以直接使用 yum 命令下载并安装最新的 MongoDB，参考命令如下：

```
sudo yum install -y mongodb-org
```

笔者这里下载的是 MongoDB 6.0.2，也可以指定要安装的具体版本，参考命令如下：

```
sudo yum install -y mongodb-org-6.0.2 mongodb-org-database-6.0.2 mongodb-org-server-6.0.2
mongodb-mongosh-6.0.2 mongodb-org-mongos-6.0.2 mongodb-org-tools-6.0.2
```

2. 启动 MongoDB

安装好 MongoDB 后，可以使用 systemd (systemctl) 和 System V Init (service) 的方式来管理 MongoDB 服务，下面以更强大的 systemd (systemctl) 服务为例进行演示。

启动 MongoDB 的参考命令如下：

```
sudo systemctl start mongod
```

关闭 MongoDB 的参考命令如下：

```
sudo systemctl stop mongod
```

重新启动 MongoDB 的参考命令如下：

```
sudo systemctl restart mongod
```

系统重启时自动启动 MongoDB 的参考命令如下：

```
sudo systemctl enable mongod
```

默认情况下，MongoDB 会使用 mongod 用户账户运行并使用以下默认目录：

- /var/lib/mongo（数据目录）。
- /var/log/mongodb（日志目录）。

这些目录在 MongoDB 安装时就创建好了。

MongoDB 启动之后，可以使用以下参考命令验证：

```
sudo systemctl status mongod
```

```
$ sudo systemctl status mongod
• mongod.service - MongoDB Database Server
Loaded: loaded (/usr/lib/systemd/system/mongod.service; enabled; vendor preset: disabled)
Active: active (running) ...
...
```

可以看到正在运行的 MongoDB 的状态信息。

3. 连接 MongoDB

使用 mongosh 命令连接 MongoDB，它会默认连接到本地的 27017 端口，参考命令如下：

```
$ mongosh
Current Mongosh Log ID:       6368b535d2a64709a8392ddf
Connecting to:                mongodb://127.0.0.1:27017/?directConnection=
true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.0
Using MongoDB:                6.0.2
Using Mongosh:                1.6.0
...
test>
```

默认连接到 test 数据库，使用 show dbs 显示所有数据库，参考命令如下：

```
test> show dbs;
admin   40.00 KiB
config  60.00 KiB
local   72.00 KiB
```

这三个数据库都是 MongoDB 预设的，可以使用 use 命令进行切换。

下面是一个基本的新增和查询操作，参考命令如下：

```
test> db.collection.insertOne({id: 1, name: 'java'})
{
  acknowledged: true,
  insertedId: ObjectId("6368b746495ae1aaf3ebce65")
}
test> db.collection.find({id: 1})
[ { _id: ObjectId("6368b746495ae1aaf3ebce65"), id: 1, name: 'java' } ]
```

新增和查询都没问题，说明连接和操作都正常了。

7.12.3 Spring Boot 集成 MongoDB

1. 集成并连接 MongoDB

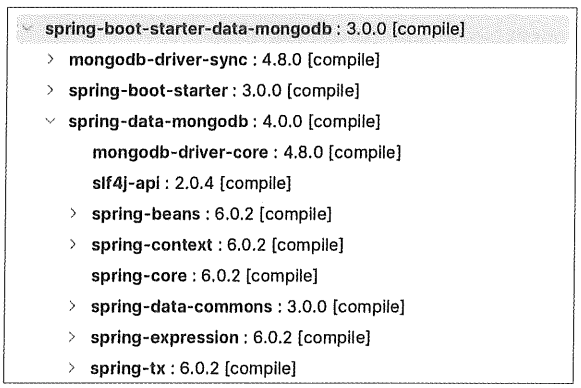
Spring Boot 为使用 MongoDB 的两种模型都提供了 Starter 启动器：

- spring-boot-starter-data-mongodb。
- spring-boot-starter-data-mongodb-reactive。

本节使用第一种方式，先添加以下依赖配置：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

该启动器依赖会导入 Spring Boot 集成 MongoDB 所需的所有依赖，如下图所示。



这里面就包含了 MongoDB 驱动和 spring-data-mongodb 数据层访问依赖。Spring Boot 在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了几个 MongoDB 相关的自动配置类，具体如以下所示。

- MongoDataAutoConfiguration。
- MongoDataAutoConfiguration。
- MongoReactiveDataAutoConfiguration。
- MongoRepositoriesAutoConfiguration。
- MongoAutoConfiguration。
- MongoReactiveAutoConfiguration。

对应的参数绑定为 MongoProperties，通过一系列 spring.data.mongodb.* 参数就能完成自动配置，比如这里添加最基本的连接参数，连接本地默认端口的 test 数据库：

```
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/test
```

2. 使用 MongoTemplate

和其他数据库模板一样，Spring Boot 也提供了 MongoDB 的 MongoTemplate 模板，自动配置后，在应用中就可以直接注入。

MongoTemplate 实例就注册在 MongoDataAutoConfiguration 自动配置类中，如以下源码所示。

```
@AutoConfiguration(after = MongoAutoConfiguration.class)
@ConditionalOnClass({ MongoClient.class, MongoTemplate.class })
@EnableConfigurationProperties(MongoProperties.class)
@Import({ MongoDataConfiguration.class, MongoDatabaseFactoryConfiguration.class,
        MongoDatabaseFactoryDependentConfiguration.class })
public class MongoDataAutoConfiguration {

}

@Configuration(proxyBeanMethods = false)
@ConditionalOnBean(MongoDatabaseFactory.class)
class MongoDatabaseFactoryDependentConfiguration {

    ...
```

```
    @Bean
    @ConditionalOnMissingBean(MongoOperations.class)
    MongoTemplate mongoTemplate(MongoDatabaseFactory factory, MongoConverter converter) {
        return new MongoTemplate(factory, converter);
    }

    ...

}
```

下面添加一个 MongoTemplate 模板的使用测试接口，示例如下：

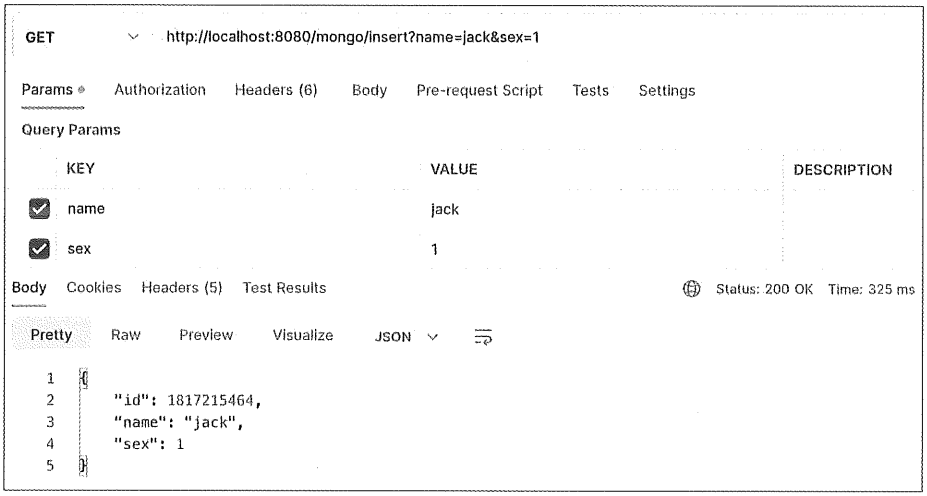
```
@RequiredArgsConstructor
@RestController
public class MongoController {

    public static final String COLLECTION_NAME = "javastack";

    private final MongoTemplate mongoTemplate;

    @RequestMapping("/mongo/insert")
    public User insert(@RequestParam("name") String name, @RequestParam("sex") int sex) {
        // 新增
        User user = new User(RandomUtils.nextInt(), name, sex);
        mongoTemplate.insert(user, COLLECTION_NAME);

        // 查询
        Query query = new Query(Criteria.where("name").is(name));
        return mongoTemplate.findOne(query, User.class, COLLECTION_NAME);
    }

}
```

User 实体类如以下示例代码所示。

```
@NoArgsConstructor
@AllArgsConstructor
@Data
public class User {

    private long id;
```

```
    private String name;

    private int sex;

}
```

通过注入 MongoTemplate 模板就能实现 CRUD 操作，先插入数据，然后根据名称查询刚插入的数据。重启应用后再调用插入数据接口，测试结果如下图所示。



结果正常返回了，说明数据插入成功了，通过命令也能成功查询到数据，如以下参考命令所示。

```
test> db.javastack.findOne({name: 'jack'})
{
  _id: 1817215464,
  name: 'jack',
  sex: 1,
  _class: 'cn.javastack.mongodb.User'
}
```

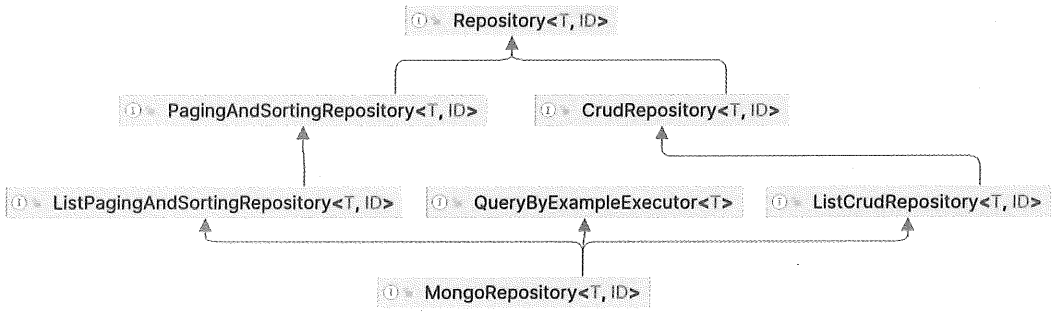
3. 使用 MongoRepository

除了使用 MongoTemplate，Spring Data 还提供了对 MongoDB 的 Repositories 存储库的支持，可以像使用 JPA Repositories 一样操作 MongoDB 数据库，比如添加一个自定义的 UserRepository，示例如下：

```
public interface UserRepository extends MongoRepository<User, Long> {
```

```
    List<User> findByName(String name);

}
```

UserRepository 继承了 MongoRepository，MongoRepository 提供了针对 MongoDB 的丰富的 CRUD 操作方法，很多 CRUD 的数据库基本操作代码就不需要写了，类继承图如下图所示。



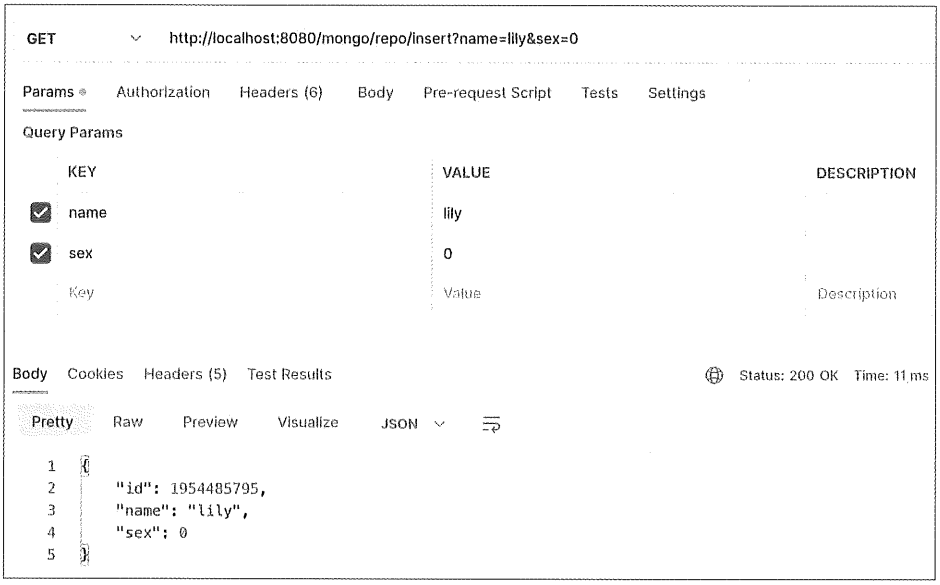
如果有需要，也可以按规范进行自定义，比如上面的 findByName 方法。这里再针对 MongoRepository 新增一个类似的接口进行测试，如以下测试代码所示。

```
@RequiredArgsConstructor
@RestController
public class MongoController {

    public static final String COLLECTION_NAME = "javastack";
    private final MongoTemplate mongoTemplate;

    private final UserRepository userRepository;

    ...

    @RequestMapping("/mongo/repo/insert")
    public User repoInsert(@RequestParam("name") String name, @RequestParam("sex")
int sex) {
        // 新增
        User user = new User(RandomUtils.nextInt(), name, sex);
        userRepository.save(user);

        // 查询
        return userRepository.findByName(name).get(0);
    }

}
```

MongoRepository 操作方法没有指定集合名称的参数，还需要在实体类上使用 @Document 注解指定集合名称，如以下示例代码所示。

```
@NoArgsConstructor
@AllArgsConstructor
@Data
@Document(collection = "javastack")
public class User {

    private long id;

    private String name;

    private int sex;

}
```

通过注入 UserRepository 存储库来实现 CRUD 操作，和之前的逻辑一样，先插入数据，再根据名称查询刚插入的数据，重启应用之后再调用插入数据接口，测试结果如下图所示。



页面正常返回了，说明数据插入成功了，通过以下命令也能成功查询到数据：

```
test> db.javastack.findOne({name: 'lily'})
{
```

```
    _id: Long("1954485795"),
    name: 'lily',
    sex: 0,
    _class: 'cn.javastack.mongodb.User'
}
```

Spring Boot 集成和使用 MongoDB 的过程十分简单，添加依赖、连接配置即可以使用 Spring Boot “搞定”简单的 MongoDB 应用。

7.13 Elasticsearch

7.13.1 概述

Elasticsearch 是 Apache 开源的一个分布式、基于 RESTful 风格的数据分析和搜索引擎，由 Java 语言开发，它拥有很方便的搜索、分析和探索数据的能力，能够达到实时搜索，稳定、可靠、快速，安装使用方便，能使数据在生产环境中变得更有价值，它是目前最受企业欢迎的搜索引擎，其次是 Apache Solr（也基于 Lucene）。

Elasticsearch 中的数据是面向文档的，一个文档代表关系型数据库中的一行数据，它是对文档进行索引、检索、排序和过滤，而不是对行或者列数据，JSON 则作为文档的序列化格式。

下面这个 JSON 文档代表了一个 user 对象：

```
{
    "email":      "xxx@javastck.cn",
    "first_name":  "javastack",
    "last_name":  "admin",
    "info": {
        "wx":       "Java 技术栈 ",
        "sites": [ "javastack.cn", "www.javastack.cn" ]
    },
    "add_date":    "2022/11/08"
}
```

Elasticsearch 索引的两种概念如下：

- **索引（名词）**：相当于关系型数据库中的一个数据库，是一个存储关系型文档的地方，索引（index）的复数词为 indices 或 indexes。
- **索引（动词）**：指存储一个文档到一个索引中，类似于 SQL 语句中的 INSERT 关键词。

Spring Boot 为 Elasticsearch 的集成及使用提供了许多便利，只需要简单的配置，基于 ElasticsearchTemplate 模板或者 MongoRepository 存储库就能“开箱即用”，十分方便。

7.13.2 Elasticsearch 环境搭建

1. 安装 Elasticsearch

Elasticsearch 支持多种操作系统的安装方式，包括 Docker 镜像，这里以 CentOS 为例，使用 yum 包管理器的方式安装 Elasticsearch。

首先创建一个 yum 管理系统配置文件，参考命令如下：

```
sudo vi /etc/yum.repos.d/elasticsearch.repo
```

配置文件的内容如下所示。

```
[elasticsearch]
name=Elasticsearch repository for 8.x packages
baseurl=https://artifacts.elastic.co/packages/8.x/yum
gpgcheck=1
gpgkey=https://artifacts.elastic.co/GPG-KEY-elasticsearch
enabled=0
autorefresh=1
type=rpm-md
```

当然，也可以不用配置 yum 仓库，选择直接从 Elasticsearch 存储库下载 Elasticsearch，配置 yum 的目的是可以直接使用 yum 命令下载并安装最新的 Elasticsearch，参考命令如下：

```
sudo yum install --enablerepo=elasticsearch elasticsearch
```

笔者这里下载的是 Elasticsearch 8.5.0，安装过程中，默认启用并配置以下安全功能：

- 启用认证和授权功能，并生成一个默认密码的 elastic 超级用户。
- 为传输层和 HTTP 层生成 TLS 的证书和密钥，并使用这些密钥和证书启用并配置 TLS。

默认生成的安全信息会显示在安装控制台上，如下所示。

```
---------------------- Security autoconfiguration information ----------------------

Authentication and authorization are enabled.
TLS for the transport and HTTP layers is enabled and configured.

The generated password for the elastic built-in superuser is : nE-5Sq92L=+-Oofr7pTR
```

```
If this node should join an existing cluster, you can reconfigure this with
'/usr/share/elasticsearch/bin/elasticsearch-reconfigure-node --enrollment-token <token-here>'
after creating an enrollment token on your existing cluster.

You can complete the following actions at any time:

Reset the password of the elastic built-in superuser with
'/usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic'.

Generate an enrollment token for Kibana instances with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana'.

Generate an enrollment token for Elasticsearch nodes with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s node'.

-------------------------------------------------------------------------------
```

比如默认的用户名和密码为 elastic/nE-5Sq92L=+-Oofr7pTR，上面也提供了修改密码的指令，如下所示。

```
./elasticsearch-reset-password -u elastic –interactive
```

默认重新生成随机密码，自定义修改密码需要设置 --interactive 参数，笔者修改密码为 123456。

2. 启动 Elasticsearch

设置 Elasticsearch 开机自启动的参考命令如下：

```
sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service
```

启动 Elasticsearch 服务的参考命令如下：

```
sudo systemctl start elasticsearch.service
```

关闭 Elasticsearch 服务的参考命令如下：

```
sudo systemctl stop elasticsearch.service
```

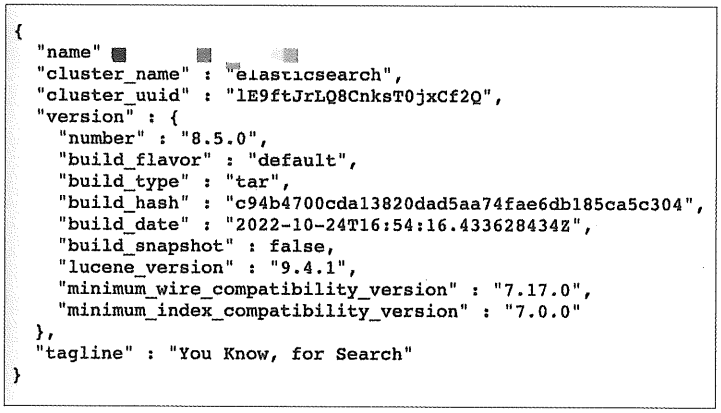
验证 Elasticsearch 服务是否运行的参考命令如下：

```
curl --cacert $ES_HOME/config/certs/http_ca.crt -u elastic https://localhost:9200
```

```
Enter host password for user 'elastic':
{
```

```
  "name" : "...",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "lE9ftJrLQ8CnksT0jxCf2Q",
  "version" : {
    "number" : "8.5.0",
    "build_flavor" : "default",
    "build_type" : "tar",
    "build_hash" : "c94b4700cda13820dad5aa74fae6db185ca5c304",
    "build_date" : "2022-10-24T16:54:16.433628434Z",
    "build_snapshot" : false,
    "lucene_version" : "9.4.1",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

或者直接在浏览器中访问 https://localhost:9200，然后输入用户名和密码也可以验证，如下图所示。



3. 连接 Elasticsearch

连接 Elasticsearch 后可以通过 Elasticsearch 提供的 Restful API 发起操作，比如查询 name=jack 的数据，参考命令如下：

```
curl -X GET “localhost:9200/javastack/_search?q=name:jack&pretty”
curl: (52) Empty reply from server
```

还可以安装一个可视化的开源工具 elasticsearch-head，开源地址为链接 9。

安装 elasticsearch-head 工具的参考命令如下：

```
git clone git://github.com/mobz/elasticsearch-head.git
cd elasticsearch-head
npm install
npm run start
```

然后需要设置 Elasticsearch API 以支持跨域，修改以下配置文件：

/config/elasticsearch.yml

在配置文件的最后添加跨域配置，参考配置的内容如下：

```
http.cors.enabled: true
http.cors.allow-origin: "*"
http.cors.allow-methods: "GET,PUT,POST,DELETE"
http.cors.allow-headers: Authorization,X-Requested-With,Content-Length,Content-Type
```

重启 Elasticsearch，之后就可以访问 elasticsearch-head 了，访问地址如下：

http://localhost:9100/?auth_user=elastic&auth_password=123456

效果如下图所示。

连接上 Elasticsearch 之后，笔者在这里添加了一个 javastack 索引，虽然后面的集成测试会用到，但这并不是必需的，应用在插入数据时也会自动创建。

7.13.3 Spring Boot 集成 Elasticsearch

1. 集成并连接 Elasticsearch

Spring Boot 支持以下多个 Elasticsearch 客户端：

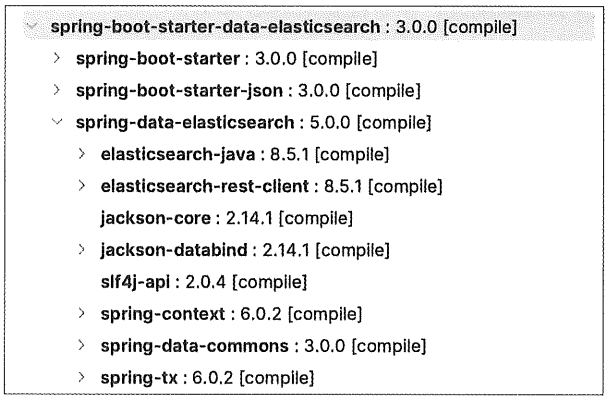
- Elasticsearch 官方低级别的 REST 客户端。
- Elasticsearch 官方的 Java API 客户端。

- Spring Data Elasticsearch 提供的 ReactiveElasticsearchClient 客户端。

并且提供了一个专用的 spring-boot-starter-data-elasticsearch 一站式启动器，如以下依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-elasticsearch</artifactId>
</dependency>
```

该启动器依赖会导入 Spring Boot 集成 Elasticsearch 所需的所有依赖，如下图所示。



在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了几个 Elasticsearch 相关的自动配置类，具体如下所示。

- ElasticsearchDataAutoConfiguration。
- ElasticsearchRepositoriesAutoConfiguration。
- ReactiveElasticsearchRepositoriesAutoConfiguration。
- ElasticsearchClientAutoConfiguration。
- ElasticsearchRestClientAutoConfiguration。
- ReactiveElasticsearchClientAutoConfiguration。

对应的参数绑定为 ElasticsearchProperties，通过一系列 spring.elasticsearch.* 参数就能完成自动配置。

这里添加最基本的连接参数，连接本地默认端口的 test 数据库，如以下配置示例所示。

```
spring:
  elasticsearch:
```

```
    uris: http://localhost:9200
    connection-timeout: 5s
    socket-timeout: 10s
    username: elastic
    password: 123456
```

因为 Elasticsearch 默认开启了 xpack 安全机制，所以只能通过 https 连接，而 Spring Boot 自动配置参数类 ElasticsearchProperties 暂时无法配置 xpack 安全信息，为了方便进行 http 测试，笔者这里先关闭 xpack 安全机制，修改配置如下：

```
xpack.security.http.ssl:
  enabled: false
  keystore.path: certs/http.p12
```

重启 Elasticsearch，随后即可通过 http 形式访问 API，应用也能顺利连接。

2. 使用 ElasticsearchTemplate

和其他数据库模板一样，Spring Boot 也提供了 Elasticsearch 的 ElasticsearchTemplate 模板，自动配置后，在应用中就可以直接注入。

ElasticsearchTemplate 就注册在 ElasticsearchDataAutoConfiguration 自动配置类中，如下面的源码所示。

```
@AutoConfiguration(
        after = { ElasticsearchClientAutoConfiguration.class, ReactiveElasticsearch-
ClientAutoConfiguration.class })
@ConditionalOnClass({ ElasticsearchTemplate.class })
@Import({ ElasticsearchDataConfiguration.BaseConfiguration.class,
        ElasticsearchDataConfiguration.JavaClientConfiguration.class,
        ElasticsearchDataConfiguration.ReactiveRestClientConfiguration.class })
public class ElasticsearchDataAutoConfiguration {

}

abstract class ElasticsearchDataConfiguration {

    ...

    @Configuration(proxyBeanMethods = false)
    @ConditionalOnClass(ElasticsearchClient.class)
```

```
    static class JavaClientConfiguration {

        @Bean
        @ConditionalOnMissingBean(value = ElasticsearchOperations.class, name =
"elasticsearchTemplate")
        @ConditionalOnBean(ElasticsearchClient.class)
        ElasticsearchTemplate elasticsearchTemplate(ElasticsearchClient client,
ElasticsearchConverter converter) {
            return new ElasticsearchTemplate(client, converter);
        }

    }

    ...

}
```

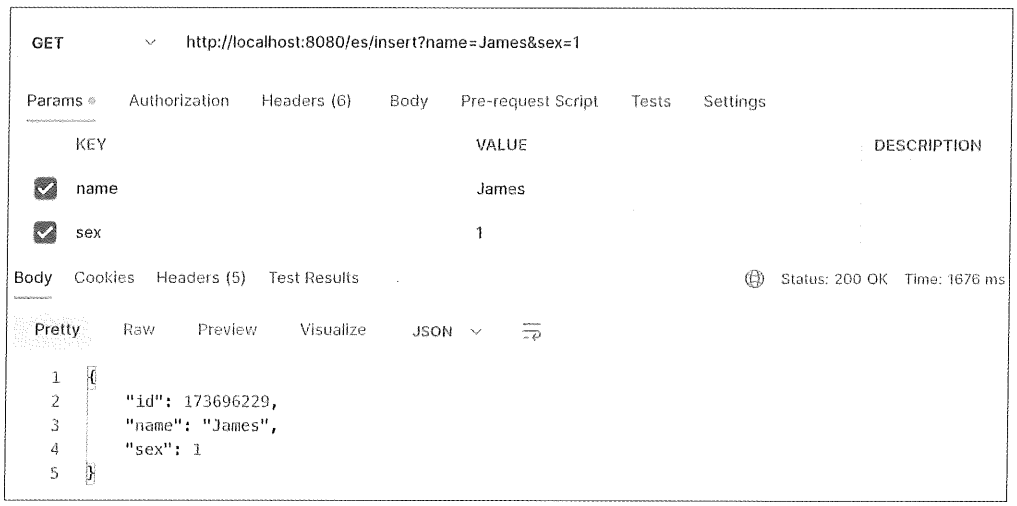
添加一个 ElasticsearchTemplate 模板的使用测试接口，如以下示例代码所示。

```
@RequiredArgsConstructor
@RestController
public class EsController {

    public static final String INDEX_JAVASTACK = "javastack";
    private final ElasticsearchTemplate elasticsearchTemplate;

    @RequestMapping("/es/insert")
    public User insert(@RequestParam("name") String name, @RequestParam("sex") int
sex) throws InterruptedException {
        // 新增
        User user = new User(RandomUtils.nextInt(), name, sex);
        IndexCoordinates indexCoordinates =  IndexCoordinates.of(INDEX_JAVASTACK);
        User save = elasticsearchTemplate.save(user, indexCoordinates);

        // 可能有延迟，休眠一秒再查询
        Thread.sleep(1000l);
        Query query = new CriteriaQuery(Criteria.where("name").is(name));
        return elasticsearchTemplate.searchOne(query, User.class, indexCoordinates).
getContent();
    }

}
```

User 实体类如以下示例代码所示。

```
@NoArgsConstructor
@AllArgsConstructor
@Data
public class User {

    private long id;

    private String name;

    private int sex;

}
```

通过注入 ElasticsearchTemplate 模板就能实现 CRUD 操作，先插入数据，再根据名称查询刚插入的数据，然后调用插入数据接口，测试结果如下图所示。



页面正常返回了，说明数据插入成功了。通过 elasticsearch-head 也能成功查询到数据，如下图所示。

还是一如既往的简单，使用 Spring Boot 及 Elasticsearch 提供的 ElasticsearchTemplate 操作模板，可以轻松“搞定”一个 Elasticsearch 搜索应用。

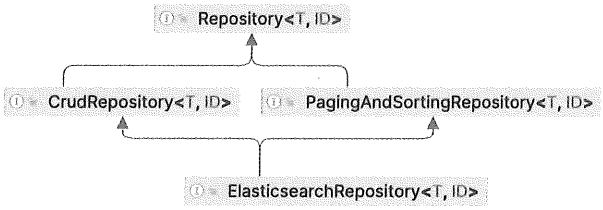
3. 使用 ElasticsearchRepository

除了使用 ElasticsearchTemplate，Spring Data 还提供了对 Elasticsearch 的 Repositories 存储库的支持，可以像使用 JPA Repositories 一样操作 Elasticsearch 数据库，这里添加一个自定义的 UserRepository，示例如下：

```
public interface UserRepository extends ElasticsearchRepository<User, Long> {

    List<User> findByName(String name);

}
```

UserRepository 继承了 ElasticsearchRepository，ElasticsearchRepository 提供了针对 Elasticsearch

的丰富的CRUD操作方法，很多CRUD的数据库基本操作代码就不需要写了，类继承图如下图所示。



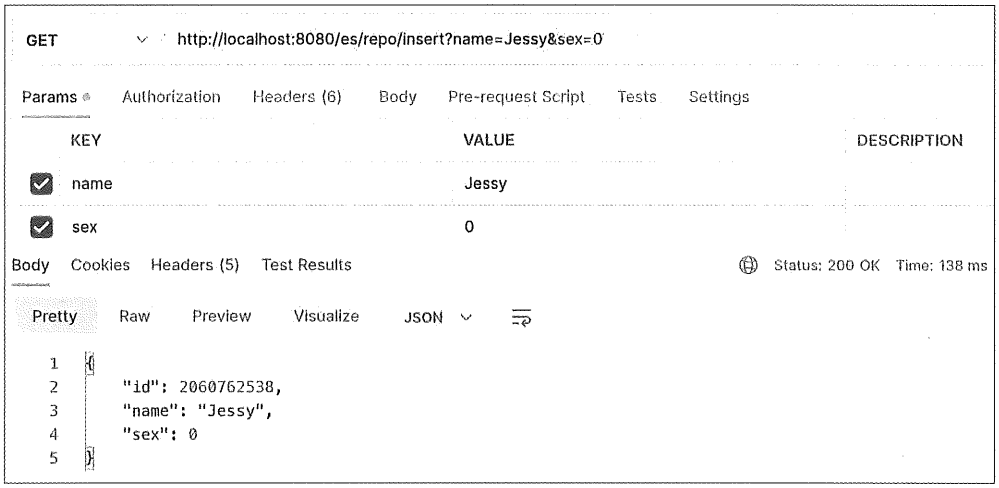
如果有需要，也可以按规范进行自定义，比如上面的findByName方法。这里针对ElasticsearchRepository再新增一个类似的接口进行测试，示例如下：

```
@RequiredArgsConstructor
@RestController
public class EsController {

    public static final String INDEX_JAVASTACK = "javastack";
    private final ElasticsearchTemplate elasticsearchTemplate;

    private final UserRepository userRepository;

    ...

    @RequestMapping("/es/repo/insert")
    public User repoInsert(@RequestParam("name") String name, @RequestParam("sex")
int sex) {
        // 新增
        User user = new User(RandomUtils.nextInt(), name, sex);
        userRepository.save(user);

        // 查询
        return userRepository.findByName(name).get(0);
    }

}
```

ElasticsearchRepository操作方法没有指定索引名称的参数，还需要在实体类上使用@Document注解指定索引名称，示例如下：

```
@NoArgsConstructor
@AllArgsConstructor
@Data
@Document(indexName = "javastack")
public class User {

    private long id;

    private String name;

    private int sex;

}
```

然后通过注入 UserRepository 存储库来实现 CRUD 操作，和之前的逻辑一样，先插入数据，再根据名称查询刚插入的数据，然后调用插入数据接口，测试结果如下图所示。



页面正常返回了，说明数据插入成功了，通过 elasticsearch-head 也能成功查询到数据，如下图所示。

本节介绍了 Spring Boot 如何快速集成 Elasticsearch 并进行了实践，整个集成和使用过程十分简单，添加依赖、连接配置即可使用 Spring Boot “搞定”简单的 Elasticsearch 应用。

第 8 章
Spring Boot 计划任务

计划任务是企业系统必不可少的部分，比如银行对账、定时发送报表、事务最终一致性补偿机制等都需要计划任务来辅助，Spring Boot 提供了对 Spring 计划任务及 Quartz 计划任务的官方支持，本章会介绍这两者的使用方式及相应的原理分析，包括基本概述、线程池工作流程、任务实现、自定义线程池配置等。

8.1 Spring 计划任务

8.1.1 概述

Spring Boot 实现计划任务不需要引入其他的依赖，它只会依赖 Spring 基础框架中的计划任务，Spring 3.1 开始支持计划任务，Spring Boot 只是在自动配置包中提供了对 Spring 计划任务的自动配置，其主要还是依赖于 Spring 中的 spring-context 包，所以只要任何一个 Spring Boot Starter 启动器包含 spring-context 包，它就能使用计划任务，比如 spring-boot-starter-web 一站式启动器。

Spring Boot 提供了对任务执行、任务调度的自动配置，对应的自动配置类如下所示。

- TaskExecutionAutoConfiguration：任务执行自动配置类。
- TaskSchedulingAutoConfiguration：任务调度自动配置类。

它们被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中，TaskExecutionAutoConfiguration 自动配置类的源码如下所示。

```
@ConditionalOnClass(ThreadPoolTaskExecutor.class)
@AutoConfiguration
@EnableConfigurationProperties(TaskExecutionProperties.class)
public class TaskExecutionAutoConfiguration {

    /**
     * Bean name of the application {@link TaskExecutor}.
     */
    public static final String APPLICATION_TASK_EXECUTOR_BEAN_NAME = "applicationTaskExecu-
tor";

    @Bean
    @ConditionalOnMissingBean
    public TaskExecutorBuilder taskExecutorBuilder(TaskExecutionProperties properties,
            ObjectProvider<TaskExecutorCustomizer> taskExecutorCustomizers,
            ObjectProvider<TaskDecorator> taskDecorator) {
        TaskExecutionProperties.Pool pool = properties.getPool();
        TaskExecutorBuilder builder = new TaskExecutorBuilder();
        builder = builder.queueCapacity(pool.getQueueCapacity());
        builder = builder.corePoolSize(pool.getCoreSize());
        builder = builder.maxPoolSize(pool.getMaxSize());
        builder = builder.allowCoreThreadTimeOut(pool.isAllowCoreThreadTimeout());
        builder = builder.keepAlive(pool.getKeepAlive());
        Shutdown shutdown = properties.getShutdown();
        builder = builder.awaitTermination(shutdown.isAwaitTermination());
        builder = builder.awaitTerminationPeriod(shutdown.getAwaitTerminationPeriod());
        builder = builder.threadNamePrefix(properties.getThreadNamePrefix());
        builder = builder.customizers(taskExecutorCustomizers.orderedStream()::iterator);
        builder = builder.taskDecorator(taskDecorator.getIfUnique());
        return builder;
    }

    @Lazy
    @Bean(name = { APPLICATION_TASK_EXECUTOR_BEAN_NAME,
            AsyncAnnotationBeanPostProcessor.DEFAULT_TASK_EXECUTOR_BEAN_NAME })
    @ConditionalOnMissingBean(Executor.class)
    public ThreadPoolTaskExecutor applicationTaskExecutor(TaskExecutorBuilder builder) {
        return builder.build();
    }

}
```

如果当前上下文中没有任何 Executor 实例，那么它会自动注册一个默认的 ThreadPoolTaskExecutor

线程池任务执行器实例，它基于JDK并发包中的Executor接口实现了自己的任务执行器。同时注册了一个TaskExecutorBuilder实例，用来实现自定义的任务执行线程池，对应的参数绑定类为TaskExecutionProperties类，通过一系列spring.task.execution.*参数可以自定义默认任务执行的线程池配置。

TaskSchedulingAutoConfiguration自动配置类的源码如下所示。

```
@ConditionalOnClass(ThreadPoolTaskScheduler.class)
@AutoConfiguration(after = TaskExecutionAutoConfiguration.class)
@EnableConfigurationProperties(TaskSchedulingProperties.class)
public class TaskSchedulingAutoConfiguration {

    @Bean
    @ConditionalOnBean(name = TaskManagementConfigUtils.SCHEDULED_ANNOTATION_PROCESSOR_BEAN_
NAME)
    @ConditionalOnMissingBean({ SchedulingConfigurer.class, TaskScheduler.class,
ScheduledExecutorService.class })
    public ThreadPoolTaskScheduler taskScheduler(TaskSchedulerBuilder builder) {
        return builder.build();
    }

    @Bean
    @ConditionalOnBean(name = TaskManagementConfigUtils.SCHEDULED_ANNOTATION_PROCESSOR_
BEAN_NAME)
    public static LazyInitializationExcludeFilter scheduledBeanLazyInitializationExcludeFilter() {
        return new ScheduledBeanLazyInitializationExcludeFilter();
    }

    @Bean
    @ConditionalOnMissingBean
    public TaskSchedulerBuilder taskSchedulerBuilder(TaskSchedulingProperties properties,
            ObjectProvider<TaskSchedulerCustomizer> taskSchedulerCustomizers) {
        TaskSchedulerBuilder builder = new TaskSchedulerBuilder();
        builder = builder.poolSize(properties.getPool().getSize());
        Shutdown shutdown = properties.getShutdown();
        builder = builder.awaitTermination(shutdown.isAwaitTermination());
        builder = builder.awaitTerminationPeriod(shutdown.getAwaitTerminationPeriod());
        builder = builder.threadNamePrefix(properties.getThreadNamePrefix());
        builder = builder.customizers(taskSchedulerCustomizers);
        return builder;
    }

}
```

任务调度自动配置类在任务执行自动配置类之后启用注册，如果当前上下文中没有以下任何一个实例：

- SchedulingConfigurer。
- TaskScheduler。
- ScheduledExecutorService。

那么它会自动注册一个默认的 ThreadPoolTaskScheduler 线程池任务调度器实例，它同样基于 JDK 并发包中的 Executor 接口实现了自己的任务调度器，除了 ScheduledExecutorService 是 JDK 并发包中的调度类，其他两个都是 Spring 调度框架中的类。

同时可以注册一个 TaskSchedulerBuilder 实例，对应的参数绑定类为 TaskSchedulingProperties，通过一系列 spring.task.scheduling.* 参数可以实现自定义的任务调度线程池配置。

8.1.2 线程池工作流程

任务执行参数绑定类的源码如下：

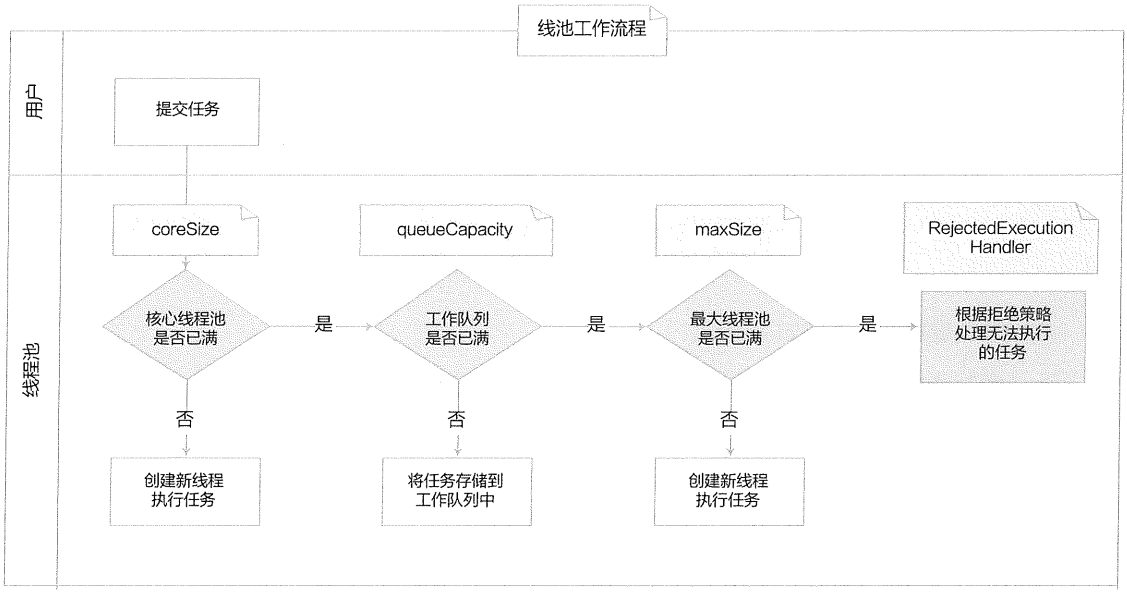
```
@ConfigurationProperties("spring.task.execution")
public class TaskExecutionProperties {

    private final Pool pool = new Pool();

    private final Shutdown shutdown = new Shutdown();

    public static class Pool {

        /**
         * Queue capacity. An unbounded capacity does not increase the pool and therefore
         * ignores the "max-size" property.
         */
        private int queueCapacity = Integer.MAX_VALUE;

        /**
         * Core number of threads.
         */
        private int coreSize = 8;

        /**
         * Maximum allowed number of threads. If tasks are filling up the queue, the pool
```

```
         * can expand up to that size to accommodate the load. Ignored if the queue is
         * unbounded.
         */
        private int maxSize = Integer.MAX_VALUE;

        /**
         * Whether core threads are allowed to time out. This enables dynamic growing and
         * shrinking of the pool.
         */
        private boolean allowCoreThreadTimeout = true;

        /**
         * Time limit for which threads may remain idle before being terminated.
         */
        private Duration keepAlive = Duration.ofSeconds(60);

        // ...

    }

    ...
```

各线程池参数的说明如下表所示。

参数名	说明	默认值
queueCapacity	队列大小	Integer 最大值
coreSize	核心线程数	8
maxSize	最大线程数	Integer 最大值
allowCoreThreadTimeout	是否允许核心线程超时	true
keepAlive	线程保活时长	60 秒

除了 allowCoreThreadTimeout 参数，其他参数都是 JDK 线程池提供的，线程池工作流程及各参数的使用如下图所示。



线程池大概的工作流程如下：

- 如果线程数 < 核心线程数 coreSize，就会一直创建新线程来处理任务。
- 如果线程数 = 核心线程数 coreSize，且任务队列 queueCapacity 没有满，就会先将任务存放到队列中，空闲线程会从队列中取出任务并执行。
- 如果线程数 = 核心线程数 coreSize，任务队列 queueCapacity 也满了，并且线程数 < 最大线程数 maxSize，就会一直创建新线程来处理任务。
- 如果核心线程数 coreSize、任务队列 queueCapacity、最大线程数 maxSize 三个都满了，则执行任务拒绝策略，默认为抛弃策略（见 ExecutorConfigurationSupport 类的定义）。
- 如果线程空闲时间达到 keepAlive 时间，则销毁线程，默认只销毁核心线程之外的线程，allowCoreThreadTimeout 用于控制是否销毁核心线程。

8.1.3 实现计划任务

Spring 中的计划任务十分轻量级，使用起来也比 Quartz 要简单得多。在使用 Spring Boot 计划任务之前，需要先开启计划任务，一般建议在应用主类上启用，如以下示例所示。

```
@EnableScheduling
```

```
@EnableAsync
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

}
```

其中最关键的两个注解如下所示。

- @EnableScheduling：表示启用任务调度，它导入了 SchedulingConfiguration 调度配置类，是通过注册一个 ScheduledAnnotationBeanPostProcessor 实例实现的。
- @EnableAsync：表示启用任务异步执行，它导入了 AsyncConfigurationSelector 配置选择器类，其根据不同的 AOP 模式选择不同的配置类。

这两个注解都是 spring-context 包中的注解，不需要引入额外的依赖，这里实现了一个简单的计划任务，每 3 秒打印一次日志，如下面的示例代码所示。

```
@Slf4j
@Component
public class SimpleTask {

    @Scheduled(cron = "*/3 * * * * *")
    public void printTask() {
        log.info("这是一个简单的计划任务。");
    }

}
```

在要实现计划任务的方法上使用 @Scheduled 注解启用任务调度，使用它需要先开启对应的 @EnableScheduling 注解，它使用的是 ThreadPoolTaskScheduler 线程池任务调度器。

启动应用后会看到应用每 3 秒打印一次日志，如下面的日志所示。

```
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
```

```
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [TaskScheduler-1] c...SimpleTask        : 这是一个简单的计划任务。
```

任务正常执行，可以注意到每次都是 TaskScheduler-1 这个任务调度线程在执行，而没有使用 Spring Boot 自动配置的 ThreadPoolTaskExecutor 线程池，需要使用 @Async 注解开启方法异步执行才会使用该线程池，即对应之前开启的 @EnableAsync 来开启异步执行注解。

现在加上 @Async 注解后再开启任务异步执行，如以下示例所示。

```
@Slf4j
@Component
public class SimpleTask {

    private int count;

    @Async
    @Scheduled(cron = "*/3 * * * * *")
    public void printTask() {
        log.info("这是一个简单的计划任务。");
    }

}
```

修改代码后，再来观察一下任务执行日志，如以下日志所示。

```
--- [     task1-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-2] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-3] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-4] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-5] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-6] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-7] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-8] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-1] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-2] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-3] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-4] c...SimpleTask        : 这是一个简单的计划任务。
--- [     task1-5] c...SimpleTask        : 这是一个简单的计划任务。
```

现在可以看到，每次任务执行的线程都不一样了，因为自动配置的任务执行线程池默认是 8 个核心线程，系统又只有一个任务在运行，所以每次都会创建一个新的线程去运行，直到达到 8 个核心线程数，当核心线程数满了之后就会按顺序取线程池中已有的线程执行。

8.1.4 Cron 表达式

Spring Task 使用了类似于 Cron 的表达式，并扩展了常规的 UN*X 中的定义，和常规的 Cron 表达式略有不同，位数也不同，也不是完全兼容常规的 Cron 表达式，这一点需要注意。

@Scheduled 注解的 Cron 表达式参数的源码如下：

```
/**
 * A cron-like expression, extending the usual UN*X definition to include triggers
 * on the second, minute, hour, day of month, month, and day of week.
 * <p>For example, {@code "0 * * * * MON-FRI"} means once per minute on weekdays
 * (at the top of the minute - the 0th second).
 * <p>The fields read from left to right are interpreted as follows.
 * <ul>
 * <li>second</li>
 * <li>minute</li>
 * <li>hour</li>
 * <li>day of month</li>
 * <li>month</li>
 * <li>day of week</li>
 * </ul>
 * <p>The special value {@link #CRON_DISABLED "-"} indicates a disabled cron
 * trigger, primarily meant for externally specified values resolved by a
 * <code>${...}</code> placeholder.
 * @return an expression that can be parsed to a cron schedule
 * @see org.springframework.scheduling.support.CronExpression#parse(String)
 */
String cron() default "";
```

可以看到，Spring 计划任务中的 Cron 表达式只能由 6 个字段组成，再继续查看 CronExpression # parse 解析方法的源码，如下面源码中的注释所示。

```
Parse the given crontab expression  string into a CronExpression. The string has six
single space-separated time and date fields:
  ┌──────────────────────── second (0-59)
  │ ┌────────────────────── minute (0 - 59)
  │ │ ┌──────────────────── hour (0 - 23)
  │ │ │ ┌────────────────── day of the month (1 - 31)
  │ │ │ │ ┌──────────────── month (1 - 12) (or JAN-DEC)
  │ │ │ │ │ ┌────────────── day of the week (0 - 7)
  │ │ │ │ │ │          (0 or 7 is Sunday, or MON-SUN)
  │ │ │ │ │ │
  * * * * * *
```

Spring Cron 表达式中的 6 个字段分别表示：

- 秒（0 ~ 59）。
- 分（0 ~ 59）。
- 小时（0 ~ 23）。
- 日期（1 ~ 31）。
- 月份（1 ~ 12，或者 JAN ~ DEC）。
- 周几（0 ~ 7，其中 0 和 7 表示周日，或者 MON ~ SUN）。

常用的 Cron 表达式的使用说明如下表所示。

Cron 表达式	说明
*/10 * * * * *	每 10 秒执行一次
0 0 * * * *	每小时执行一次
0 0 8-10 * * *	每天 8 ~ 10 点整点执行一次
0 0 6,19 * * *	每天 6 点和 19 点执行一次
0 0/30 8-10 * * *	每天 8 ~ 10 点之间每 30 分钟执行一次
0 0 9-17 * * MON-FRI	每周一到周五 9 点到 17 点整点执行一次
0 0 0 11 11 ?	每年双 11 的 0 点执行一次

支持以下关键字代替对应的字段：

- @yearly（每年）。
- @monthly（每月）。
- @weekly（每周）。
- @daily（每日）。
- @hourly（每小时）。

比如，每年 1 月 1 号 0 点执行一次 0 0 0 1 1 *，就可以直接写成 @yearly。

更多的用法可以参考以下方法：

org.springframework.scheduling.support.CronExpression#parse

Spring Cron 表达式没有指定年份的字段，默认表示每年，那么问题来了：

如果想在具体的某年某日，如 2025/02/25 这一天执行任务，那么怎么配置呢？

笔者也没有找到办法，Spring Cron 表达式不能指定年份，实际工作中也很少会有跨年的计划任

务，如果需要跨年配置就只能换成 Quartz 计划任务框架了，Quartz 计划任务框架的功能要更强大。

8.1.5 自定义线程池

从线程池工作流程一节中可以知道，Spring Boot 任务执行的默认队列大小为 Integer 的最大值，即 231-1，可以理解为无限大，如果存放大量的执行任务，就可能导致内存溢出，这也是为什么不建议使用默认线程池的原因。

实际工作中需要自定义任务线程池，以规避默认线程池带来的内存溢出风险，既可以通过配置更改默认的线程池参数，也可以通过完全自定义一个线程池 Bean 来实现。

1. 通过参数配置线程池

比如以下自定义线程池配置示例：

```
spring:
  task:
    execution:
      pool:
        coreSize: 5
        maxSize: 10
        queueCapacity: 50
        keepAlive: 10s
        allowCoreThreadTimeout: false
    scheduling:
      pool:
        size: 3
```

这里配置了任务执行线程池及任务调度线程池，任务执行线程池的配置如下：

- **核心线程数**：5。
- **最大线程数**：10。
- **任务队列大小**：50。
- **线程保活时长**：10 秒。
- **是否允许核心线程超时**：否。

任务调度线程池（默认为 1）的配置如下：

- **线程数**：3。

任务执行拒绝策略不能通过参数配置实现，但可以通过自定义线程池 Bean 的方式实现。

2. 自定义线程池 Bean

根据前面章节的任务自动配置过程可以知道，自定义线程池可以通过注册自定义的 TaskExecutorBuilder 或者 Executor 来实现，因为 TaskExecutorBuilder 不能自定义策略，所以这里通过注册自定义 Executor 的扩展类 ThreadPoolTaskExecutor 的方式来实现。

自定义线程池 Bean 的配置如以下示例所示。

```
@Configuration
public class TaskConfig {

    @Lazy
    @Bean
    public ThreadPoolTaskExecutor taskExecutor(TaskExecutionProperties
taskExecutionProperties) {
        ThreadPoolTaskExecutor taskExecutor = new ThreadPoolTaskExecutor();

        PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
        TaskExecutionProperties.Pool pool = taskExecutionProperties.getPool();
        map.from(pool::getQueueCapacity).to(taskExecutor::setQueueCapacity);
        map.from(pool::getCoreSize).to(taskExecutor::setCorePoolSize);
        map.from(pool::getMaxSize).to(taskExecutor::setMaxPoolSize);
        map.from(pool::getKeepAlive).asInt(Duration::getSeconds).to(taskExecutor::
setKeepAliveSeconds);
        map.from(pool::isAllowCoreThreadTimeout).to(taskExecutor::
setAllowCoreThreadTimeOut);
        map.from("my-task-").whenHasText().to(taskExecutor::setThreadNamePrefix);

        // 默认不设置就是 AbortPolicy
        taskExecutor.setRejectedExecutionHandler(new ThreadPoolExecutor.AbortPolicy());

        return taskExecutor;
    }

}
```

只要命名为 taskExecutor，@Async 注解就会自动选用这个线程池，也可以配置多个线程池，不同的任务类型使用不同的线程池，比如这里再注册一个 taskExecutor2，如以下配置示例所示。

```
@Configuration
public class TaskConfig {

    @Lazy
```

```
    @Bean
    public ThreadPoolTaskExecutor taskExecutor1(TaskExecutionProperties taskExecution-
Properties) {
        ThreadPoolTaskExecutor taskExecutor = new ThreadPoolTaskExecutor();

        PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
        TaskExecutionProperties.Pool pool = taskExecutionProperties.getPool();
        map.from(pool::getQueueCapacity).to(taskExecutor::setQueueCapacity);
        map.from(pool::getCoreSize).to(taskExecutor::setCorePoolSize);
        map.from(pool::getMaxSize).to(taskExecutor::setMaxPoolSize);
        map.from(pool::getKeepAlive).asInt(Duration::getSeconds).to (taskExecutor::
setKeepAliveSeconds);
        map.from(pool::isAllowCoreThreadTimeout).to(taskExecutor::
setAllowCoreThreadTimeOut);
        map.from("my-task1-").whenHasText().to(taskExecutor::setThreadNamePrefix);

        // 默认不设置就是 AbortPolicy
        taskExecutor.setRejectedExecutionHandler(new ThreadPoolExecutor.AbortPolicy());

        return taskExecutor;
    }

    @Lazy
    @Bean
    public ThreadPoolTaskExecutor taskExecutor2() {
        ThreadPoolTaskExecutor taskExecutor = new ThreadPoolTaskExecutor();

        PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
        TaskExecutionProperties.Pool pool = taskExecutionProperties.getPool();
        map.from(10).to(taskExecutor::setQueueCapacity);
        map.from(3).to(taskExecutor::setCorePoolSize);
        map.from(5).to(taskExecutor::setMaxPoolSize);
        map.from(20).to(taskExecutor::setKeepAliveSeconds);
        map.from(true).to(taskExecutor::setAllowCoreThreadTimeOut);
        map.from("my-task2-").whenHasText().to(taskExecutor::setThreadNamePrefix);

        return taskExecutor;
    }


}
```

如果存在多个线程池 Bean，但没有一个被命名为 taskExecutor，那么使用 @Async 注解时就需

要指定要使用的线程池 Bean 的名称，示例如下：

```
@Slf4j
@Component
public class SimpleTask {

    @Async("taskExecutor2")
    @Scheduled(cron = "*/3 * * * * *")
    public void printTask() {
        log.info("这是一个简单的计划任务。");
    }

}
```

这里指定了使用 taskExecutor2 线程池，如果不指定，也可以在线程池 Bean 上使用 @Primary 标识默认的线程池，如果 @Async 注解没有指定线程池名称就会使用默认的线程池，比如：

```
@Configuration
public class TaskConfig {

    @Lazy
    @Bean
    @Primary
    public ThreadPoolTaskExecutor taskExecutor1(TaskExecutionProperties
taskExecutionProperties) {

// ...
```

这里指定了 taskExecutor1 为默认的线程池，@Async 注解就能默认直接使用这个线程池。然后重启应用，再来观察任务日志，如以下日志所示。

```
--- [     my-task1-1] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-2] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-3] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-4] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-5] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-1] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-2] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-3] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-4] c...SimpleTask       : 这是一个简单的计划任务。
--- [     my-task1-5] c...SimpleTask       : 这是一个简单的计划任务。
```

从日志可以看到，自定义任务线程池后，执行任务的线程名称和任务执行频次都有了变化，更

多的任务线程池的扩展用法可以参考本节内容。

自定义任务调度不再赘述。

8.2 Quartz 计划任务

8.2.1 概述

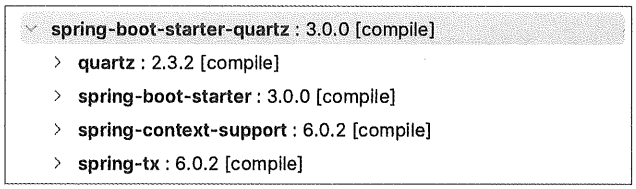
一般简单的计划任务可以使用 Spring 计划任务，更为复杂的计划任务建议使用专业的 Quartz 框架，或者其他分布式任务框架，比如 Elastic Job、xxl-job 等。据笔者了解，它们也都支持 Spring Boot Starter 一站式启动器快速集成，本节就以 Quartz 为例进行介绍，这三个框架中只有 Quartz 是 Spring Boot 官方支持的。

Quartz 是一款老牌、开源的 Java 作业调度框架，功能十分丰富，也十分成熟稳定，自笔者工作以来，就一直流行至今，专业且强大，是 Java 领域使用最广泛的框架之一。

Spring Boot 除了自动配置 Spring 自带的计划任务和调度框架，还支持对 Quartz 作业调度框架的自动配置，提供了 spring-boot-starter-quartz 一站式启动器，如以下依赖所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-quartz</artifactId>
</dependency>
```

只要加入该启动器依赖就能实现自动配置，它会自动引入 Quartz 框架及相关必要的依赖，如下图所示。



自动配置类为 QuartzAutoConfiguration，通过自动注册 Spring 中的 SchedulerFactoryBean 就能完成 Scheduler 调度器的创建、配置、生命周期管理等。对应的参数绑定类为 QuartzProperties，通过一系列 spring.quartz 参数可以自定义 SchedulerFactoryBean 配置。

8.2.2 实现计划任务

和前面的章节一样，这里使用 Quartz 实现了一个简单的计划任务，每 3 秒打印一次日志，如下面的示例所示。

```
@Slf4j
public class SimpleTask extends QuartzJobBean {

    @Override
    protected void executeInternal(JobExecutionContext context) {
        log.info("这是一个简单的 Quartz 计划任务。");
    }

}
```

在 Spring Boot 中，只需要继承 Spring 提供的 QuartzJobBean 抽象类并实现其 executeInternal 抽象方法即可，QuartzJobBean 抽象类实现并封装了 Quartz 的 Job 作业接口。

和 Spring 任务不同的是，Quartz 需要针对每个任务提供额外的配置，以下几个类型的 Bean 也会自动关联到 Scheduler 调度器：

- JobDetail：用于创建一个具体的作业实例。
- Calendar：用于指定 / 排除特定的时间。
- Trigger：用于定义作业触发时机。

下面是 JobDetaill 和 Trigger 的使用配置示例：

```
@Configuration
public class TaskConfig {

    @Bean
    public JobDetail simpleTask() {
        return JobBuilder.newJob(SimpleTask.class)
                .withIdentity("simple-task")
                .withDescription("简单任务")
                .storeDurably()
                .build();
    }

    @Bean
    public Trigger simpleTaskTrigger() {
```

```
        CronScheduleBuilder cronScheduleBuilder = CronScheduleBuilder.cronSchedule
("0/3 * * * * ? *");
        return TriggerBuilder.newTrigger()
                .withIdentity("simple-task-trigger")
                .forJob(simpleTask())
                .withSchedule(cronScheduleBuilder)
                .build();
    }

}
```

Quartz 的 Cron 表达式的格式如下所示。

```
秒 分 时 日 月 周 [年]
```

相比 Spring 计划任务，Quartz 可以多配置一位，即最后一位年，比如“0/3 * * * * ? 2022-2030”即表示 2022 到 2030 年之间每 3 秒执行一次。表达式可以是 6 位，也可以是 7 位，最后一位年并不是必需的。

任务配置好之后，查看应用启动后的部分日志，如以下启动日志所示。

```
RAMJobStore initialized.
Scheduler meta-data: Quartz Scheduler (v2.3.2) 'quartzScheduler' with instanceId
'NON_CLUSTERED'
  Scheduler class: 'org.quartz.core.QuartzScheduler' - running locally.
  NOT STARTED.
  Currently in standby mode.
  Number of jobs executed: 0
  Using thread pool 'org.quartz.simpl.SimpleThreadPool' - with 10 threads.
  Using job-store 'org.quartz.simpl.RAMJobStore' - which does not support persistence.
and is not clustered.

Starting Quartz Scheduler now
Scheduler quartzScheduler_$_NON_CLUSTERED started.
Started Application in 1.501 seconds (process running for 1.765)
```

可以看到一些 Quartz 初始化的相关信息，包括存储类型、线程池类型及数量，以及是否集群模式等。

任务执行日志如下所示。

```
--- [eduler_Worker-1] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-2] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
```

```
--- [eduler_Worker-3] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-4] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-5] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-6] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-7] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-8] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-9] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-10] c....SimpleTask       : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-1] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-2] c....SimpleTask        : 这是一个简单的 Quartz 计划任务。
```

Quartz 默认是异步多线程执行的，可以看到默认的 10 个线程在依次执行任务。

8.2.3 自定义配置

Quartz 默认会加载 Quartz 包目录中的 /org/quartz/quartz.properties 配置文件，包括 Quartz 调度器、线程池、持久化的相关配置，该配置文件的全部内容如下所示。

```
# Default Properties file for use by StdSchedulerFactory
# to create a Quartz Scheduler Instance, if a different
# properties file is not explicitly specified.
#

org.quartz.scheduler.instanceName: DefaultQuartzScheduler
org.quartz.scheduler.rmi.export: false
org.quartz.scheduler.rmi.proxy: false
org.quartz.scheduler.wrapJobExecutionInUserTransaction: false

org.quartz.threadPool.class: org.quartz.simpl.SimpleThreadPool
org.quartz.threadPool.threadCount: 10
org.quartz.threadPool.threadPriority: 5
org.quartz.threadPool.threadsInheritContextClassLoaderOfInitializingThread: true

org.quartz.jobStore.misfireThreshold: 60000

org.quartz.jobStore.class: org.quartz.simpl.RAMJobStore
```

Spring Boot 中的 QuartzProperties 参数绑定类提供了一个 Map 类型的 properties 参数，可以自定义覆盖这些配置，如以下源码所示。

```
@ConfigurationProperties("spring.quartz")
public class QuartzProperties {
```

```
    // ...

      /**
       * Additional Quartz Scheduler properties.
       */
    private final Map<String, String> properties = new HashMap<>();

    // ...

}
```

比如要更改默认的线程数，如以下配置示例所示。

```
spring:
  quartz:
    properties:
      org:
        quartz:
          threadPool:
            threadCount: 5
```

再重启应用查看任务执行日志，如以下日志所示。

```
--- [eduler_Worker-1] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-2] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-3] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-4] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-5] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-1] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
--- [eduler_Worker-2] c....SimpleTask         : 这是一个简单的 Quartz 计划任务。
```

从日志线程名可以看到，现在线程池中只有 5 个线程在执行任务。

8.2.4 持久化任务数据

首先看一下 QuartzProperties 参数类的源码：

```
@ConfigurationProperties("spring.quartz")
public class QuartzProperties {

    /**
```

```
     * Quartz job store type.
     */
    private JobStoreType jobStoreType = JobStoreType.MEMORY;

    ...

}

public enum JobStoreType {

    /**
     * Store jobs in memory.
     */
    MEMORY,

    /**
     * Store jobs in the database.
     */
    JDBC

}
```

如源码所示，Quartz 支持两种作业持久化存储方式：

- MEMORY。
- JDBC。

Quartz 存放作业的相关数据默认使用基于 MEMORY 内存的方式，基于内存意味着每次应用重启后作业数据就会丢失，如果需要持久化作业数据，那么 Spring Boot 还支持基于 JDBC 存储的自动配置，只要 DataSource 数据源已注册，就可以通过配置切换持久化方式，如以下配置示例所示。

```
spring:
  quartz:
    job-store-type: jdbc
    jdbc:
      initialize-schema: always
```

参数 initialize-schema 支持以下三个枚举值：

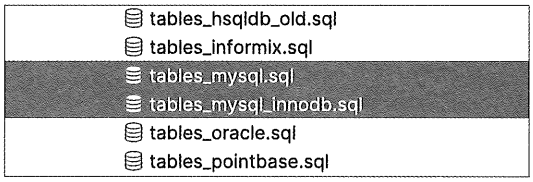
- ALWAYS：总是初始化数据库。
- EMBEDDED：只初始化嵌入式数据库。
- NEVER：从不初始化数据库。

参数 initialize-schema 的值为 ALWAYS，表示脚本在每次应用重启后都会删除重建表，这样持久化就没有什么意义了，可以在应用首次启动之前手动执行一次 Quartz 中自带的数据库脚本，或者通过 spring.quartz.jdbc.schema 参数指定自定义的数据库构建脚本。

Quartz 中自带的数据库脚本路径可以参考 QuartzProperties 参数类，如下面的源码所示。

```
@ConfigurationProperties("spring.quartz")
public class QuartzProperties {

    // ...

    public static class Jdbc {

        private static final String DEFAULT_SCHEMA_LOCATION = "classpath:org/quartz/impl/"
                + "jdbcjobstore/tables_@@platform@@.sql";
```

可以看到在 Quartz 对应的 org/quartz/impl/ 包目录中，Quartz 提供了多个主流数据库的构建脚本，以 MySQL 数据库为例，它提供了两个不同数据库引擎的构建脚本，如下图所示。



上面的一个脚本使用的是 MySQL 当前版本的默认数据库引擎，下面的脚本则使用指定的 InnoDB 数据库引擎。

在应用启动并完成数据库初始化后，可以通过 show tables 命令查看 Quartz 持久化相关的数据表，如以下参考命令所示。

```
mysql> show tables;
+----------------------------+
| Tables_in_javastack        |
+----------------------------+
| QRTZ_BLOB_TRIGGERS         |
| QRTZ_CALENDARS             |
| QRTZ_CRON_TRIGGERS         |
| QRTZ_FIRED_TRIGGERS        |
| QRTZ_JOB_DETAILS           |
| QRTZ_LOCKS                 |
| QRTZ_PAUSED_TRIGGER_GRPS   |
| QRTZ_SCHEDULER_STATE       |
| QRTZ_SIMPLE_TRIGGERS       |
```

```
| QRTZ_SIMPROP_TRIGGERS       |
| QRTZ_TRIGGERS               |
+-----------------------------+
11 rows in set (0.00 sec)
```

默认情况下以数据库中持久化的任务配置为准，比如现在修改任务触发机制为每 5 秒执行一次，如以下示例代码所示。

```
@Bean
public Trigger simpleTaskTrigger() {
    CronScheduleBuilder cronScheduleBuilder = CronScheduleBuilder.cronSchedule
("0/5 * * * * ? *");
    return TriggerBuilder.newTrigger()
            .withIdentity("simple-task-trigger")
            .forJob(simpleTask())
            .withSchedule(cronScheduleBuilder)
            .build();
}
```

重启应用，该任务还是每 3 秒执行一次，因为应用中的新配置不会覆盖数据库中持久化的配置，如果需要覆盖持久化的任务配置，则可以通过以下参数进行配置：

```
spring:
  quartz:
    overwrite-existing-jobs: true
```

这样应用中最新的任务配置就会覆盖数据库中已持久化的配置，再次重启应用，该任务就会变成每 5 秒执行一次。

8.2.5 动态维护任务

以上是通过 Bean 的方式配置任务，如果任务过多，应用就会出现大量的配置代码，不是很优雅，也不好维护。

除了使用 Bean 配置任务的方式，还可以使用前面介绍的 SchedulerFactoryBean 来动态维护任务，比如动态添加、删除任务等，我们把上面的 Bean 配置方式改一下，如以下配置示例所示。

```
@RequiredArgsConstructor
@Configuration
public class TaskConfig {

    public static final String SIMPLE_TASK = "simple-task";
```

```
    private final SchedulerFactoryBean schedulerFactoryBean;

    /**
     * 动态添加任务
     * @throws SchedulerException
     */
    @PostConstruct
    public void init() throws SchedulerException {
        Scheduler scheduler = schedulerFactoryBean.getScheduler();
        boolean exists = scheduler.checkExists(JobKey.jobKey(SIMPLE_TASK));
        if (!exists) {
            scheduler.scheduleJob(simpleTask(), simpleTaskTrigger());
        }
    }

    // @Bean
    public JobDetail simpleTask() {
        return JobBuilder.newJob(SimpleTask.class)
                .withIdentity(SIMPLE_TASK)
                .withDescription("简单任务")
                .storeDurably()
                .build();
    }

    // @Bean
    public Trigger simpleTaskTrigger() {
        CronScheduleBuilder cronScheduleBuilder = CronScheduleBuilder.cronSchedule
("0/3 * * * * ? *");
        return TriggerBuilder.newTrigger()
                .withIdentity("simple-task-trigger")
                .forJob(simpleTask())
                .withSchedule(cronScheduleBuilder)
                .build();
    }

}
```

首先把 @Bean 注解都注释掉，然后使用注入的 SchedulerFactoryBean 实例获取 Quartz 调度器来添加并启动任务，这里的逻辑是，如果任务没有持久化才添加并启动任务，读者可以自行扩展。

实际开发中可以把创建任务的各个方法抽取出来形成公共方法，这样做的好处就是既可以省去大量的配置代码，还可以把任务配置放到配置文件中或者配置中心来统一管理维护，如果有需要，还能以对外 API 接口的形式来动态维护任务。

第 9 章 Spring Boot 缓存与消息队列

本章会介绍 Spring Boot 缓存与消息队列的集成和使用，涉及 ActiveMQ、RabbitMQ、Kafka、Redis 等，这些都是 Java 应用必不可少的中间件技术，它们都是支撑互联网高吞吐量、高并发的基石。Spring Boot 为这些技术都提供了自动配置，并简化了这些第三方技术的使用难度。

9.1 缓存

9.1.1 概述

Spring 框架提供了对缓存操作的基本抽象：

- org.springframework.cache.Cache。
- org.springframework.cache.CacheManager。

Spring 框架只提供抽象，不提供具体的缓存存储，底层需要依赖第三方存储组件，如果当前应用没有注册 CacheManager 或者 CacheResolver 实例，那么 Spring Boot 会按以下顺序来检测缓存组件：

Generic → JCache（JSR-107）（EhCache 3、Hazelcast、Infinispan 等）→ Hazelcast → Infinispan → Couchbase → Redis → Caffeine → Cache2k → Simple

Spring Boot 在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中提供了对缓存的自动配置，自动配置类为 CacheAutoConfiguration，对应的参数绑定类为 CacheProperties，通过一系列 spring.cache.* 参数可以自定义缓存配置。

在 CacheAutoConfiguration 缓存自动配置类中引入了 CacheConfigurationImportSelector 配置选择器，如下面的源码所示。

```
@AutoConfiguration(after = { CouchbaseDataAutoConfiguration.class,
HazelcastAutoConfiguration.class,
HibernateJpaAutoConfiguration.class, RedisAutoConfiguration.class })
@ConditionalOnClass(CacheManager.class)
@ConditionalOnBean(CacheAspectSupport.class)
@ConditionalOnMissingBean(value = CacheManager.class, name = "cacheResolver")
@EnableConfigurationProperties(CacheProperties.class)
@Import({ CacheConfigurationImportSelector.class, CacheManagerEntityManagerFactoryDepends-
OnPostProcessor.class })
public class CacheAutoConfiguration {

    ...

static class CacheConfigurationImportSelector implements ImportSelector {

    @Override
    public String[] selectImports(AnnotationMetadata importingClassMetadata) {
        CacheType[] types = CacheType.values();
        String[] imports = new String[types.length];
        for (int i = 0; i < types.length; i++) {
            imports[i] = CacheConfigurations.getConfigurationClass(types[i]);
        }
        return imports;
    }

}
```

由该选择器通过 CacheConfigurations 缓存配置类来选择对应的缓存组件的配置类，如下面的源码所示。

```
final class CacheConfigurations {

private static final Map<CacheType, String> MAPPINGS;

static {
Map<CacheType, String> mappings = new EnumMap<>(CacheType.class);
mappings.put(CacheType.GENERIC, GenericCacheConfiguration.class.getName());
mappings.put(CacheType.HAZELCAST, HazelcastCacheConfiguration.class.getName());
mappings.put(CacheType.INFINISPAN, InfinispanCacheConfiguration.class.getName());
mappings.put(CacheType.JCACHE, JCacheCacheConfiguration.class.getName());
```

```
mappings.put(CacheType.COUCHBASE, CouchbaseCacheConfiguration.class.getName());
mappings.put(CacheType.REDIS, RedisCacheConfiguration.class.getName());
mappings.put(CacheType.CAFFEINE, CaffeineCacheConfiguration.class.getName());
mappings.put(CacheType.CACHE2K, Cache2kCacheConfiguration.class.getName());
mappings.put(CacheType.SIMPLE, SimpleCacheConfiguration.class.getName());
mappings.put(CacheType.NONE, NoOpCacheConfiguration.class.getName());
MAPPINGS = Collections.unmodifiableMap(mappings);
}

...

}
```

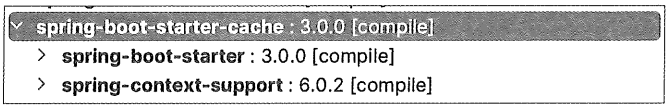
mappings 集合维护了所有 Spring Boot 支持的缓存组件的映射关系，由具体的缓存类型 CacheType 映射到具体的缓存配置类，以 Redis 为例，它会选择并启用 RedisCacheConfiguration 缓存配置类。

9.1.2 开启缓存

Spring Boot 为缓存提供了一个 spring-boot-starter-cache 一站式启动器，如以下依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

它主要添加了 spring-boot-starter 基础依赖和缓存所需要的 spring-context-support 依赖，如下图所示。



加入该启动器依赖后，在应用主配置上添加 @EnableCaching 注解即可开启缓存。还可以通过参数指定要使用的缓存组件，比如，使用 Redis 作为缓存组件，如下面的配置示例所示。

```
spring:
  cache:
    type: redis
```

也可以在某种应用中或者某些场合下禁用缓存，如下面的配置示例所示。

```
spring:
  cache:
    type: none
```

缓存类型 type 的可选配置值有 COUCHBASE、GENERIC、REDIS、HAZELCAST、CACHE2K、CAFFEINE、JCACHE、INFINISPAN、NONE、SIMPLE。

读者也可以参考 CacheType 枚举类进行配置。

9.1.3 默认简单缓存

在前面的缓存概述中，笔者介绍了 Spring Boot 检测缓存组件的顺序，可以看到最后检测到的是 Simple 类型，也就是说，当检测到没有使用任何缓存组件时，Spring Boot 默认使用的是基于内存的简单缓存，它使用基于线程安全的 ConcurrentHashMap 集合进行存储。

从 CacheConfigurations 缓存配置类映射集合中可以找到 Simple 类型对应的配置类是 SimpleCacheConfiguration 类，该类的源码如下所示。

```
@Configuration(proxyBeanMethods = false)
@ConditionalOnMissingBean(CacheManager.class)
@Conditional(CacheCondition.class)
class SimpleCacheConfiguration {

@Bean
ConcurrentMapCacheManager cacheManager(CacheProperties cacheProperties,
CacheManagerCustomizers cacheManagerCustomizers) {
ConcurrentMapCacheManager cacheManager = new ConcurrentMapCacheManager();
List<String> cacheNames = cacheProperties.getCacheNames();
if (!cacheNames.isEmpty()) {
cacheManager.setCacheNames(cacheNames);
}
return cacheManagerCustomizers.customize(cacheManager);
}

}
```

它注册了一个 ConcurrentMapCacheManager 缓存管理器实例，该类的源码如下所示。

```
public class ConcurrentMapCacheManager implements CacheManager, BeanClassLoaderAware {

    private final ConcurrentMap<String, Cache> cacheMap = new ConcurrentHashMap(16);
```

```
    ...

}
```

可以看到它是使用 ConcurrentHashMap 集合进行内存式存储的，这意味着一旦应用重启就会丢失所有缓存中的数据，所以建议仅在学习和测试中使用这种类型，不建议在线上及访问量较大的应用中使用，否则数据丢失会造成缓存雪崩效应，导致大量请求到达后端数据库，从而导致数据库崩溃，甚至整个系统崩溃。

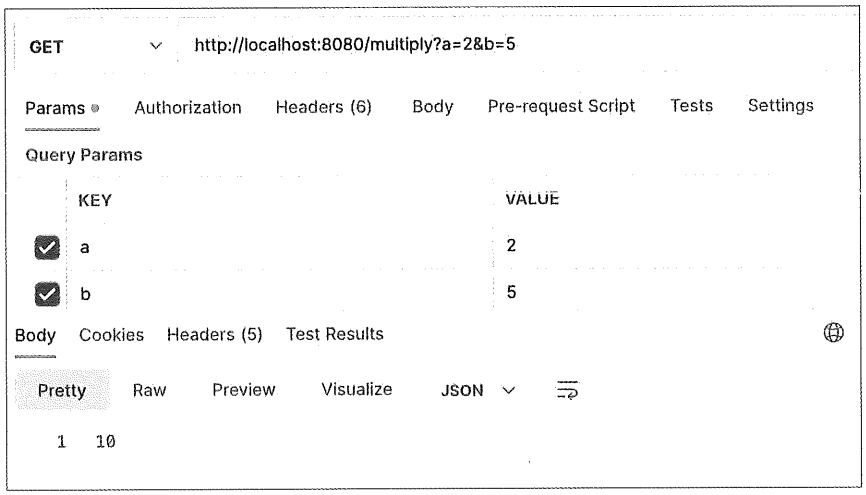
下面在不使用任何缓存组件的情况下来应用简单缓存，首先创建一个两数相乘的缓存服务类，如下面的示例代码所示。

```
@Slf4j
@Service
public class CacheService {

    @Cacheable("calc")
    public int multiply(int a, int b) {
        int c = a * b;
        log.info("{} * {} = {}", a, b, c);
        return c;
    }

}
```

在服务类入口方法上使用 @Cacheable 注解标识该方法使用缓存，然后创建一个缓存测试接口，注入并调用缓存服务类来获取两个数的乘积结果，如下面的示例代码所示。

```
@RequiredArgsConstructor
@RestController
public class CacheController {

    private final CacheService cacheService;

    @RequestMapping("/multiply")
    public int multiply(@RequestParam("a") int a,
                        @RequestParam("b") int b) {
        return cacheService.multiply(a, b);
    }

}
```

再访问该接口，测试效果如下图所示。



首次调用该接口时会进入设置了缓存的 CacheService#multiply 方法，然后输出日志，并把返回数据存储到缓存中，后续调用时，只要全部参数相同就不会进入该缓存方法，而是直接从缓存中读取数据并返回。如果任意一个参数不同，则会当作一个新缓存来处理。

这里的 @Cacheable 注解便是 Spring AOP 的切面编程的应用，背后的原理便是动态代理，通过在程序运行时创建代理对象来代理原始对象中的方法，然后在调用原始对象方法的前后植入逻辑以实现缓存操作的目的。

9.1.4 Redis 缓存

本节还以 Redis 数据库为例，介绍 Spring Boot 如何集成 Redis 来使用缓存，Redis 的安装、集成在第 7 章介绍过了，这里不再赘述。

首先添加 spring-boot-starter-data-redis 一站式启动器依赖，如下面的配置所示。

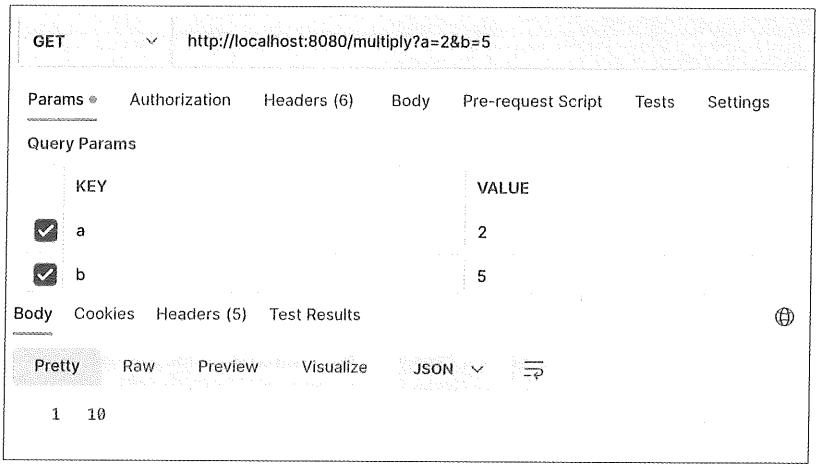
```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

只需要一个依赖，Spring Boot 就会自动配置 Redis 缓存管理器，在 RedisCacheConfiguration 配置类中注册了一个 RedisCacheManager 缓存管理器，也就是 CacheManager 接口的 Redis 实现。

这里使用默认的 Redis 本地连接配置和缓存配置，如有需要请根据 RedisProperties 参数类进行

配置，缓存相关的配置则可以根据 CacheProperties 参数类进行定制，Redis 缓存配置的参数形式为 spring.cache.redis.*。

重启应用，再访问上一节简单缓存中添加的接口，测试结果如下图所示。



在使用了几次不同参数调用之后，查询方法是否在 Redis 中缓存了结果：

```
127.0.0.1:6379> keys *
1) "calc::SimpleKey [2,3]"
2) "calc::SimpleKey [2,5]"
3) "calc::SimpleKey [2,51]"
```

可以看到 Redis 缓存了方法结果，格式为：

方法名 ::KEY [*方法参数数组*]

Spring Boot 支持在应用启动时创建缓存名称，还能对 Redis 中的缓存设置过期时间，如下面的配置示例所示。

```
spring:
  cache:
    cache-names: "calc,test"
    redis:
      time-to-live: "10s"
```

这里创建了 calc 和 test 两个缓存名称，并且都只有 10 秒的有效期，也就是说，10 秒之后缓存就会失效，调用相同参数的接口会重新进入方法并重新缓存。

application 配置文件只能对所有缓存进行统一设置，如果需要对缓存进行更多配置控制，比如

对多个不同的缓存设置不同的过期时间，则可以注册一个 =RedisCacheManagerBuilderCustomizer 类型的实例，参考示例如下所示。

```
@Configuration
public class CacheConfiguration {

    /**
     * 优先配置文件中的配置
     * @return
     */
    @Bean
    public RedisCacheManagerBuilderCustomizer myRedisCacheManagerBuilderCustomizer() {
        return (builder) -> builder
                .withCacheConfiguration("calc", RedisCacheConfiguration
                        .defaultCacheConfig().entryTtl(Duration.ofSeconds(5)))
                .withCacheConfiguration("test", RedisCacheConfiguration
                        .defaultCacheConfig().entryTtl(Duration.ofMinutes(10)));

    }

}
```

这里设置了 calc 类型的缓存过期时间为 5 秒，test 类型的缓存过期时间为 10 秒，通过 RedisCacheManagerBuilderCustomizer 自定义的缓存配置要优先于 application 配置文件中的缓存配置，也就是说，如果同时设置了缓存配置，则以 RedisCacheManagerBuilderCustomizer 中的为准。

还可以添加自定义的 RedisCacheConfiguration 配置类，通过覆盖缓存相关 Bean 的方式来完全控制默认的组件配置，这在想自定义默认序列化策略时会很有用。

> 其他缓存组件都是类似的用法，可以参考其自动配置类和参数配置类。

9.2 消息系统

9.2.1 概述

Java 程序员在面试时，经常会被面试官问到的一个问题：

> 有了多线程，为什么还要消息队列？

笔者曾经就被问到过这样的问题，这两个显然不能放在一起比较，它们是两个不同维度的东西，

所应用的场景和发挥的能力也不一样，一般是结合使用的。所以，如果一定要比较，那么可以从下面几个方面来分析。

1. 业务解耦

使用多线程，应用的所有业务代码都彼此关联在一起，耦合性太强。消息队列则不存在业务关联关系，各个业务系统彼此独立，各个系统之间通过发送消息来实现异步消费，可以很方便地进行解耦。

2. 削峰填谷

使用多线程，资源消耗还是在本机器上，如果要处理突如其来的高并发流量，则不能很快地进行动态扩展。而消息队列独立存储消息，充分利用了各个应用服务器资源，可以将多条消息按照一定的订阅规则分发到多个消费者的应用服务器上，可以很好地应对高并发流量并进行削峰填谷。

3. 故障处理

使用多线程，一是很难跟踪每个线程的执行情况，二是出现问题时很难对问题线程进行恢复处理。而消息队列则拥有标准的消息管理机制，可以可视化地看到每条消息的执行情况，出现问题时可以设置重试策略，或者手动重新发送消息等。

4. 平台限制

使用多线程会受限于具体的编程语言，它解决的是编程层面的问题。而消息队列则不限定具体的编程语言，消息的存储和实现都在消息队列服务器端，它解决的是架构层面的问题。

消息队列的优势远不止上面几点，虽然相较于多线程有很多优势，但需要引进第三方消息组件，所以它同时会给系统带来可用性、复杂性、一致性等诸多问题，比如要考虑消息中间件的稳定性问题，以及消息重复消费、消息丢失、顺序消费等问题。

Spring Boot 支持以下几种常用的消息协议 / 系统：

- JMS，全称为 Java Message Service，即 Java 消息服务。Spring 框架提供了 JmsTemplate 模板，它简化了 JMS API 的使用，Spring Boot 还提供了该模板的自动配置。
- AMQP，全称为 Advanced Message Queuing Protocol，即高级消息队列协议。Spring 框架也为 AMQP 的主要实现 RabbitMQ 提供了类似的模板 RabbitTemplate，Spring Boot 还为 RabbitTemplate 和 RabbitMQ 提供了自动配置。
- Apache Kafka，Spring 框架同样提供了对 KafkaTemplate 模板的支持，Spring Boot 则提供了类似的自动配置。

- STOMP，全称为 Simple Text-Orientated Messaging Protocol，即面向消息的简单文本协议，Spring Boot 同样提供了类似的自动配置。

可以说 Spring Boot 几乎支持所有主流的消息系统，它为“开箱即用”的 API 提供了自动配置，Spring Boot 在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中提供了这几种消息系统的自动配置，如下所示。

- JmsAutoConfiguration。
- ArtemisAutoConfiguration。
- RabbitAutoConfiguration。
- KafkaAutoConfiguration。

这几种消息协议 / 系统的 Spring 依赖如下表所示。

消息协议 / 系统	Spring 依赖
JMS（ActiveMQ Artemis）	spring-boot-starter-artemis
AMQP（RabbitMQ）	spring-boot-starter-amqp
Kafka	spring-kafka

需要注意的是，Spring Boot 提供了对 Kafka 消息系统的自动配置类，不过并没有提供 Starter 启动器依赖，它是通过 spring-kafka 这个项目提供支持的。另外，目前 RocketMQ 在国内的应用也比较多，并已成为 Apache 顶级项目。虽然 Spring Boot 没有提供官方的对 RocketMQ 的支持，但是可以在 RocketMQ/Spring Cloud Alibaba 中找到相关的 Starter 启动器依赖。

本章将以目前最主流的、也是官方支持的消息系统 ActiveMQ、RabbitMQ 和 Kafka 为例，介绍 Spring Boot 如何集成并简单使用这些消息系统。

9.2.2 ActiveMQ

1. 概述

JMS 是 Java 开发平台上面向消息通信的技术标准和规范，可用于在多个应用之间或分布式系统中发送消息进行异步通信。它是一个与具体平台无关的消息通信 API，绝大多数提供商都支持 JMS 规范，JMS 最主流的开源技术有 ActiveMQ、RocketMQ 等。

直白点理解，JMS 和 JDBC 的概念一样，JMS 只是一种消息规范，提供了一套消息通信 API，ActiveMQ、RocketMQ 等都是 JMS 规范的实现，这两个都是 Apache 旗下的顶级开源项目，也都支持 JMS 规范。

JMS 规范目前有以下两个版本：

- JMS 1.1。
- JMS 2.0。

JMS 2.0 主要在 JMS 1.1 的基础上进行了改进，简化了消息通信 API，同时向后兼容 JMS 1.1 的 API，JMS 2.0 不仅在易用性方面得到了大大提升，也大大减少了开发代码量。

ActiveMQ 是 JMS 规范最早、最流行、最典型的代表，由 Java 语言开发，是 Apache 开源的最主流的消息系统之一，ActiveMQ 主要有以下两个版本：

- ActiveMQ Classic：它指的是 ActiveMQ 最原始、最经典的版本，基于 JMS 1.1 规范。
- ActiveMQ Artemis：它指的是下一代的 ActiveMQ，用来代替 ActiveMQ Classic。ActiveMQ Artemis 不但支持最新的 JMS 2.0 规范，还支持 AMQP、STOMP、MQTT、OpenWire 等多种消息协议，底层还使用了基于 Netty 的异步 I/O，大大提升了性能。

ActiveMQ 5.x 及之前的版本叫 ActiveMQ Classic，现在的 ActiveMQ 6.x+ 叫 ActiveMQ Artemis，功能全面增强，所以现在推荐基于 ActiveMQ Artemis 集成消息系统。

2. ActiveMQ Artemis 环境搭建

1）安装 ActiveMQ Artemis

去官网下载最新的二进制包，地址为链接 10。

笔者这里下载了最新的版本 apache-artemis-2.26.0-bin.tar.gz，下载之后进行解压，并设置 ARTEMIS_HOME 和 PATH 环境变量，参考命令如下所示。

```
$ tar -zvxf apache-artemis-2.26.0-bin.tar.gz
$ vi ~/.bash_profile
$ source ~/.bash_profile
```

2）创建一个 Broker 实例

Broker 实例是一个包含所有配置和运行数据的目录，所以建议不要在 ARTEMIS_HOME 主目录下创建 Broker 实例，避免下次升级 ActiveMQ Artemis 时造成 Broker 实例数据丢失。

创建一个 Broker 目录及实例，参考命令如下：

```
$ mkdir /data/ActiveMQ
$ cd /data/ActiveMQ
$ artemis create mybroker
```

这里的 artemis 命令也支持 Windows 版本（artemis.cmd），在创建 Broker 实例时，控制台会提示创建默认用户名等初始化信息，如下面的信息所示。

```
Creating ActiveMQ Artemis instance at: /data/ActiveMQ/mybroker

--user:
Please provide the default username:
admin

--password: is mandatory with this configuration:
Please provide the default password:


--allow-anonymous | --require-login:
Allow anonymous access?, valid values are Y,N,True,False
n

Auto tuning journal ...
done! Your system can make 0.05 writes per millisecond, your journal-buffer-timeout
will be 19964000

You can now start the broker by executing:

   "/data/ActiveMQ/mybroker/bin/artemis" run

Or you can run the broker in the background using:

   "/data/ActiveMQ/mybroker/bin/artemis-service" start
```

笔者这里创建了一个简单的用户 admin/123456，且不允许匿名访问。

3）启动 Broker 实例

因为 ActiveMQ 是由 Java 语言开发的，所以运行 ActiveMQ Artemis 最新版本需要 JDK 11+ 环境。

在上一节创建 Broker 实例的控制台可以看到以下启动命令：

```
$ /data/ActiveMQ/mybroker/bin/artemis
```

如果想在后台启动，则参考命令如下：

```
$ /data/ActiveMQ/mybroker/bin/artemis-service
```

笔者这里使用后者，在后台启动 Broker 实例的参考命令如下：

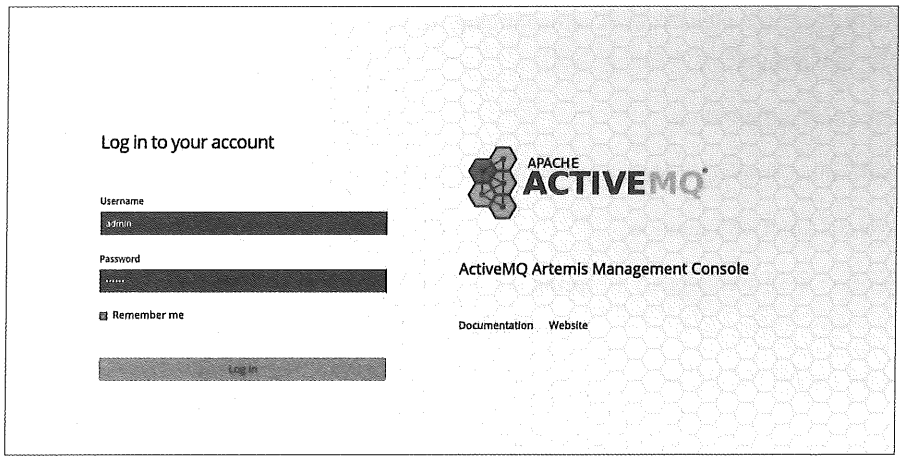
```
$ /data/ActiveMQ/mybroker/bin/artemis-service start
Starting artemis-service
artemis-service is now running (17397)
```

除了使用 start 参数，还支持使用 stop、restart、force-stop、status 等命令参数，启动之后可以访问 ActiveMQ Artemis 控制台了。

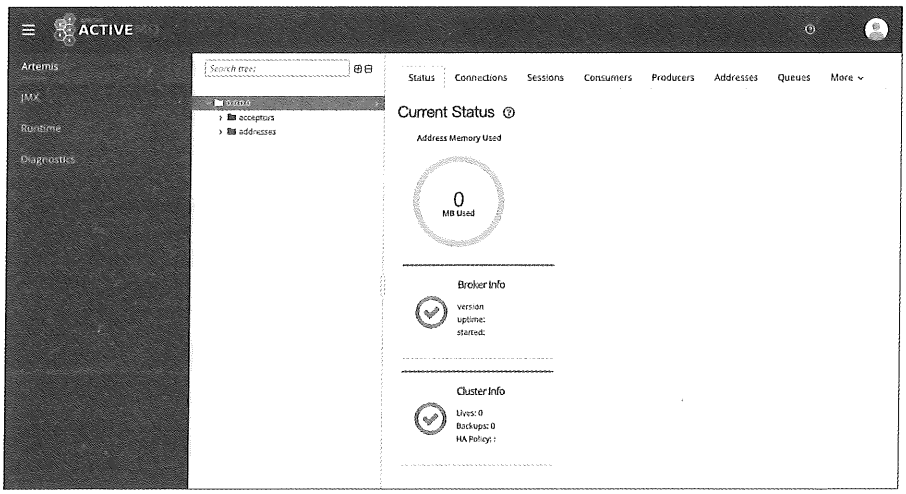
ActiveMQ Artemis 控制台的登录地址如下：

```
http://localhost:8161/
```

输入刚才创建 Broker 实例时的用户名和密码即可登录，如下图所示。



登录后的结果如下图所示。



在 ActiveMQ Artemis 控制台可以查看一些实例运行的基础信息，以及连接、会话、生产者、消费者、队列等相关信息的搜索和操作菜单。

3. Spring Boot 集成 ActiveMQ Artemis

Spring Boot 提供了 spring-boot-starter-artemis 一站式启动器，如下面的依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-artemis</artifactId>
</dependency>
```

在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了以下 JMS 和 ActiveMQ 相关的自动配置类：

- JmsAutoConfiguration。
- JndiConnectionFactoryAutoConfiguration。
- ArtemisAutoConfiguration。

只需配置一系列 spring.jms.* 和 spring.artemis.* 参数就可以完成自动配置，如下面的配置示例所示。

```
spring:
  jms:
    cache:
      enabled: true # 默认 true
      session-cache-size: 5 # 默认 1
  artemis:
    mode: native
    broker-url: tcp://localhost:61616
    user: admin
    password: 123456
```

在 ArtemisAutoConfiguration 自动配置类中，spring.jms.cache.enabled 默认为缓存开启状态（true），默认注册使用的是可缓存的 CachingConnectionFactory 工厂创建连接池，否则（false）使用的是 ActiveMQConnectionFactory。

如果使用了 spring.artemis.pool.enabled 连接池开启参数，则此时使用的是原生的 JmsPoolConnectionFactory 工厂创建连接池，如以下配置所示。

```
spring:
```

```
  jms:
    cache:
      enabled: true # 默认 true
      session-cache-size: 5 # 默认 1
  artemis:
    mode: native
    broker-url: tcp://localhost:61616
    user: admin
    password: 123456
    pool:
      enabled: true # 默认 false
      max-connections: 50 # 默认 1
      idle-timeout: 5s # 默认 30s
      max-sessions-per-connection: 100 # 默认 500
```

此外还需要加上 pooled-jms 连接池的依赖，如以下依赖配置所示。

```
<dependency>
    <groupId>org.messaginghub</groupId>
    <artifactId>pooled-jms</artifactId>
</dependency>
```

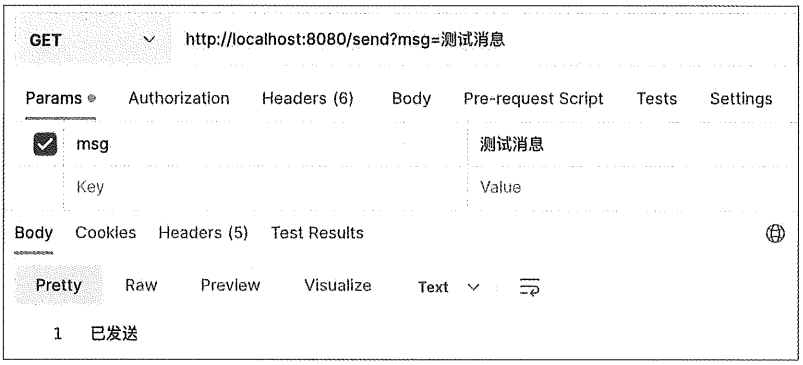
Spring Boot 已经对该依赖纳入了管理，所以无须额外的版本号，更多的配置参数可以查看 JMS 和 ActiveMQ 对应的参数绑定类 JmsProperties 和 ArtemisProperties。

这里创建一个发送消息接口，以及接收消息的队列监听方法。在发送消息时可以直接注入 JmsTemplate 模板并使用，然后通过 @JmsListener 注解接收消息，如以下示例所示。

```
@RequiredArgsConstructor
@RestController
@Slf4j
public class MsgController {

    private final JmsTemplate jmsTemplate;

    /**
     * 发送消息
     * @param msg
     * @return
     */
    @RequestMapping("/send")
    public String sendMsg(@RequestParam("msg") String msg) {
        jmsTemplate.convertAndSend("test-queue", msg);
```

```
        return "已发送";
    }

    /**
     * 接收消息
     * @param msg
     */
    @JmsListener(destination = "test-queue")
    public void receiveMsg(String msg) {
        log.info("收到 ActiveMQ 消息: {}", msg);
    }

}
```

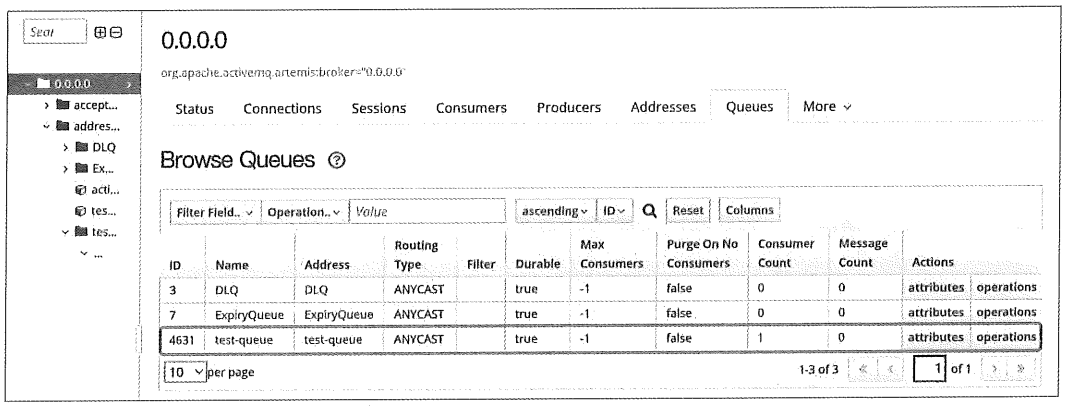
调用发送消息接口进行测试，如下图所示。



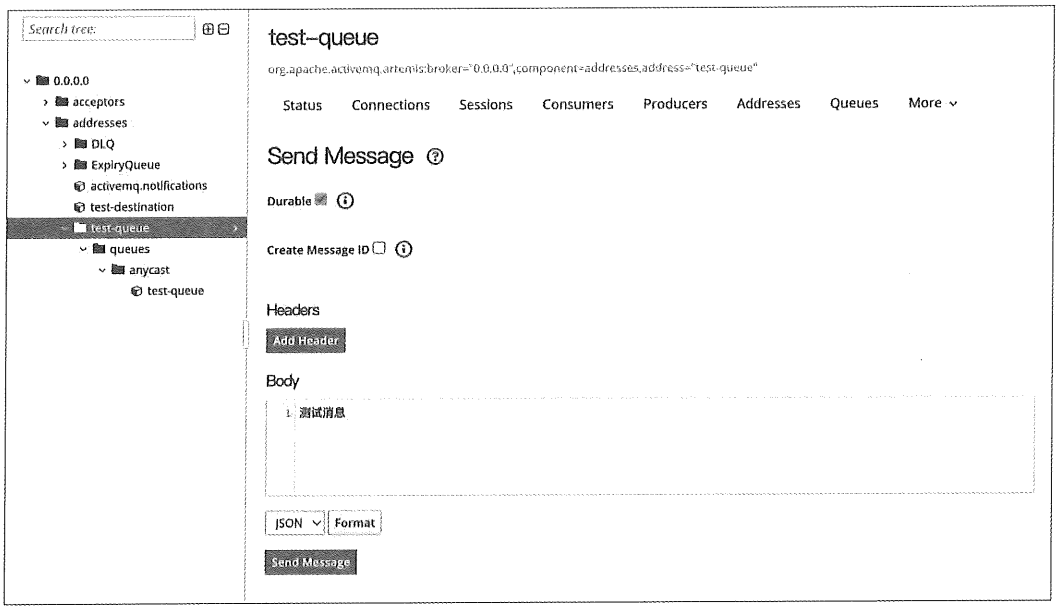
结果如下图所示。

```
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http)
c.j.springboot.activemq.Application      : Started Application in 2.244 seconds (p
o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet '
o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet
o.s.web.servlet.DispatcherServlet        : Completed initialization in 0 ms
c.j.springboot.activemq.MsgController    : 收到 ActiveMQ 消息: 测试消息
```

从日志可以看到消息成功发送并被消费了，再回到 ActiveMQ 控制台 Queues 队列操作页面，可以看到队列也自动创建了，如下图所示。



单击具体的队列可以对该队列进行查看等操作，比如手动发送消息到该队列，如下图所示。



以上只是介绍了如何使用 Spring Boot 集成 JMS&ActiveMQ，以及进行简单的消息发送和接收操作，更多的应用读者可以参考其自动配置类并进行扩展。

9.2.3 RabbitMQ

1. 概述

RabbitMQ 是基于 AMQP 的轻量级、可靠的、可扩展、可移植的消息系统，基于 Erlang 语言编

写，是目前应用最广泛的 MQ 系统之一，在使用 Spring Boot 集成 RabbitMQ 之前需要了解以下核心知识点。

RabbitMQ 包含的重要组件如下：

- ConnectionFactory（**连接管理器**）：用于应用与 RabbitMQ 之间建立连接的管理器。
- Channel（**信道**）：消息推送时使用的通道。
- Exchange（**交换器**）：用于分发消息。
- Queue（**队列**）：用于存储生产者写入的消息。
- RoutingKey（**路由键**）：用于交换器把生成者的消息写入不同的队列。
- BindingKey（**绑定键**）：用于将交换器绑定到不同的队列。

RabbitMQ 交换器（Exchange）支持的消息分发模式如下：

- direct（**默认**）：路由模式，发送消息时指定不同的 Routing Key，交换器会根据不同的 Routing Key 将消息写入不同的队列。
- fanout：广播模式，交换器会将消息发送到它绑定的所有队列中，也是性能最好、使用最多的模式。
- headers：交换器使用消息内容中的 headers 属性值与队列定义的参数进行匹配，性能较差，几乎不使用。
- topic：匹配模式，和路由模式差不多，但允许使用通配符匹配来分发消息。

RabbitMQ 的使用非常广泛，几乎是 Java 应用领域消息队列事实上的标准。

2. RabbitMQ 环境搭建

1）安装 RabbitMQ

RabbitMQ 几乎支持所有主流的操作系统，包括 Docker 镜像，这里以 CentOS 为例进行安装演示。

RabbitMQ 是基于 Erlang 语言编写的，所以需要安装 Erlang 语言环境，和 Java 应用一样，不同的 RabbitMQ 版本也需要依赖兼容的 Erlang 语言版本，下图是 RabbitMQ 官方提供的兼容表。

RabbitMQ and Erlang/OTP Compatibility Matrix

The table below provides an Erlang compatibility matrix of currently supported RabbitMQ release series. For RabbitMQ releases that have reached end of life, see Unsupported Series Compatibility Matrix.

RabbitMQ version	Minimum required Erlang/OTP	Maximum supported Erlang/OTP	Notes
3.11.2 3.11.1 3.11.0	25.0	25.1	• As of Erlang 25.1, OpenSSL 3.0 support in Erlang is considered to be mature enough for production. • Erlang 25 before 25.0.2 is affected by CVE-2022-37026, a CVE with critical severity (CVSS 3.x Base Store: 9.8)
3.10.10 3.10.9 3.10.8	24.2	25.1	• As of Erlang 25.1, OpenSSL 3.0 support in Erlang is considered to be mature enough to consider for production. • Erlang 25 before 25.0.2 and 24 before 24.3.4.2 are affected by CVE-2022-37026, a CVE with critical severity (CVSS 3.x Base Store: 9.8)
3.10.7 3.10.6 3.10.5	23.2	25.1	• Erlang 25 is the recommended series. • Erlang 25 before 25.0.2 and 24 before 24.3.4.2 are affected by CVE-2022-37026, a CVE with critical severity (CVSS 3.x Base Store: 9.8) • Erlang 23 support was discontinued on July 31st, 2022.

市面上有多个可用于 Erlang/OTP 的 RPM 安装包，但推荐使用来自 RabbitMQ 团队的零依赖 Erlang RPM 安装包，它紧跟最新的 Erlang/OTP 补丁发布，下载地址为链接 11。

可以通过命令一并下载安装，相关参考命令如下：

```
$sudo yum update -y
$ sudo yum -q makecache -y --disablerepo=' *' --enablerepo=' rabbitmq_erlang'
--enablerepo=' rabbitmq_server'
$ sudo yum install socat logrotate -y
$ sudo yum install --repo rabbitmq_erlang --repo rabbitmq_server erlang rabbitmq-server -y
```

2）启动 RabbitMQ

安装 RabbitMQ 捆绑包时，RabbitMQ 服务器默认不作为守护进程启动，在系统启动时默认启动守护进程，需要以管理员身份运行命令，参考命令如下：

```
$ sudo chkconfig rabbitmq-server on
```

启动、状态查询、停止的相关参考命令如下：

```
$ service rabbitmq-server start
$ service rabbitmq-server status
```

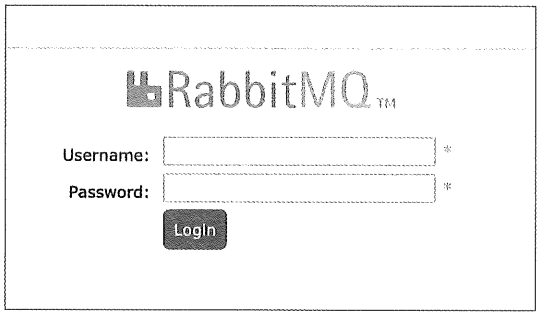
```
$ service rabbitmq-server stop
```

3）启用 Web控制台

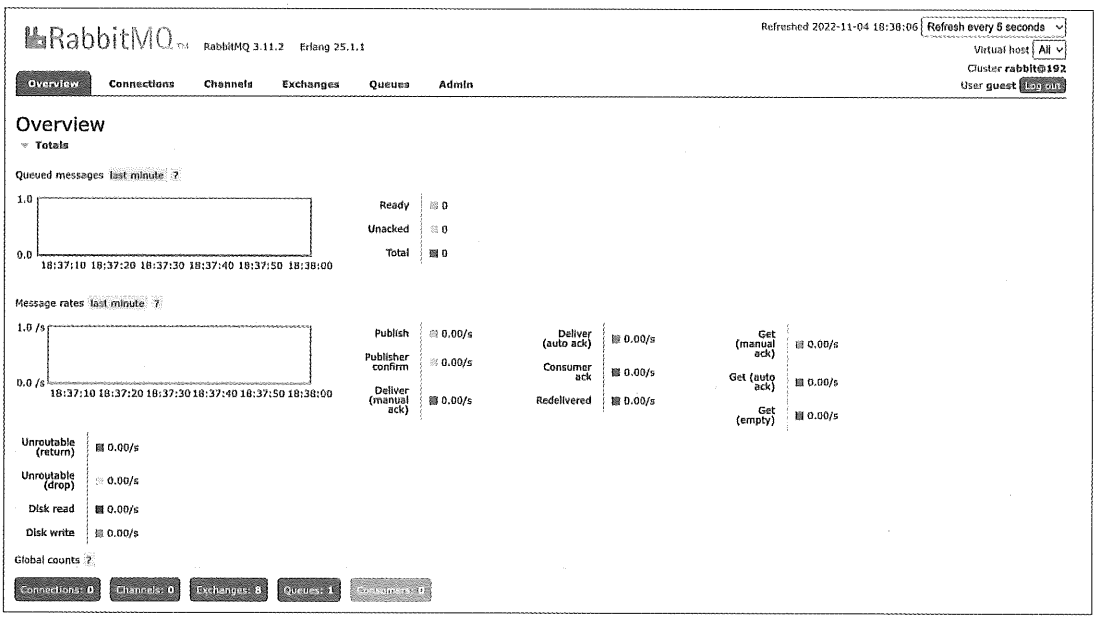
安装好 RabbitMQ 之后，还需要安装一个 Web 控制台，Web 控制台提供了各种可视化的操作管理，有助于运维和排查问题，这需要用到 RabbitMQ 管理插件，它默认包含在 RabbitMQ 发行版中，但必须先启用才能使用，参考命令如下：

```
$ rabbitmq-plugins enable rabbitmq_management
```

启用 Web 控制台之后，访问 http://xxxx:15672/ 控制台页面，如下图所示。



输入默认的游客账号和密码（guest/guest）即可登录，登录后进入控制台首页，如下图所示。



4）添加远程用户

默认的 guest 用户仅限于 localhost 本机访问，后续与 Spring Boot 集成就不行了，所以需要添加一个用户，这里先添加一个超级管理员。

添加账号，参考命令如下：

```
$ ./rabbitmqctl add_user admin 123456
```

添加访问权限，参考命令如下：

```
$ ./rabbitmqctl set_permissions -p "/" admin ".*" ".*" ".*"
```

设置超级权限，参考命令如下：

```
$ ./rabbitmqctl set_user_tags admin administrator
```

设置之后就可以通过 admin 用户登录控制台和远程连接了。

3. Spring Boot 集成 RabbitMQ

Spring Boot 提供了 spring-boot-starter-amqp 一站式启动器，如下面的依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了 RabbitMQ 的自动配置类 RabbitAutoConfiguration，对应的参数绑定类为 RabbitProperties，只需配置一系列 spring.rabbitmq.* 参数就可以完成自动配置。

连接配置示例如下：

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: admin
    password: 123456
```

还可以像数据库数据源一样通过一个地址参数直连，如以下配置示例所示。

```
spring:
  rabbitmq:
    addresses: "amqp://admin:123456@localhost"
```

更多的配置参数可以查看 RabbitMQ 对应的参数绑定类 RabbitProperties。

这里创建一个发送消息接口，以及接收消息的队列监听方法，在发送消息时可以直接注入 RabbitTemplate 模板并使用，然后通过 @RabbitListener 注解接收消息，如以下示例所示。

```
@RequiredArgsConstructor
@RestController
@Slf4j
public class MsgController {

    private final RabbitTemplate rabbitTemplate;

    /**
     * 发送 Direct 模式消息
     * @param msg
     * @return
     */
    @RequestMapping("/send")
    public String sendMsg(@RequestParam("msg") String msg) {
        rabbitTemplate.convertAndSend("test-direct-exchange",
                "test-direct-routing-key", msg);
        return "已发送";
    }

    /**
     * 接收 Direct 模式消息
     * @param msg
     */
    @RabbitListener(queues = "test-direct-queue")
    public void receiveMsg(String msg) {
        log.info("收到 RabbitMQ 消息：{}", msg);
    }

}
```

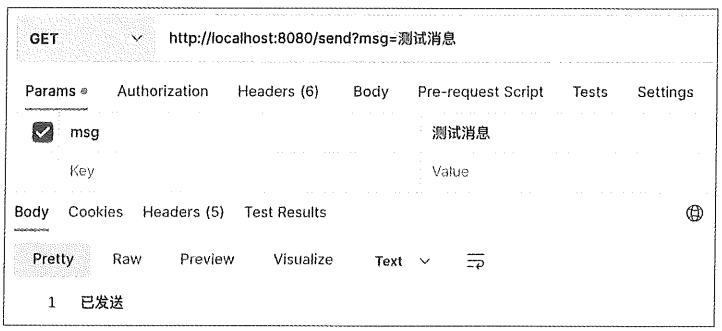
RabbitMQ 不能自动创建对应的队列、交换器，以及队列与交换器的绑定，它们可以在 RabbitMQ 控制台页面中创建，也可以通过代码的方式创建，比如以下配置示例：

```
@Configuration
public class RabbitMQConfig {

    /**
     * 创建 Direct 模式队列
```

```
     * @return
     */
    @Bean
    public Queue testDirectQueue() {
        return new Queue("test-direct-queue");
    }

    /**
     * 创建 Direct 模式交换器
     * @return
     */
    @Bean
    public DirectExchange TestDirectExchange() {
        return new DirectExchange("test-direct-exchange");
    }

    /**
     * 创建 Direct 队列与交换器绑定
     * @param testDirectQueue
     * @return
     */
    @Bean
    public Binding testDirectBinding(Queue testDirectQueue) {
        return BindingBuilder.bind(testDirectQueue)
                .to(TestDirectExchange()).with("test-direct-routing-key");
    }

}
```

上面创建的是 Direct 直连类型的交换器，即根据消息携带的路由键（routing key）将消息投递到与之关联的队列中。

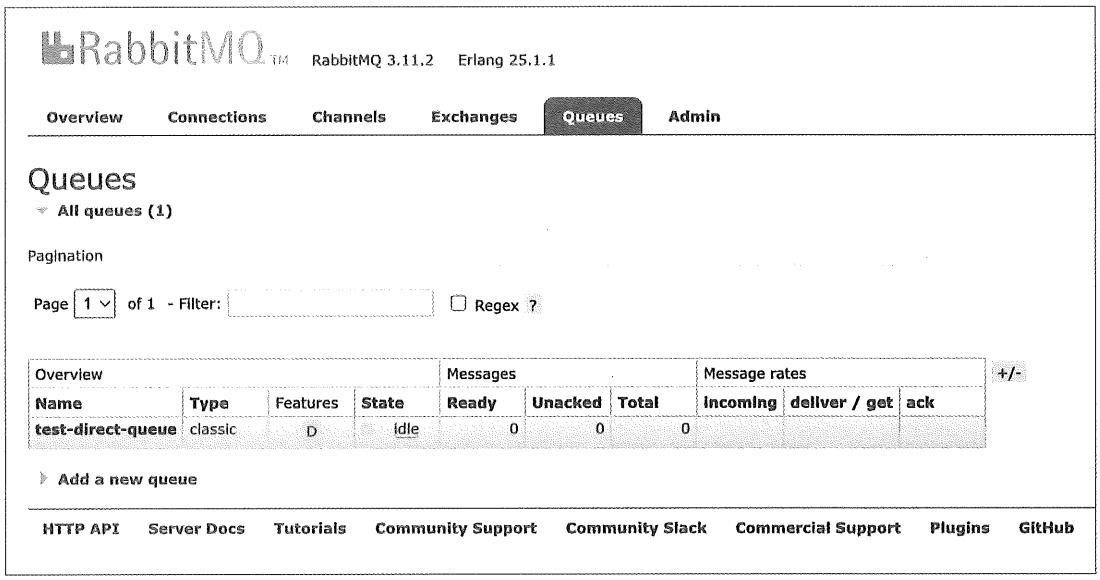
调用发送消息接口进行测试，如下图所示。

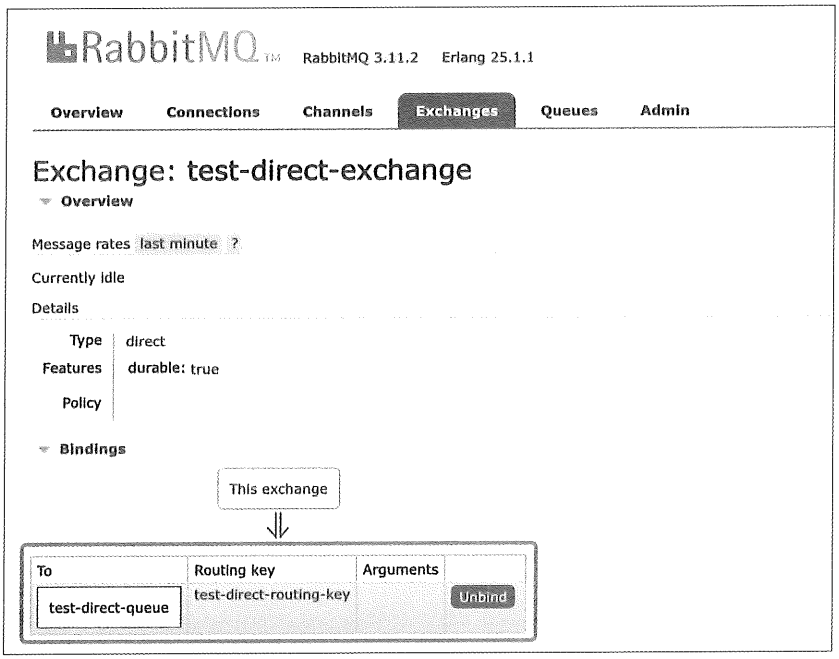


结果如下图所示。

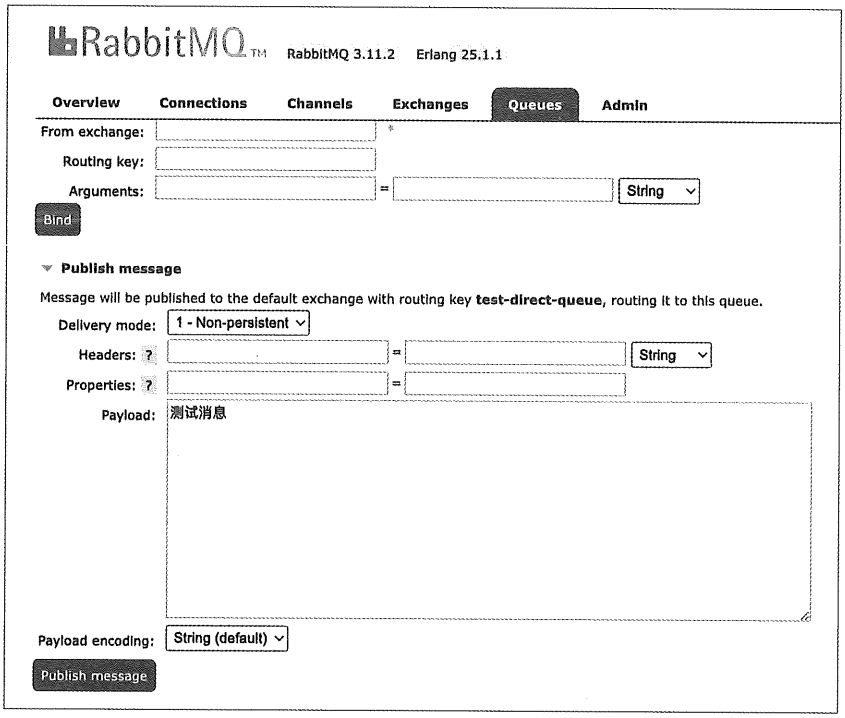
```
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) wi
o.s.a.r.c.CachingConnectionFactory       : Attempting to connect to: [localhost:5672
o.s.a.r.c.CachingConnectionFactory       : Created new connection: rabbitConnectionF
c.j.springboot.rabbitmq.Application      : Started Application in 2.139 seconds (pro
o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'di
o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
o.s.web.servlet.DispatcherServlet        : Completed initialization in 2 ms
c.j.springboot.rabbitmq.MsgController    : 收到 RabbitMQ 消息：测试消息
```

从日志可以看到消息成功发送并被消费了，再回到RabbitMQ控制台页面，可以看到创建的队列、交换器等信息，如下面的三个图所示。





在 Queues菜单页上单击 test-direct-queue 项进入该队列，也能进行交换器和路由键的绑定、手动发送消息等操作，如下图所示。



以上只是介绍了如何使用 Spring Boot 集成 RabbitMQ，以及简单的使用方式，更多的高级用法可以参考其自动配置类进行扩展。

9.2.4 Kafka

1. 概述

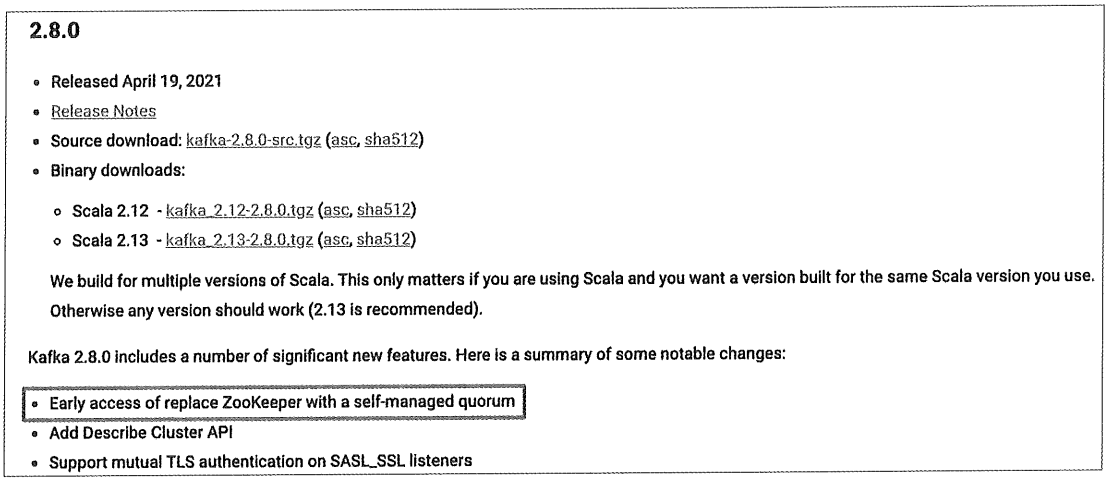
Kafka 是一个高吞吐量、分布式的开源消息系统，它更是一个开源的分布式事件流平台，它可以进行及时、海量的流数据处理，现在已经被广泛用于高性能的数据管道、数据流分析、数据集成等负责核心任务的应用。

常见的 Kafka 的应用场景如下：

- 消息队列（MQ）。
- 流处理。
- 日志收集。
- 用户活动跟踪。
- 指标收集。

Kafka 最初由 LinkedIn 公司开发，后来捐献给了 Apache 软件基金会并成为顶级项目。Kafka 是由 Scala 和 Java 语言编写的，它们都是 JVM 系语言，所以 Kafka 编译后的源代码就是 .class 文件。

Kafka 2.8.0 中已经有了对 ZooKeeper 的早期替换方案，如下图的官方版本说明所示。



但目前最新的 Kafka 3.3.1 中还没有彻底移除 ZooKeeper，还可以使用 ZooKeeper 来管理 Kafka 集群，同时还可以使用最新的 KRaft 来管理 Kafka 集群，并且新增了一个 @metadata 的内部 Topic 主题来存储元数据信息。在后续的版本中，Kafka 会彻底移除对 ZooKeeper 的依赖。

移除 ZooKeeper 的原因可能有以下两点：

（1）Kafka 需要维护另外一个中间件，增加了额外的运维成本，完全没有必要。

（2）依赖 ZooKeeper 使得 Kafka 显得比较笨重，移除 ZooKeeper 依赖可以促使自己变得更轻量化，带来更优的性能、

2. Kafka 环境搭建

1）安装 Kafka

首先在官网下载 Kafka，下载地址为链接 12。

Kafka 几乎支持所有主流的操作系统，包括 Docker 镜像，这里以 CentOS 为例进行安装演示。

下载之后解压即可，参考命令如下：

```
$ tar -xzf kafka_2.13-3.3.1.tgz
$ cd kafka_2.13-3.3.1
```

2）启动 Kafka

Kafka 启动运行需要依赖 Java 8+。

虽然 Apache Kafka 准备移除 ZooKeeper，但目前（Kafka 3.3.1）还是没有正式移除，现在还支持使用 ZooKeeper 或 KRaft 的方式启动 Kafka，二者选其一。

以 ZooKeeper 方式启动 Kafka

先启动 ZooKeeper 服务，参考命令如下：

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties &
```

再启动 Kafka 服务，参考命令如下：

```
$ bin/kafka-server-start.sh config/server.properties &
```

以 KRaft 方式启动 Kafka

生成一个 Cluster UUID，参考命令如下：

```
$ KAFKA_CLUSTER_ID=" $(bin/kafka-storage.sh random-uuid)"
```

格式化日志目录，参考命令如下：

```
$ bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
```

启动 Kafka 服务，参考命令如下：

```
$ bin/kafka-server-start.sh config/kraft/server.properties &
```

3）发送 / 消费 Kafka 消息

成功启动 Kafka 服务后，就有了一个基本的 Kafka 运行环境并可以使用了，下面使用 Kafka 命令行工具来进行简单的发送 / 消费消息的测试。

首先创建一个 spring-boot-test 测试主题，参考命令如下：

```
$ bin/kafka-topics.sh --create --topic spring-boot-test --bootstrap-server localhost:9092
```

然后启动一个生产者客户端发送消息，参考命令如下：

```
$ bin/kafka-console-producer.sh --topic spring-boot-test --bootstrap-server localhost:9092
> 测试消息 1
> 测试消息 2
> 结束
>^C%
```

每一行输入结束后，再按回车键，即表示发送多条独立的消息。

然后新开一个命令行窗口，启动一个消费者客户端消费消息，参考命令如下：

```
$ bin/kafka-console-consumer.sh --topic spring-boot-test --from-beginning --bootstrap-server localhost:9092
测试消息 1
测试消息 2
结束
```

按 Ctrl+C 键可以结束运行生产者和消费者客户端，只要生产者和消费者客户端没有被关闭，就可以一直进行发送消息和消费消息的测试。

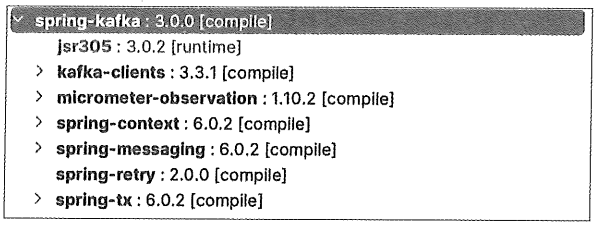
消息一旦生产就被持久化了，消费者可以一直进行消费。Kafka 中有一个偏移量的概念，每个消费者会保存消费的偏移量值，用来标识它消费的位置，当消费消息的时候，偏移量会线性增加。偏移量由消费者进行控制，消费者可以将偏移量重置到之前的位置以重新消费消息，也可以跳过一些消息消费。

3. Spring Boot 集成 Kafka

前面介绍了 Kafka 的自动配置是由 spring-kafka 项目提供的，Spring Boot 并没有再提供专用的 Starter 启动器，Spring Boot 集成 Kafka 时只需要引入 spring-kafka 项目依赖，如以下依赖配置所示。

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

spring-kafka 项目依赖主要包含了 Spring 消息、事务、重试模块，以及 Kafka 客户端等，如下图所示。



在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中注册了 Kafka 的自动配置类 KafkaAutoConfiguration，对应的参数绑定类为 KafkaProperties，只需配置一系列 spring.kafka.* 参数就可以完成自动配置。

连接配置示例如下所示。

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: testGroup
```

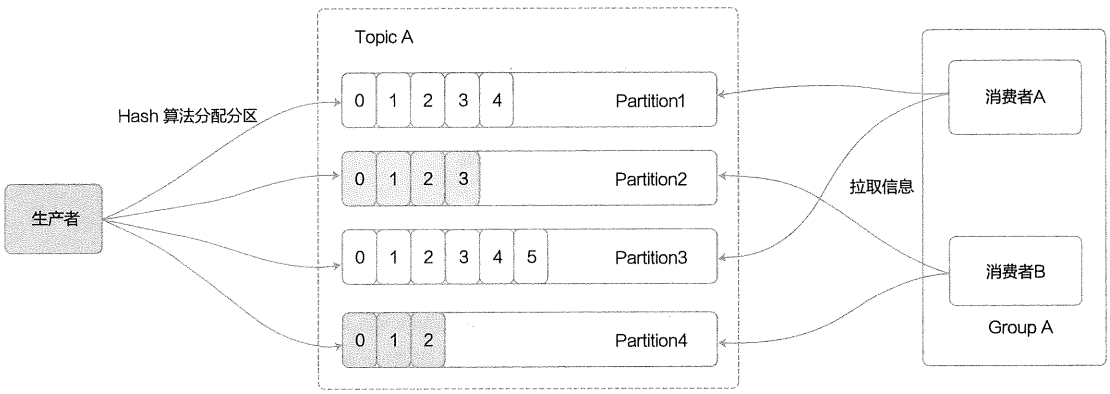
更多的配置参数可以查看 Kafka 对应的参数绑定类 KafkaProperties。

在集成 Kafka 并发送消息之前需要先创建一个主题，既可以通过之前介绍的 Kafka 命令行创建主题，也可以通过代码的方式创建主题，比如以下配置示例：

```
@Configuration
public class KafkaConfig {

    public static final String SPRING_BOOT_TEST_TOPIC = "spring-boot-test-topic";

    @Bean
    public NewTopic testTopic() {
        return new NewTopic(SPRING_BOOT_TEST_TOPIC, 4, (short) 1);
    }

}
```

Spring Boot 启动时会加载该配置类并创建一个区分数为 4、副本数为 1 的测试主题，如果该主题已经创建过，则会忽略。

Kafka 中的分区是指对 Topic（主题）进行分区，即把一个主题拆分成多个分区，如下图所示。



对 Topic 分区的好处是，每个 Topic 可以不受单台服务器的限制，扩展性非常好，可以实现负载均衡，支持消费者并行消费，提高吞吐量以处理更多的消息。

如上图所示，该主题被分成了 4 个分区，两个不同的生产者客户端向该主题写入消息，如果消息指定了 KEY，则使用 KEY 策略，KEY 相同的消息写入同一个分区，如果消息没有指定 KEY，则默认使用轮询策略，另外还可以使用随机策略，每条消息随机写入不同的分区。

消息只能被写入其中一个分区，每个分区都是有序且顺序不可变的记录集，并且可以持续追加，分区中的每条消息记录都会有唯一的 offset（偏移量）进行区分。一个消费者可以消费多个分区的消息，但一个分区只能被一个消费者组中的一个消费者消费。另外，如果一个 Topic 只有一个分区，那么就只能有一个消费者进行消费。

每个 Topic 可以建立多个分区，而副本是对分区的复制，每个分区可以建立多个副本，可以有多个追随者副本（Follower Replica），但只能有一个领导者副本（Leader Replica）。

需要注意的是：

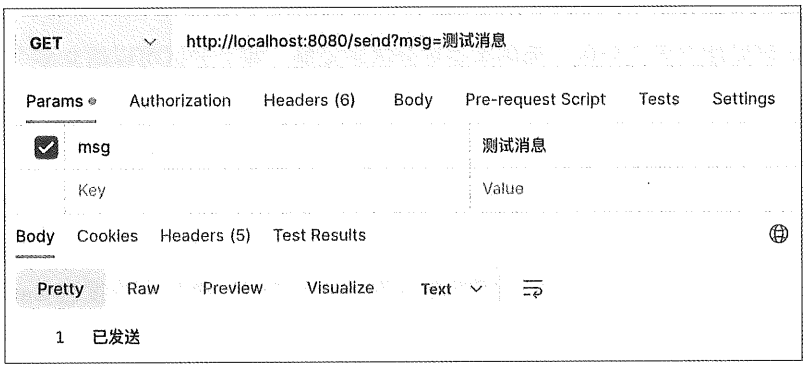
Kafka 不支持减少 Topic 分区数，不然会导致数据丢失，但可以通过删除原来的 Topic，然后创建新的 Topic 的方式重新调整分区数。

这里创建一个发送消息接口，以及消费指定主题的监听方法，在应用中发送 Kafka 消息时可以直接注入 KafkaTemplate 模板并使用，然后通过 @KafkaListener 注解监听对应的主题即可消费消息，如以下示例所示。

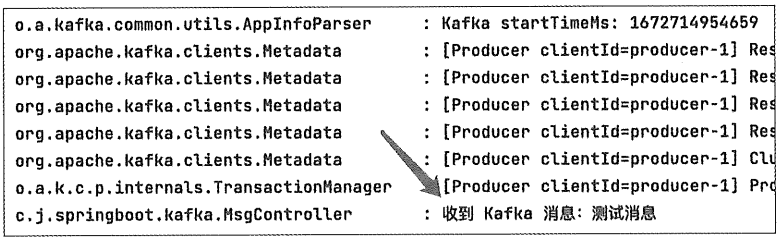
```
@RequiredArgsConstructor
@RestController
@Slf4j
public class MsgController {

    private final KafkaTemplate kafkaTemplate;

    /**
     * 发送消息
     * @param msg
     * @return
     */
    @RequestMapping("/send")
    public String sendMsg(@RequestParam("msg") String msg) {
        kafkaTemplate.send(KafkaConfig.SPRING_BOOT_TEST_TOPIC, "test-key", msg);
        return "已发送";
    }

    /**
     * 接收消息
     * @param msg
     */
```

```
    @KafkaListener(topics = KafkaConfig.SPRING_BOOT_TEST_TOPIC)
    public void receiveMsg(String msg) {
        log.info("收到 Kafka 消息: {}", msg);
    }

}
```

调用发送消息接口进行测试，如下图所示。



结果如下图所示。



从日志可以看到消息成功发送并被消费了。

以上只是介绍了如何使用 Spring Boot 集成 Kafka，以及简单的使用方式，更多的高级用法可以参考其自动配置类并进行扩展。

第 10 章 Spring Boot 调试与单元测试

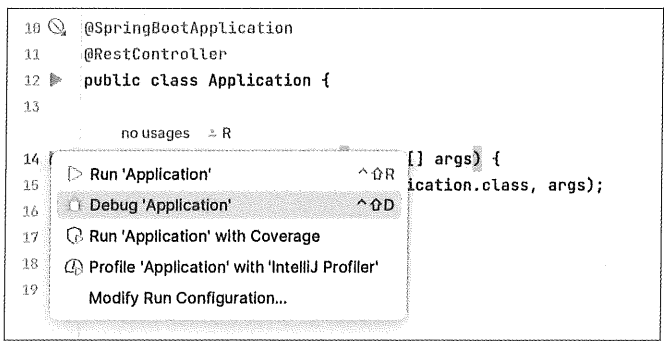
任何项目在上线之前都必须经过本地开发、调试、测试阶段，本章将介绍如何在 Spring Boot 项目中进行断点跟踪调试，这和传统的项目略有区别。本章还将介绍 Spring Boot 特有的开发者工具，它可以帮助开发者在开发阶段调试应用，帮助开发者大大提升开发效率，最后介绍 Spring Boot 如何正确地进行单元测试。

10.1 断点调试

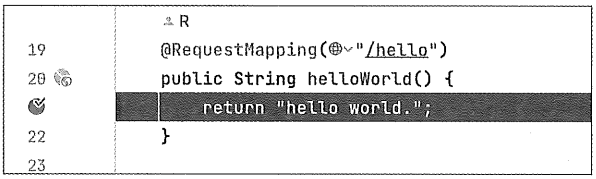
10.1.1 使用 main 方法启动调试

根据 Spring Boot 项目不同的启动方式，断点调试的方式也不一样。

如果直接使用 main 方法启动应用，那么选择 Debug 模式启动即可，如下图所示。



当代码运行到断点处的时候就可以进行调试了，如下图所示。



10.1.2 使用 Maven 插件启动调试

使用 main 方法启动调试有一定的缺陷，因为启动应用时不能使用 Maven 的相关插件，比如不能使用 Maven 资源插件过滤配置资源，导致无法过滤参数占位符而应用启动失败，所以实际项目中使用 Spring Boot Maven 插件启动应用的方式较多。

使用 Spring Boot Maven 插件启动应用，它会“fork”一个子进程来运行应用，所以它不能像 main 方法那样直接“debug”启动调试，而需要像调试远程应用一样调试，断点调试起来略显麻烦。首先需要在Spring Boot Maven插件中配置jvmArguments调试参数启动应用，然后新建一个远程调试。

1. 启用调试模式

启用调试模式有以下两种方式：

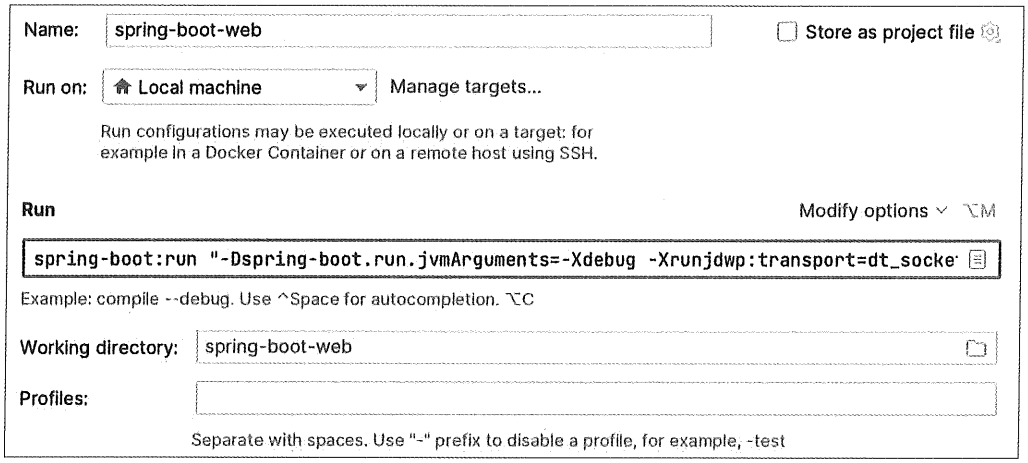
（1）在插件配置中指定调试模式的 jvmArguments 参数，如以下配置示例所示。

```
<project>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <jvmArguments>
                        -Xdebug -Xrunjdwp:transport=dt_socket,server=y,suspend=y,address=5005
                    </jvmArguments>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

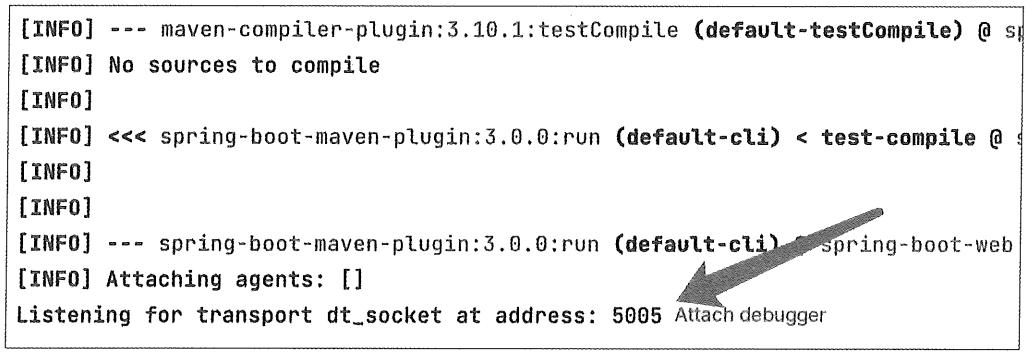
（2）使用 mvn 命令启动时，指定调试模式的 jvmArguments 参数，完整运行命令如下所示。

```
$ mvn spring-boot:run -Dspring-boot.run.jvmArguments="-Xdebug -Xrunjdwp:transport=dt_socket, ser
ver=y,suspend=y,address=5005"
```

主流的开发工具都集成了 Maven 插件，无须 mvn 关键字，否则可能运行出错，如下图所示。



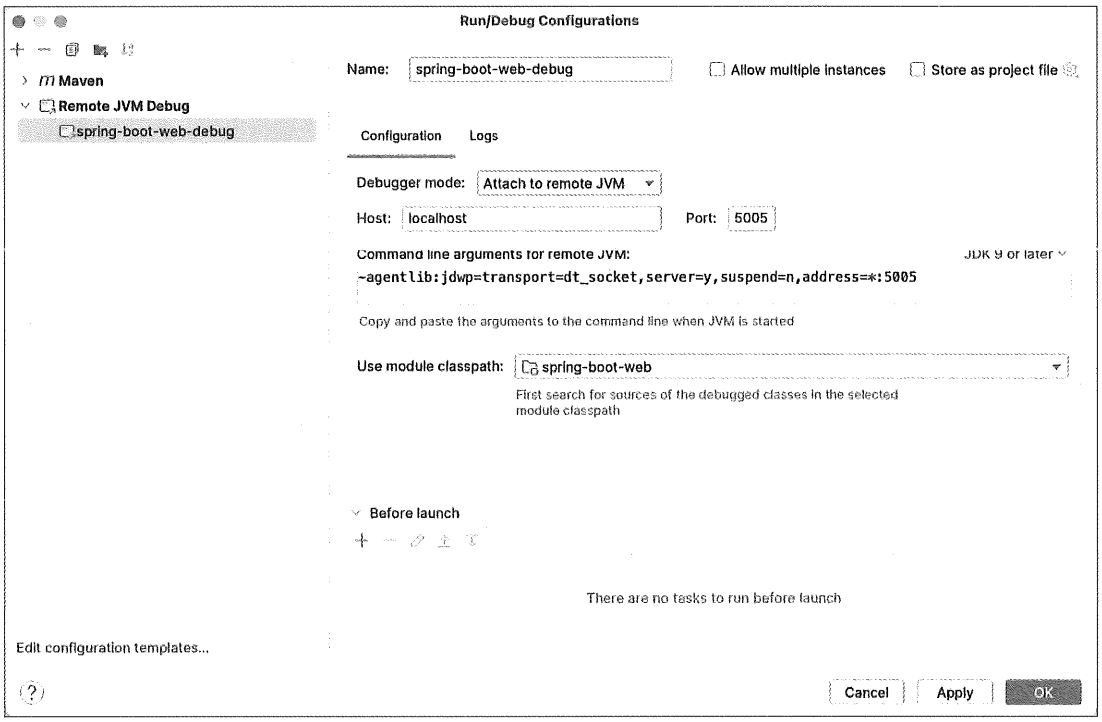
以上创建了一个 spring-boot-web 启动项，然后使用正常运行模式运行即可，应用会在指定的 5005 端口上监听，如下图所示。



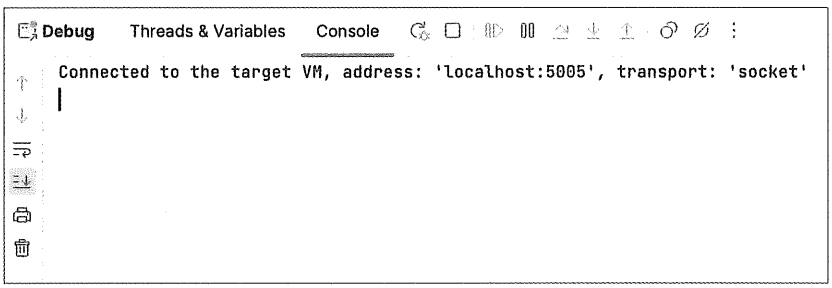
一般建议在命令行中指定参数进入调试模式，只在有需要的时候启用调试。

2. 远程调试

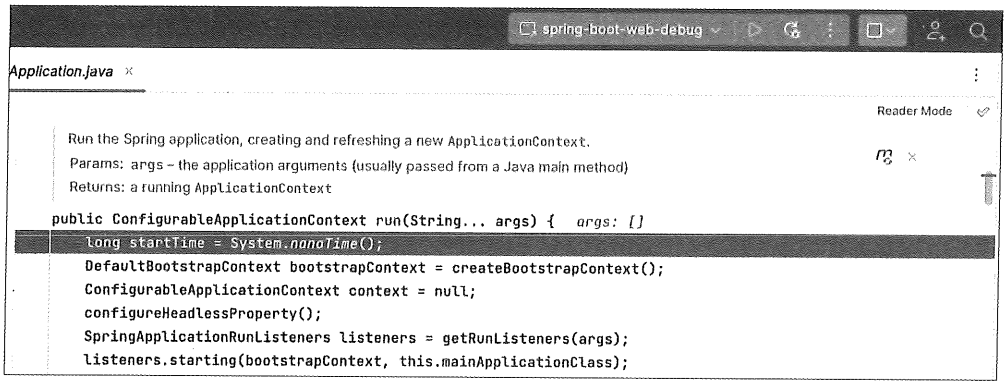
在 IDE中新建一个 Remote 调试任务，主流的 IDE 开发工具都是支持的，如下图所示。



为了与上一步的 spring-boot-web 启动项进行区分，这里将对应的远程调试命名为 spring-boot-web-debug，一般只需要确定 Host 和 Port 参数，然后使用 Debug 模式启动该 Remote 调试任务，Debug 控制台会显示已经连接成功的日志，如下图所示。



一旦连接成功，上一步的 spring-boot-web 启动项才会继续完成启动，之后就可以打断点调试了，如下图所示。



所以，Spring Boot Maven插件启动的应用，断点调试都需要经过这两次启动操作才行，略显麻烦。

10.2 开发者工具

10.2.1 概述

Spring Boot提供了一系列开发者工具，主要用于应用开发阶段，作用包括自动禁用缓存、支持应用自动重启、实时重载等，使用开发者工具可以大大提高开发效率。

只需要引入开发者工具依赖即可启用开发者工具，如下所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <optional>true</optional>
</dependency>
```

其中，Optional表示该依赖只能在本项目及其子项目中传递，而不会传递到引用该项目的项目中，其他项目需要主动引用该依赖才行。

开发者工具是在开发阶段使用的，所以，打包应用时，Spring Boot默认不会把开发者工具打包进去，如果有需要，也可以通过配置把开发者工具打进包中，如以下配置所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludeDevtools>false</excludeDevtools>
            </configuration>
        </plugin>
    </plugins>
</build>
```

其中，excludeDevtools 参数设置为 false 表示不排除开发者工具依赖。另外，因为开发者工具是开发阶段才使用的工具，所以使用 java -jar 命令运行整包时，开发者工具会被自动禁用，虽然可以通过 -Dspring.devtools.restart.enabled=true 参数来启用生产环境中的开发者工具，但不建议这样做，有安全风险，建议不要在生产环境中包含开发者工具。

10.2.2 默认值

下表是 Spring Boot 开发者工具使用的参数默认值。

参数名称	默认值
server.error.include-binding-errors	always
server.error.include-message	always
server.error.include-stacktrace	always
server.servlet.jsp.init-parameters.development	true
server.servlet.session.persistent	true
spring.freemarker.cache	false
spring.graphql.graphiql.enabled	true
spring.groovy.template.cache	false
spring.h2.console.enabled	true
spring.mustache.servlet.cache	false
spring.mvc.log-resolved-exception	true
spring.reactor.debug	true
spring.template.provider.cache	false
spring.thymeleaf.cache	false
spring.web.resources.cache.period	0
spring.web.resources.chain.cache	false

这里以缓存项为例说明为什么需要默认值。Spring Boot 的某些功能组件可能会提供缓存，这在生产环境中肯定是能提升性能的，但在开发阶段可能会因为没有及时生效而排查困难，所以 Spring Boot 开发者工具默认是禁用所有缓存项的。

如果不想让应用使用这些开发者工具带来的默认参数值，则可以通过指定参数来禁用默认值，如以下配置示例所示。

```
spring:
  devtools:
    add-properties: false
```

10.2.3 自动重启

1. 概述

Spring Boot 开发者工具会监控类路径下的文件，一旦有文件发生变更，应用就能自动快速重启，而不需要手动重启应用。自动重启技术使用了两个类加载器：

- **基础类加载器**：用于加载不会变更的文件，比如第三方的 jar 包。
- **重启类加载器**：用于加载应用类文件。

这样做的好处是每次重启只需要创建一个重启类加载器即可，基础类加载器已加载好的不用动，虽然不能和 JRebel 热部署方式相比，但和冷启动相比，自动重启技术拥有更高的启动效率。

需要注意的是：

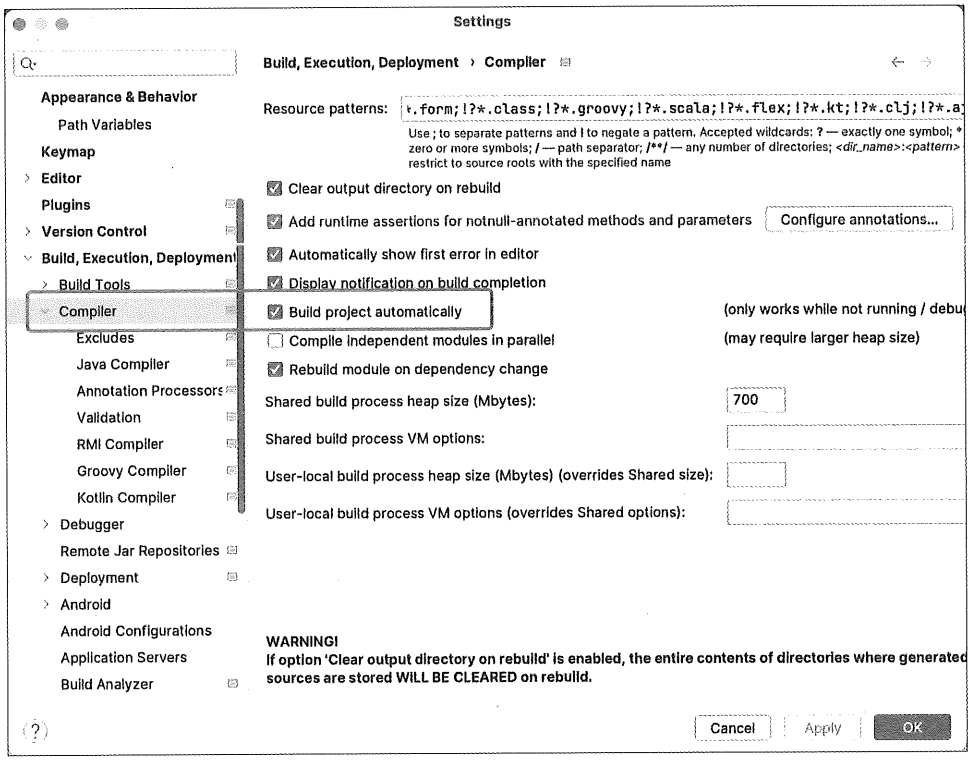
使用 AspectJ 切面编程时不支持自动重启。

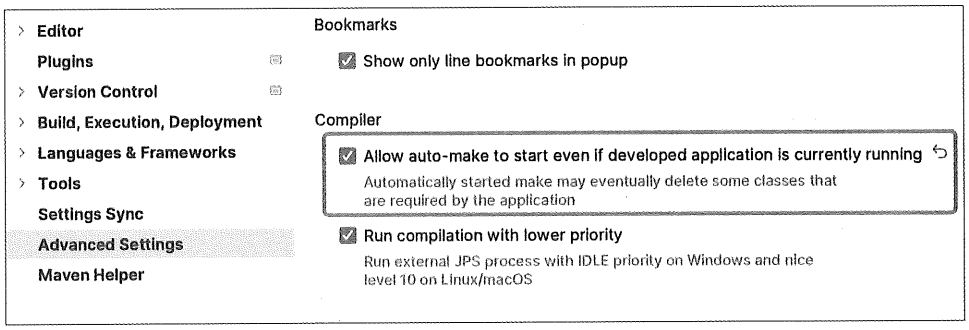
2. 触发机制

自动重启触发机制如下：

- Eclipse：修改保存就能自动重启。
- IntelliJ IDEA：需要重新构建（Build）项目才能自动重启。
- **构建工具**：在 IDE 构建插件中使用 mvn compile/gradle build 命令也能触发重启。

IntelliJ IDEA 默认是手动构建应用的，当然也可以开启自动构建应用，如下面两个图所示（IntelliJ IDEA 的版本为 2022.3）。





IDE 开发工具每次修改文件之后都会自动编译并重启，一方面会频繁消耗资源，另一方面可能不符合开发者的要求，比如，开发者想在修改了一定量文件之后再决定是否自动重启，这时可以把 IDE 修改为手动编译模式，或者指定一个触发文件，只有该触发文件被修改之后应用才自动重启。

配置示例如下所示。

```
spring:
  devtools:
```